BUILDING PRODUCTION-READY REST APIS

WITH PYTHON



LOGISTICS



Class Hours:

- Instructor will set class start and end times.
- There will be regular breaks in class.



Telecommunication:

- Turn off or set electronic devices to silent (not vibrate)
- Reading or attending to devices can be distracting to other students
- Try to delay until breaks or after class

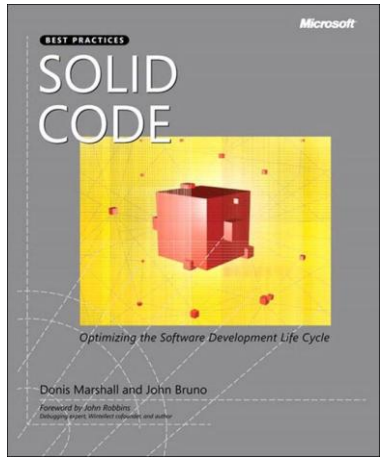
Miscellaneous:

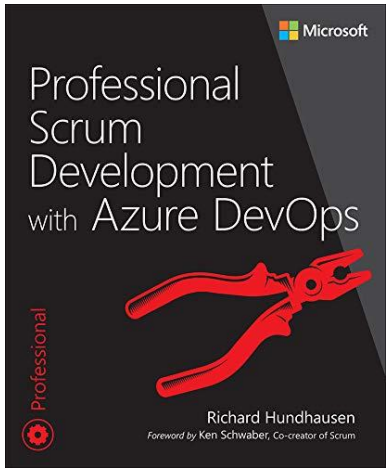
- Courseware
- Bathroom
- Fire drills

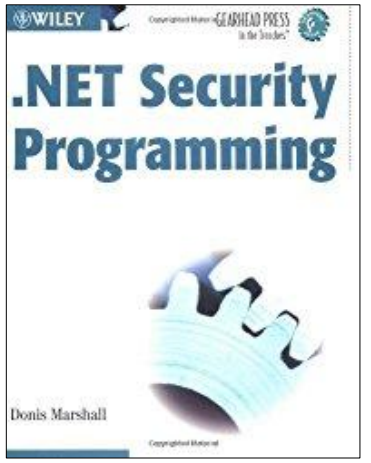
© 2026 Innovation In Software Corporation

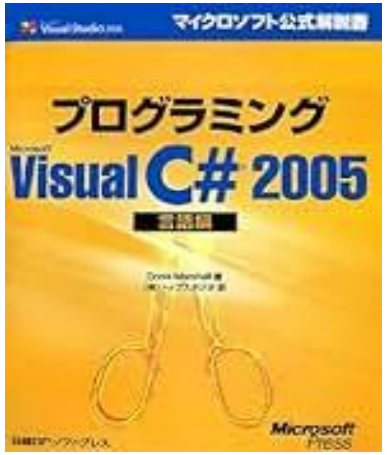
DONIS MARSHALL

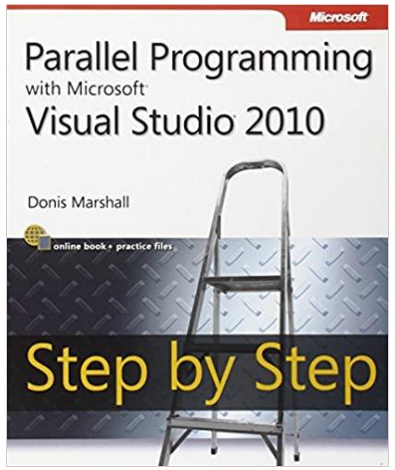
Microsoft MVP
Microsoft Certified
Python Programmer
Enterprise Architect
Author

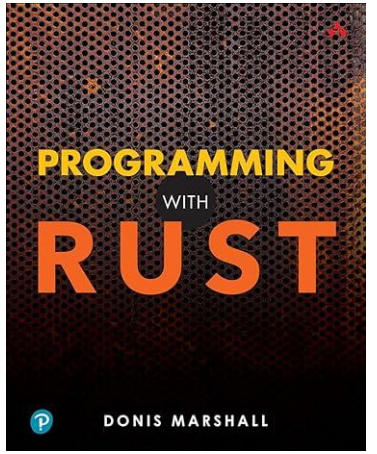












© 2026 Innovation In Software Corporation

INTRODUCE YOURSELF

Time to introduce yourself:

- Name
- What is your role in the organization
- Indicate API experience



© 2026 Innovation In Software Corporation

FASTAPI



FastAPI is a modern, high-performance Python web framework for building RESTful APIs and backend services. It is based on standard Python type hints, making applications easier to develop, validate, and maintain while providing automatic request validation and interactive API documentation.

It remains one of the most popular Python frameworks for building high-performance APIs due to its speed, developer productivity, and seamless integration with the Python ecosystem.

© 2026 Innovation In Software Corporation

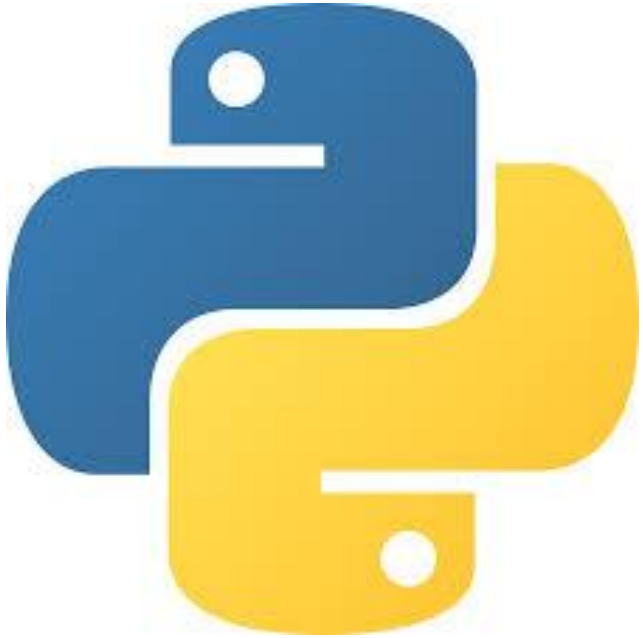
CONTRACT FIRST

Contract-First Design defines the API before writing code. The API contract, created with the OpenAPI Specification, describes the endpoints, requests, responses, and data formats. It serves as a blueprint that both frontend and backend developers follow.

With FastAPI, developers implement the API to match the OpenAPI contract. FastAPI validates requests, generates interactive API documentation, and helps ensure the implementation follows the defined contract, reducing integration errors and simplifying development.

© 2026 Innovation In Software Corporation

PYTHON



Python is a high-level, general-purpose programming language known for its simple, readable syntax. It is easy to learn, highly productive, and widely used by both beginners and experienced developers.

Python is commonly used for web development, automation, data science, artificial intelligence, machine learning, cybersecurity, cloud computing, and scripting. Its extensive libraries and strong community make it one of the world's most popular programming languages.

© 2026 Innovation In Software Corporation

TRUNK-BASED DEVELOPMENT (TBD)

Trunk-Based Development is a source control approach in which developers store their code in a shared repository and regularly merge changes into a single main branch, often called the main trunk. Instead of working on long-lived branches, developers make small, frequent changes that are easier to review, test, and integrate.

For a FastAPI project, your API code is stored in a repository so it can be shared, backed up, and tracked over time. As you add features or fix bugs, you commit your changes and merge them into the main branch, allowing the team to work from the latest version while reducing merge conflicts.

© 2026 Innovation In Software Corporation

SECURITY



FastAPI includes built-in support for implementing authentication and authorization using common security standards such as OAuth 2.x, JWT (JSON Web Tokens), and API keys. These features help protect APIs from unauthorized access, ensuring that only authenticated users with the appropriate permissions can use sensitive application functionality or data.

© 2026 Innovation In Software Corporation

STRIDE

Explain the acronyms of STRIDE?





5 MINUTES

© 2026 Innovation In Software Corporation

STRIDE

- **Spoofing**: Impersonating a user or system to gain unauthorized access.
- **Tampering**: Unauthorized modification of data, code, or system resources.
- **Repudiation**: Denying an action because no audit trail exists.
- **Information Disclosure**: Exposing sensitive information to unauthorized users.
- **Denial of Service (DoS)**: Preventing legitimate users from accessing a system.
- **Elevation of Privilege**: Gaining permissions beyond those originally granted.

© 2026 Innovation In Software Corporation

GENERAL SECURITY



This course focuses on developing APIs with FastAPI and does not provide comprehensive coverage of application security. However, security is a consideration in every application.

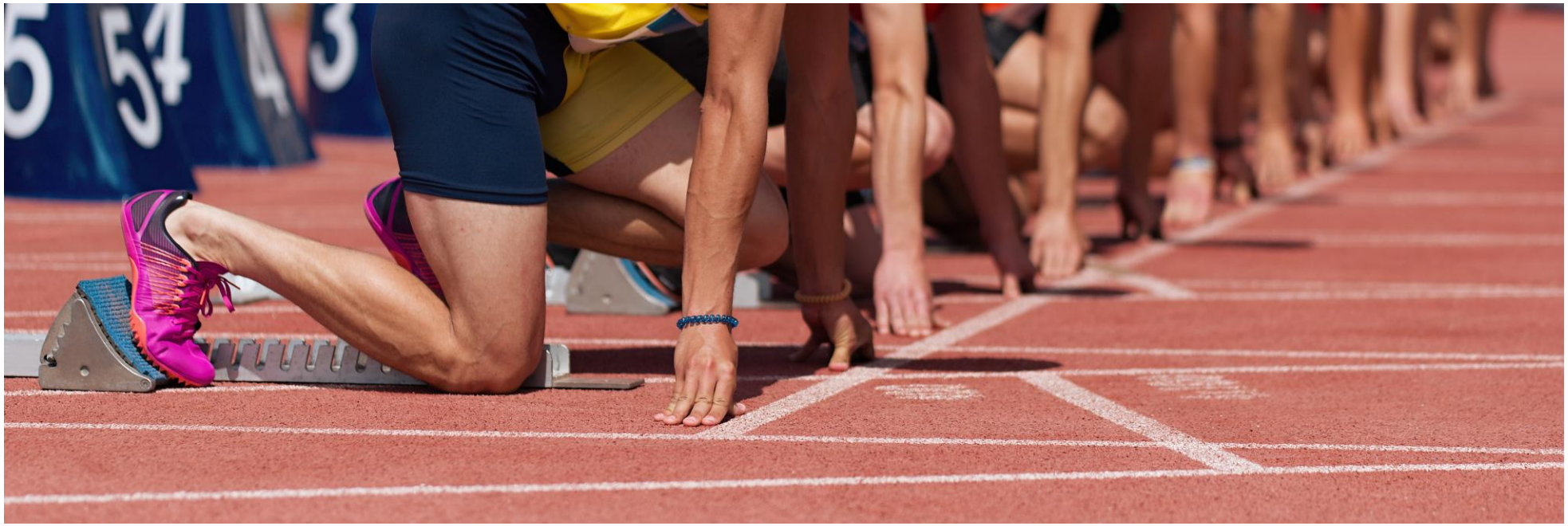
You should follow secure coding practices and industry standards to help prevent common vulnerabilities such as injection, spoofing, and data exposure attacks.

© 2026 Innovation In Software Corporation

PERFORMANCE

API performance is a critical responsibility of every developer. Clients expect APIs to be fast, responsive, and capable of handling increasing workloads efficiently.

Throughout this course, performance is treated as a key design principle, with an emphasis on building FastAPI applications that are scalable, efficient, and optimized for real-world production environments.



© 2026 Innovation In Software Corporation

PERFORMANCE - 2

Here are some of the FastAPI performance features:

- ASGI + async — FastAPI runs on Starlette over ASGI, so async endpoints handle many concurrent I/O-bound requests (DB calls, HTTP calls) on a single worker without blocking, unlike traditional WSGI frameworks.
- Fast serialization — recent versions can use Pydantic v2, whose validation/serialization core is written in Rust, making request parsing and response building significantly faster than pure-Python v1.
- Efficient dependency injection — dependencies are resolved per-request but cached within a request, so a shared dependency isn't recomputed multiple times in one call.
- Automatic response model handling — when you set response_model on an endpoint, FastAPI takes whatever you return (a dict, a Pydantic object, an ORM row) and does two things for you: it filters the output down to only the fields declared in the model, and it serializes that result into JSON. You just return your data and FastAPI handles shaping and encoding it.

© 2026 Innovation In Software Corporation

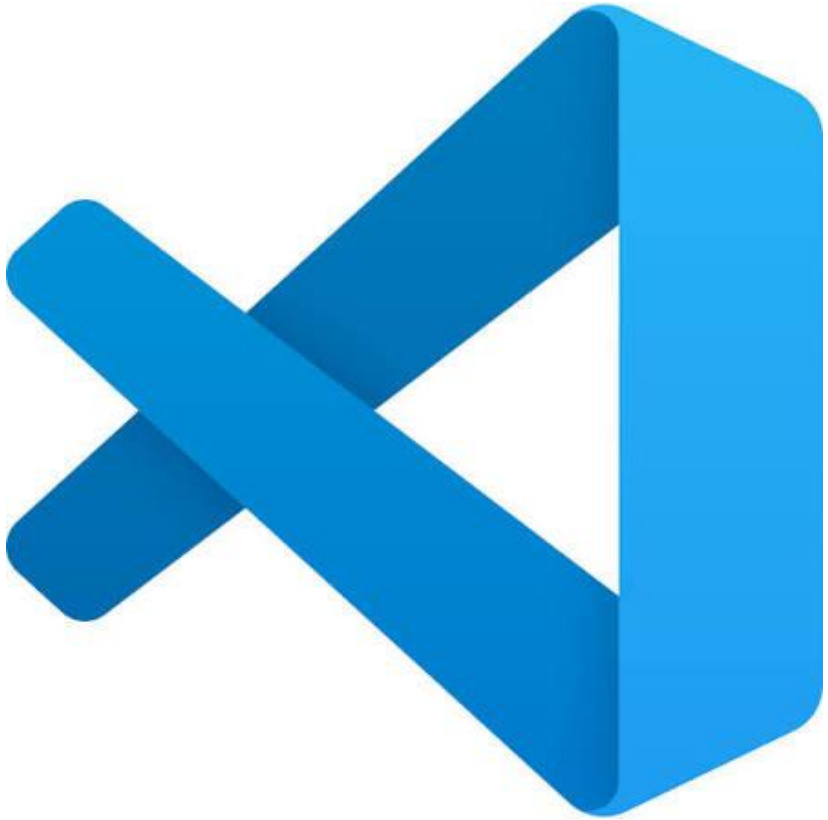
TOOLS AND TECHNOLOGIES

The following tools and technologies are used or referenced throughout this course. While not the primary focus, they are commonly used when developing and testing FastAPI applications.

- **Postman**: A popular tool for testing, debugging, and documenting REST APIs.
- **cURL**: A command-line utility for sending HTTP requests and testing APIs.
- **Starlette**: The high-performance ASGI framework that FastAPI is built upon.
- **Pydantic**: A Python library used by FastAPI for data validation and serialization.
- **Swagger**: An interactive web interface for exploring and testing APIs generated from an OpenAPI specification.

© 2026 Innovation In Software Corporation

VSCODE



Visual Studio Code (VS Code) is the development environment used throughout this course. VS Code is a free, lightweight, and powerful code editor from Microsoft that supports Python, FastAPI, and hundreds of other programming languages through extensions.

You receive features such as syntax highlighting, intelligent code completion, debugging, an integrated terminal, and source control integration.

© 2026 Innovation In Software Corporation

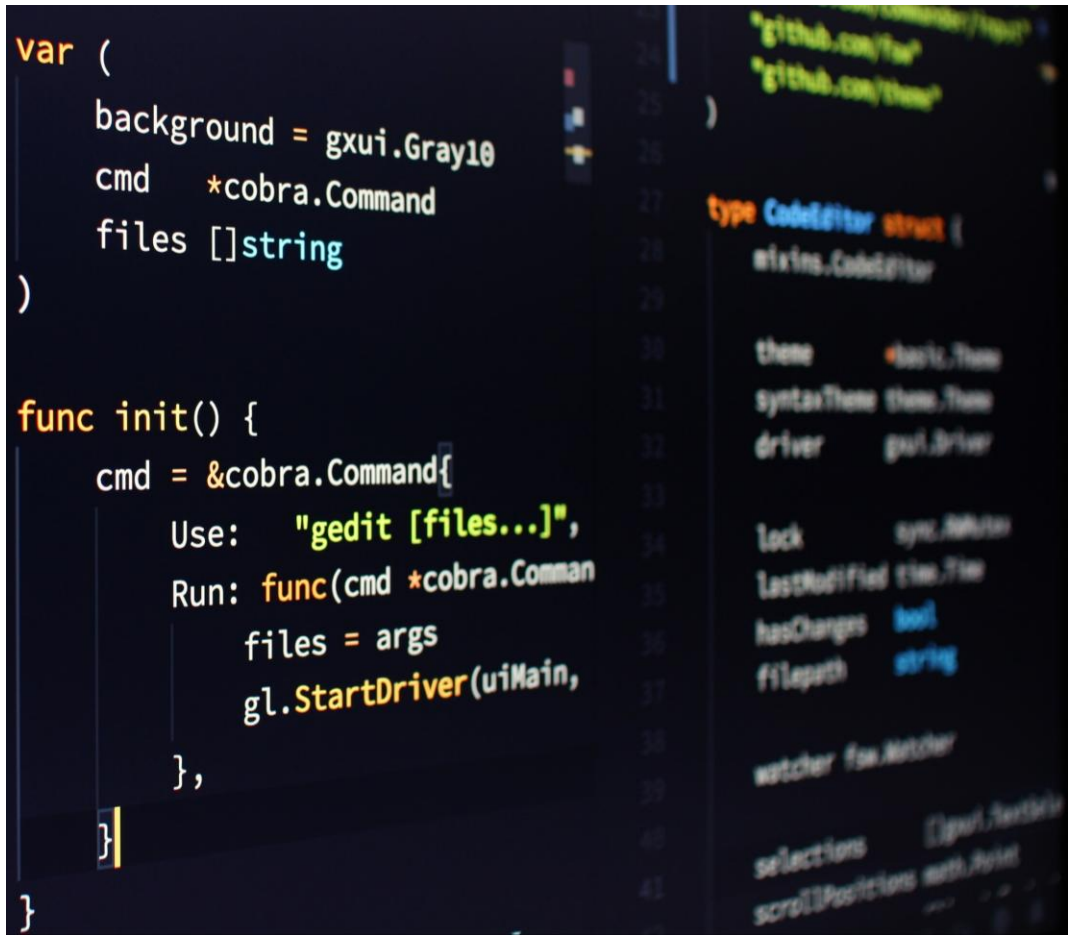
GITHUB COPILOT



GitHub Copilot will not be used in this course to generate coding solutions. The goal is to help you understand how to design, write, debug, and maintain FastAPI applications yourself. Relying on AI-generated code too early can hide important concepts and reduce the overall learning experience. For that reason, GitHub Copilot is not directly included in class.

© 2026 Innovation In Software Corporation

SAMPLE CODE



The sample code presented in this course is intended to illustrate concepts and techniques, not to serve as production-ready software. To keep examples clear and focused, some code samples may omit error handling, panic recovery, input validation, logging, security controls, and other robustness features.

In many cases, brevity and instructional clarity take precedence over completeness.

© 2026 Innovation In Software Corporation

LABS

Learning is better when hands-on. Some of the modules have a companion lab reinforcing a kinesthetic learning experience.

- Labs review and reinforce important concepts
- Don’t be surprised - many of the labs extend the topics introduced in the module
- The labs offer an opportunity for real-world experience
- Pair programming can be effective when working on the labs

© 2026 Innovation In Software Corporation

COURSE AGENDA

1. FastAPI
2. HTTP Headers
3. Contract first
4. Observability
5. Swagger
6. Authentication / Authorization
7. Trunk-Based Development
8. Versioning and Deprecation
9. API Testing
10. Splunk
11. Definition of Done
12. Best practices and patterns

© 2026 Innovation In Software Corporation

YOUR CLASS!

Yes, this is your class. What does this mean? You determine the value.

- What is the most important ingredient of class – your participation!
- Your feedback and questions are always welcomed.
- There is no protocol in class. Speak up anytime!

© 2026 Innovation In Software Corporation

LAB 1 – WEATHER API



FIRST LAB



This lab is your first hands-on example of using an API — one of many throughout this course.

An API (Application Programming Interface) is how one program requests data or services from another. In this lab, your Python program will act as a client: it sends a request over the internet to a public API and gets live weather data back.

© 2026 Innovation In Software Corporation

WEATHER API

A public Weather API is a service, hosted by someone else on the internet, that anyone can send requests to.

- You (the client): ask a question — "What's the weather at these coordinates?"
- The API (the server): sends back an answer as structured data

You don't need to know *how* the weather service works internally — only how to ask correctly and read the reply. That contract is the whole idea behind APIs.

© 2026 Innovation In Software Corporation

CLIENT

Your Python program acts as the client in three steps:

- Pick a city (you supply its coordinates)
- Send a request to the Open-Meteo public weather API
- Read the reply and print the temperature

The hardest part is given — the code that reaches out to the internet. Your mission: write the code that *feeds* it and the code that *shows* the result.

```
Enter a city number (1-50): 1
Weather in Tokyo:
Temperature: 29.4°C
Wind speed: 6.1 km/h
```


© 2026 Innovation In Software Corporation

CLIENT - 2

Every program — and every API call — follows the same rhythm: get input → do the work → show output.

Stage	Who writes it	What it does
Get input	**You**	Read cities, ask user to pick one
Do the work (**the API call**)	**Given**	Client sends request, gets data back
Show output	**You**	Print the temperature

The given code is the actual client-to-API call. You build the beginning and the end around it.

© 2026 Innovation In Software Corporation

REQUESTS TOOL

Install the API library once (it's not built into Python):

```
pip install requests
```

Then the first line of your program loads it. Fill in the blank:

```
_______ requests
```

The keyword that loads outside code is **import**.

© 2026 Innovation In Software Corporation

CITIES

Each line of cities.txt looks like 1,Tokyo,35.6895,139.6917. This loop cuts each line apart and stores it as a dictionary in the cities list.

Complete the blanks:

```
cities = []
with open("cities.txt") as f:
    for line in f:
        number, name, lat, lon = line.strip().split(",")
        cities._____({"number": int(number), "name": name,
                      "lat": _____(lat), "lon": float(lon)})
```

Add to a list with .append(...). Coordinates must be decimals — use float(...). (Text from a file stays text until converted.)

© 2026 Innovation In Software Corporation

USER INPUT

The input(...) function returns text, but you need a number to match.

Fill in the blank:

```
choice = _______(input("Enter a city number (1-50): "))
```

Whole numbers use int(...).

© 2026 Innovation In Software Corporation

FINDING MATCH

Search cities for the one whose number equals choice. Complete the blanks:

```
city = None
for c in cities:
    if c["number"] == _______:       # the user's choice
        city = c
        _______                      # stop once found
```

Exit a loop early with *break*. After this, *city* exists — ready for the API call.

The == operator is "equal?"; a single = is an assignment.

© 2026 Innovation In Software Corporation

API REPLY

The API's reply, data, nests the weather values inside a section called current_weather:

```
data = {
    "current_weather": {          # ← the part we want
        "temperature": 18.2,
        "windspeed": 9.4
    }
}
```

Reach a value from the outside in — inner section first, then the value:

```
data["current_weather"]["temperature"]   # 18.2
```

© 2026 Innovation In Software Corporation

API REPLY - 2

First save the inner section as weather, then read each value from it. Complete the blanks:

```
weather = data["______"]                    # the inner section
print(f"Weather in {city['name']}:")
print(f"Temperature: {weather['______']}°C")
print(f"Wind speed: {weather['windspeed']} km/h")
```

Why save weather first? It's a shortcut. Once weather points at the inner section, you write the short weather['temperature'] instead of the long data['current_weather']['temperature'] every time.

© 2026 Innovation In Software Corporation

TEST

1. Run it and check a known city.

```
python weather.py
Enter a city number (1-50): 1
```

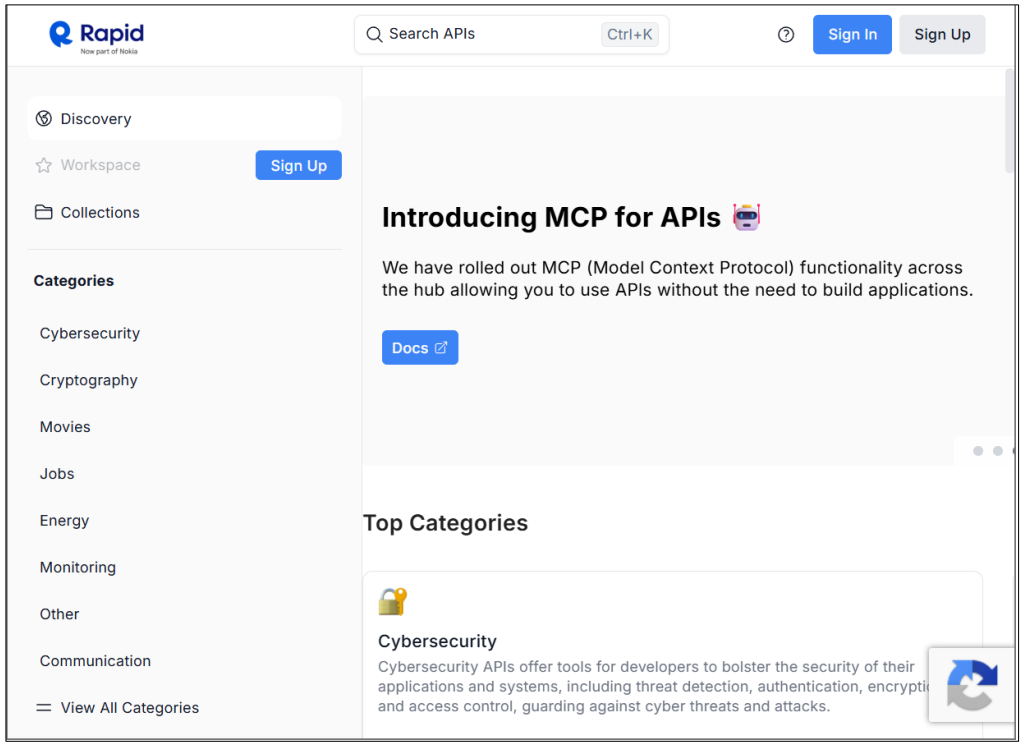
- You should see Tokyo's weather. Getting *any* real temperature back means your whole client-to-API chain works.
- Verify the answer is actually correct.

2. Try a few different numbers.

   Run it again with 11 (New York) and 37 (London). Different cities should give different results — proof that the coordinates are really driving the request.

© 2026 Innovation In Software Corporation

PUBLIC API REPOSITORIES



Public API repositories are online catalogs — such as RapidAPI and the Postman Public API Network — where you can browse, search, and test thousands of ready-to-use APIs through a graphical interface, often with a single account and key.

Go to each and explore!

RapidAPI: https://rapidapi.com/hub

Postman Public API Network: https://www.postman.com/explore

© 2026 Innovation In Software Corporation

Lab completed



PYTHON

VIRTUAL ENVIRONMENTS, DEPENDENCIES, TYPE HINTS, AND MORE



INTRODUCTION



In this module, you will set up the Python groundwork that FastAPI builds on. You will learn to isolate each project in its own virtual environment, manage dependencies with pip, and use type hints to describe your data.

You will then cover async functions for handling work that waits, Pydantic for validating data against a model, and decorators — the mechanism FastAPI uses to turn a plain function into an API route.

© 2026 Innovation In Software Corporation

VIRTUAL ENVIRONMENT



one toolbox per job, not shared

A virtual environment is a private, isolated copy of Python for a single project.

- Instead of one system-wide Python shared by everything, each project gets its own interpreter and its own installed packages.
- Think of it as a separate toolbox per project — what you install for Project A can't affect Project B.
- It keeps your global Python clean and untouched.

© 2026 Innovation In Software Corporation

DEPENDENCIES

Dependencies are the external libraries your project relies on — code others wrote so you don't have to. These libraries *are* your project's dependencies: your code depends on them to run.

You install these libraries with pip, Python's package installer. pip downloads them from PyPI (pypi.org), the public Python Package Index.

```
pip install requests

pip install fastapi
```

Many libraries depend on other libraries — pip installs that whole chain of dependencies for you automatically.

© 2026 Innovation In Software Corporation

DLL HELL



Different projects often need different versions of the same tools (dependency).

Project A needs Requests 2.20 — Project B needs Requests 2.31.

With one shared Python, installing one version breaks the other.

Virtual environments, with different tools, exist side by side, each in its own project.

© 2026 Innovation In Software Corporation

VIRTUAL ENVIRONMENT – COMMON COMMANDS

Six commands cover almost everything you'll do. The normal order:

- create the environment (once per project)
- activate it (every time you work)
- install the libraries you need
- freeze the list so others can rebuild it
- install from the list on another machine (optional)
- deactivate when you're done

© 2026 Innovation In Software Corporation

CREATE VIRTUAL ENVIRONMENT

Move to your project's root directory first:

```
cd path/to/your-project
```

Create the Environment

```
python -m venv .venv
```

Makes a .venv folder holding a fresh, private copy of Python — no extra libraries yet. Run once at project start, not every time.

© 2026 Innovation In Software Corporation

ACTIVATE VIRTUAL ENVIRONMENT

Activate the environment for your terminal session — pick the command that matches your operating system.

```
.venv\Scripts\Activate.ps1
```

- Switches your terminal to use the project's tools instead of the system one.
- You'll see (.venv) appear at the start of your prompt
- Do this every time you open a new terminal to work on the project.

If (.venv) isn't showing, you're not in the environment yet.

© 2026 Innovation In Software Corporation

INSTALL DEPENDENCIES

With the environment active, install the libraries your project needs using pip.

```
pip install requests
pip install fastapi
```

pip downloads the library from PyPI and installs it into your active environment. Install as many as you need — each one becomes a dependency of your project.

Make sure (.venv) is showing first, so it installs into the project, not the global Python environment.

© 2026 Innovation In Software Corporation

SAVE ENVIRONMENT

These two commands let you save your exact setup and rebuild it anywhere.

```
pip freeze > requirements.txt
pip install -r requirements.txt
```

- **pip freeze > requirements.txt** writes every library and its exact version into requirements.txt. The > creates or overwrites the file.
- **pip install -r requirements.txt** reads that file and installs everything at the listed versions.

Re-run freeze whenever you add libraries.

© 2026 Innovation In Software Corporation

EXIT ENVIRONMENT

Leave the Environment

```
deactivate
```

This command exits the virtual environment and returns your terminal and the global environment.

- The (.venv) marker disappears from your prompt.
- Your environment isn't deleted — it's just switched off until you activate again.

To delete an environment entirely, just delete the .venv folder. Rebuild it anytime from requirements.txt.

© 2026 Innovation In Software Corporation

PYTHON INTERPRETERS

Different Python versions can be installed per virtual environment. Each venv will be tied to one specific Python interpreter, so one project can run on 3.11 while another runs on 3.12, and so on.

Key point: A venv does not install Python. It points at a Python interpreter already on your system. To use different versions, install those versions first, then create each venv from the one you want.

- py -3.11 -m venv venv311
- py -3.12 -m venv venv312

© 2026 Innovation In Software Corporation

TYPE HINTS

```
# no hint — just a label
age = 30


# hinted — 'age is an int'
age: int = 30
name: str = "Ada"
```

A hint tells Python what kind of value a variable or argument should be.

- Python doesn't enforce them at runtime — but FastAPI reads them to enforce strong type checking.
- From one hint, FastAPI validates input, converts types, and writes your API docs.
- Syntax: put a colon and the type after the variable name.

© 2026 Innovation In Software Corporation

TYPE HINTS - 2

```
age: int="hello" #runs fine, no error

print(age)
```

Type hints "activate" only when something deliberately reads them:

- Never at runtime (default) — Python stores the hint as metadata and moves on.
- A type checker, before you run — mypy, Pyright, or your editor (VS Code/Pylance), flags mismatches as you write, at compile time.
- A library, at runtime — FastAPI and Pydantic inspect your hints via introspection, then build validation, conversion, and docs.

© 2026 Innovation In Software Corporation

PREVIOUS SLIDE

When does VS Code display a warning with the code on the previous slide?

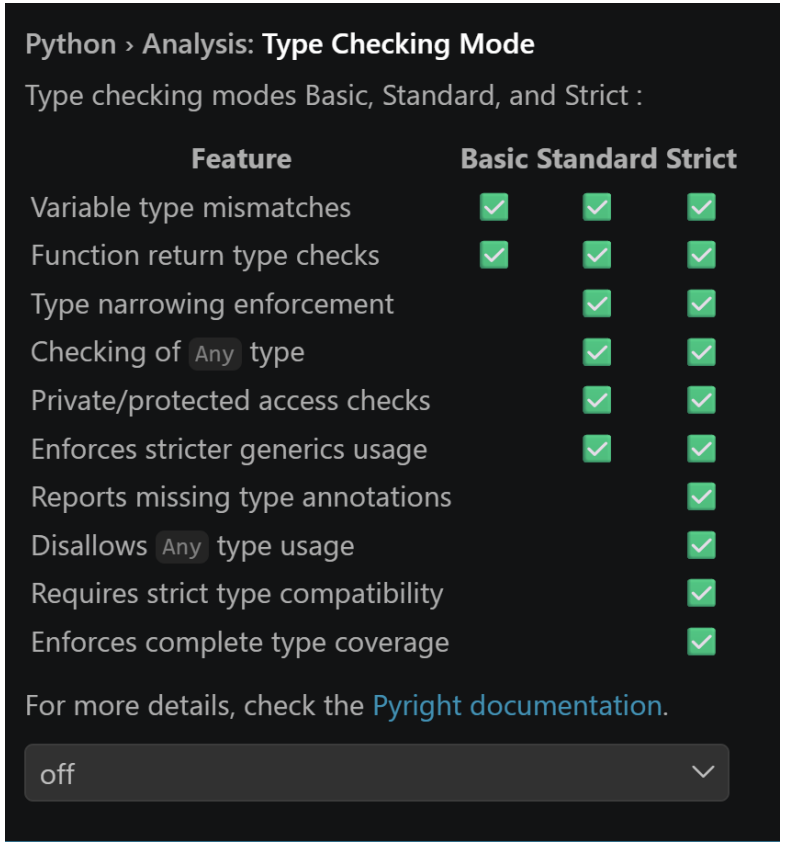




5 MINUTES

© 2026 Innovation In Software Corporation

ENABLE TYPE HINTS (VS CODE)



To enable type hint warnings in VS Code, open settings, and search for "type checking," then find the Python › Analysis: Type Checking Mode setting. Change the dropdown from off to Basic (or Standard/Strict for progressively stricter checks). Basic is enough to catch common mistakes like assigning a string to an int-hinted variable.

Keep in mind this is an editor-only warning: VS Code flags the mismatch visually, but Python itself still ignores type hints at runtime, so the file continues to run normally.

© 2026 Innovation In Software Corporation

TYPE HINTS – 2 *(IGNORE)*

Standard Python ignores type hints at runtime — it never checks or enforces them.

```
age: int = "hello"   # runs fine, no error
```

Type hints "activate" only when something deliberately reads them:

- Never at runtime (default) — Python stores the hint as metadata and moves on. No convert, no validate, no reject.
- A type checker, before you run — mypy, Pyright, or your editor (VS Code/Pylance) flag mismatches as you write. The main payoff: catching bugs at author time.
- A library, at runtime — FastAPI and Pydantic inspect your hints via introspection, then build validation, conversion, and docs. That's the library's doing, not Python's.

© 2026 Innovation In Software Corporation

PYDANTIC



Pydantic is a Python library that makes sure your data is correct before your program uses it. You describe what your data *should* look like using normal Python type hints, and Pydantic checks the real data against that description — automatically.

Think of it as a quality-control inspection at a factory: every item is measured against the specification before it moves down the line. Minor inconsistencies get corrected to standard.

© 2026 Innovation In Software Corporation

PYDANTIC - 2



Why it matters:

- Data from the outside world — web forms, APIs, files — is messy and can't be trusted.
- Without Pydantic, you'd hand-write endless checks: *Is this a number? Is a field missing? Is this blank?*
- With Pydantic, you define the specification once and it does all that checking for you.

© 2026 Innovation In Software Corporation

PYDANTIC - IMPLEMENTATION

Every Pydantic model is a Python class that inherits from BaseModel, which you import from the pydantic library. Inside the class, you declare your fields as variables annotated with type hints — the type hint is the rule Pydantic enforces.

```
from pydantic import BaseModel



class User(BaseModel):
    name: str
    age: int
    email: str
```

© 2026 Innovation In Software Corporation

PYDANTIC - EXAMPLE

You define the rules by listing fields and their types, then assign data:

```
from pydantic import BaseModel


class User(BaseModel):
    name: str
    age: int


print(User(name="Alice", age=30))
print(User(name="Bob", age="25"))
print(User(name="Carol", age="hello"))
```

© 2026 Innovation In Software Corporation

VALIDATIONERROR

When Pydantic validates incoming data against your model and something doesn't fit, ValidationError is raised — a special exception (imported from Pydantic). Instead of a vague crash, you get a structured report: for each bad field it tells you which field failed, what went wrong, and the value that caused it.

You can catch ValidationError with a normal try/except block and inspect the details programmatically. The error.errors() function returns a list of dictionaries you can loop through, or error.json() gives the same information as JSON).

© 2026 Innovation In Software Corporation

VALIDATIONERROR - EXAMPLE

```python
from pydantic import BaseModel, ValidationError

class User(BaseModel):
    name: str
    age: int

print(User(name="Alice", age=30))
print(User(name="Bob", age="25"))


try:
    print(User(name="Carol", age="hello"))
except ValidationError as error:
    print(error)
```

© 2026 Innovation In Software Corporation

ERROR

What function does print(error) call?





5 MINUTES

© 2026 Innovation In Software Corporation

ERRORS

A ValidationError is an exception object, not a list or dict itself. Calling its .errors() method returns a list of dicts — one dict per failed field.

Because .errors() gives you a list of dicts, you iterate it with a for loop and pull values out of each dict by key: error["loc"], error["msg"], error["input"].

The print(error) calls the __str__ function that relies on the same code as .errors.

© 2026 Innovation In Software Corporation

ASYNC



An async function in Python allows a program to start a task and then do other work while waiting for that task to finish.

This is useful for activities that spend a lot of time waiting, such as making web requests, interacting with a database, or reading from a file.

Instead of sitting idle during the wait, Python can switch to another task and keep the application responsive.

© 2026 Innovation In Software Corporation

COFFEE SHOP



A good analogy is a coffee shop worker. In a synchronous system, the worker starts a coffee machine and waits for it to finish before helping the next customer. In an asynchronous system, the worker starts the coffee, serves other customers, and returns when the machine signals that the coffee is ready.

For API servers, this allows a server to handle many requests efficiently without needing large numbers of threads.

© 2026 Innovation In Software Corporation

NO ASYNC

```
def make_coffee():
    print("Start coffee")
    print("Wait while coffee brews")
    print("Coffee done")


def make_toast():
    print("Start toast")
    print("Wait while toast cooks")
    print("Toast done")


make_coffee()
make_toast()
```

Coffee and toast:

> You start the coffee.
> You stand there doing nothing until the coffee is finished.
> Only after the coffee is done do you start the toast.

That is simple, but inefficient. The wasted time is here:

> **Wait while coffee brews**

During that waiting time, you could have started the toast.

© 2026 Innovation In Software Corporation

ASYNC DETAILS

async

- Defines an asynchronous function (coroutine).
- The function can be paused and resumed.
- Calling it does not execute it immediately.
- Calling it creates a coroutine object.

await

- await something()
- Waits for another coroutine or async operation to complete.
- While waiting, control is returned to the event loop.
- Other async tasks may run during this time.
- When the awaited operation completes, execution resumes at the next line.

© 2026 Innovation In Software Corporation

ASYNC DETAILS - 2

asyncio.gather

- Starts multiple coroutines.
- Allows them to make progress concurrently.
- Waits until all of them finish.
- Returns their results as a list.

```
await asyncio.gather(
    make_coffee(),
    make_toast()
)
```

© 2026 Innovation In Software Corporation

COFFEE - ASYNC

```python
import asyncio
import time


async def make_coffee():
    print("Start coffee")
    print("Coffee brewing (3 seconds)...")
    await asyncio.sleep(3)
    print("Coffee done")

async def make_toast():
    print("Start toast")
    print("Toast toasting (2 seconds)...")
    await asyncio.sleep(2)
    print("Toast done")
```

```python
async def main():
    start = time.perf_counter()


    await asyncio.gather(
        make_coffee(),
        make_toast(),
    )

    end = time.perf_counter()

    print(f"\nTotal duration: "
        f"{end - start:.2f} seconds" )

asyncio.run(main())
```

© 2026 Innovation In Software Corporation

DECORATOR



A decorator is a way to add extra behavior to a function without changing the function's own code. It "wraps" the function in another function.

In FastAPI, this is exactly how you define your API endpoints—you attach a decorator to a plain Python function to turn it into a route.

© 2026 Innovation In Software Corporation

DECORATOR - IMPLEMENTATION

In Python, functions are objects — just like a number or a string, a function is a value you can store in a variable, pass into another function, and return out of one.

A decorator uses that fact to wrap extra behavior *around* your function without touching the function's own code.

The syntax has two halves.

- First you write a decorator function that takes a function as input and returns a new, enhanced version of it.
- Then you apply it by writing the @ symbol followed by the decorator's name on the line directly above your function.

© 2026 Innovation In Software Corporation

DECORATOR – IMPLEMENTATION - 2

The @ is simply a shortcut. Writing @my_decorator above def greet() does the same thing as writing greet = my_decorator(greet) by hand: it passes greet into the decorator, gets back the wrapped version, and reassigns the name greet to point to that wrapped version.

So after the @ line runs, calling greet() actually calls the wrapped function — you get the extra behavior automatically, without changing greet's own code.

© 2026 Innovation In Software Corporation

DECORATOR - EXAMPLE

Here is an implementation of a decorator.

```
def my_decorator(func):

    def wrapper():
        print("Before the function runs")
        func()
        print("After the function runs")
    return wrapper


@my_decorator
def greet():
    print("Hello!")
```

© 2026 Innovation In Software Corporation

DECORATOR APPLIED TO A CLASS

The exact same @ syntax works on a class, not just a function. @my_decorator above a class means MyClass = my_decorator(MyClass) — identical rule to functions, just wrapping a class instead.

The most important real-world example is @dataclass, which auto-generates boilerplate (__init__, __repr__, etc.) for you:

```
from dataclasses import dataclass


@dataclass
class Point:
    x: int
    y: int
```

```
p = Point(3, 4)
```

© 2026 Innovation In Software Corporation

DECORATOR CLASS



A decorator just has to be callable and return something callable — it doesn't have to be a function. This is for a decorator written *as* a class (decorating a function).

You can use a class as the decorator by defining __call__, which makes instances of the class behave like functions.

© 2026 Innovation In Software Corporation

DECORATOR CLASS - EXAMPLE

```
class CountCalls:
    def __init__(self, func):
        self.func = func
        self.count = 0


    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"Call #{self.count} of {self.func.__name__}")
        return self.func(*args, **kwargs)

@CountCalls
def greet():
    print("Hello!")

greet()
greet()
```

© 2026 Innovation In Software Corporation

FASTAPI

For FastAPI, decorators connect a function to a URL. You write a function that returns data, then add a decorator above it telling FastAPI which address and method should trigger it — FastAPI handles the request and turns your return value into JSON.

@app.get("/hello") registers say_hello for GET requests to /hello. The verb sets the method: get (read), post (create), put (update), delete (remove).

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/hello")
def say_hello():
    return {"message": "Hello!"}
```


© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned to create and manage virtual environments — activating them, installing dependencies from PyPI, and freezing them to requirements.txt to isolate each project. You also learned why Python ignores type hints at runtime while type checkers, Pydantic, and FastAPI read them.

You then explored async and await for I/O concurrency, Pydantic models for validating data and catching a ValidationError, and decorators — the @ syntax FastAPI uses for routing.

© 2026 Innovation In Software Corporation

LAB 2 – PYDANTIC / DECORATOR



GOAL

In this lab, you will rely upon two foundational Python skills for FastAPI work: managing dependencies with virtual environments and validating data with Pydantic. You will create a virtual environment, install a third-party library, and prove the isolation firsthand by watching an import succeed inside the environment and fail outside it.

You will then build a Pydantic model that catches bad data through type hints, add their own @field_validator rules for age, salary, dates, and departments, and write a decorator plus a try/except block so bad rows never crash the program. Scaffolded as six fill-in-the-blank TODOs with tips. The final solution will cleanly separate 8 valid records from 12 flawed ones.

© 2026 Innovation In Software Corporation

CREATE VIRTUAL ENVIRONMENT

Open PowerShell, move into the lab folder, and create the environment there:

1. Go to the lab folder

```
cd labs\lab2
```

2. Create the environment (once)

```
python -m venv .venv
```

3. Activate it (every new terminal)

```
.\.venv\Scripts\Activate.ps1
```

Verification: Your prompt should start with (.venv):

© 2026 Innovation In Software Corporation

INSTALL DEPENDENCIES

1. Install the dependencies this lab needs

```
pip install pandas "pydantic[email]"
```

2. Confirm it was installed in the virtual environment

```
python -c "import pandas; print(pandas.__version__)"
```

3. Open the lab file and find the TODOs

```
code messy.py    # or open it in your editor
```


© 2026 Innovation In Software Corporation

INSTALL DEPENDENCIES – 2

Because the venv was active, pip installed pandas into labs\lab2\.venv — not your system Python. If you deactivate and use the import, it fails with ModuleNotFoundError, because the package only exists inside this lab's environment.

That is the whole benefit: each project keeps its own packages, and installing one never breaks another.

© 2026 Innovation In Software Corporation

DECORATOR



A decorator is a function that wraps another function to add behavior, without changing that function's code. You apply one by writing @name on the line above a function.

You'll write a decorator called timer that measures how long a function runs and prints the time, then attach it to the function loads a database.

FastAPI's @app.get("/users") works the same way — it's a decorator.

© 2026 Innovation In Software Corporation

TODO 1

A decorator wraps a function to add behavior without changing that function. In messy.py the timer decorator is already written except for the line that reports the time. Find TODO 1 inside the wrapper and complete the print.

Where: the timer function, near the top of messy.py.

What to do: print a message using the elapsed variable, which contains the number of seconds the function took.

Hint: use an f-string. The answer has the shape f"...{elapsed:.3f}s" — the .3f keeps it to three decimals.

© 2026 Innovation In Software Corporation

PYDANTIC: FIELD_VALIDATOR

Data from outside your program — a CSV, a form, an API — can't be trusted to be clean. Pydantic lets you describe your data's shape once, as a class, using type hints. It then checks and converts each field automatically, turning "34" into 34 and rejecting "thirty".

When type hints aren't enough, you add rules with a @field_validator method (age 0–120, salary ≥ 0, and so on). If a field fails, Pydantic raises a ValidationError to indicate what went wrong.

© 2026 Innovation In Software Corporation

TYPE HINTS

Open the Employee class in messy.py. Each field is labelled with a type — employee_id is an int, email is an EmailStr, and so on. Pydantic uses those labels to convert and check incoming data automatically, before any of your own rules run.

email → EmailStr: a malformed address like carol.white[at]corp.com is rejected on sight.

age → int: the word "thirty" can't become an integer, so it's rejected.

salary → float: a blank cell isn't a number and is rejected.

Takeaway: type hints catch the obvious problems for free — you only write validators for the rules they can't express.

© 2026 Innovation In Software Corporation

TODO 2 AND TODO 3

Type hints already reject the obvious errors (a non-numeric age, a malformed email). For everything else you add a rule. In messy.py the four validator methods are written for you; each has one blank line marked with a TODO. Complete all four.

TODO 2 — age: reject any age that is not between 0 and 120. The check is a chained comparison: 0 is less-than-or-equal-to the value, which is less-than-or-equal-to 120.

TODO 3 — salary: reject a salary below zero — compare the value to 0.

© 2026 Innovation In Software Corporation

TODO 4 AND TODO 5

TODO 4 — start_date: confirm the date matches YYYY-MM-DD by parsing it with the pattern that uses year, month, day codes separated by dashes.

TODO 5 — department: reject any department that is not one of the four known names in the provided list.

© 2026 Innovation In Software Corporation

BAD DATA

When Pydantic rejects a row it raises a ValidationError. If nothing catches it, the whole program stops — so one bad row would block every good one.

A try / except block fixes this: build the Employee inside try; if it fails, the except block logs the problem and moves to the next row. Good data flows through, bad data is logged. The finished lab accepts 8 rows and rejects 12.

© 2026 Innovation In Software Corporation

TODO 6

The final TODO adds a short summary. In messy.py a loop already adds every valid employee; you complete the one line that increases each department's count. Find TODO 6 near the bottom of the file.

What to do: add one to the running count for that employee's department.

Hint: the line uses dict.get(department, 0), which returns 0 the first time a department appears — so you only need to add 1 to whatever it returns.

Check: run the file; a short headcount prints after the valid employees.

Note: Sales shows nothing, because both Sales rows failed other checks and were rejected.

© 2026 Innovation In Software Corporation

DECORATOR

After completing the TODOs, run the file. Your terminal should look like the screenshot below — 8 rows accepted and 12 rejected, each rejection naming the field that failed.

```
Terminal

> python messy.py

  1004  age -5 out of range 0-120
  1005  Input should be a valid integer
  1012  start_date '15-03-2020' not YYYY-MM-DD
  1019  value is not a valid email address
  ...
  Valid: 8  |  Rejected: 12
  load_employees: 0.024s
```

© 2026 Innovation In Software Corporation

Lab completed



FASTAPI

AN INTRODUCTION



INTRODUCTION



In this module, you will build your first FastAPI application, learning how it wires together Starlette and Pydantic around type hints for automatic validation and free interactive docs via Swagger UI.

You will configure the FastAPI object, map URLs to endpoint functions with route decorators, run the app with fastapi dev, and build an items API tested with curl.

© 2026 Innovation In Software Corporation

WHAT IS FASTAPI

A high-performance Python framework for building HTTP APIs, built on Starlette and Pydantic. It uses standard type hints to validate, serialize, and document your data automatically.

- Type-driven — Type hints drive request parsing and validation.
- Automatic docs — Swagger UI and ReDoc generated for you.
- High performance — Async support makes it among the fastest Python frameworks.
- Easy to learn — A working API is only a handful of lines.

© 2026 Innovation In Software Corporation

STARLETTE



Starlette is a lightweight Python toolkit for building fast web services. It handles the basic plumbing of a web app: routing (deciding which code runs for which URL), requests and responses, middleware, WebSockets, and serving files.

Because it's built on the modern ASGI standard and uses Python's async/await, it can juggle lots of connections at once without slowing down—great for apps that spend time waiting on databases or other APIs.

© 2026 Innovation In Software Corporation

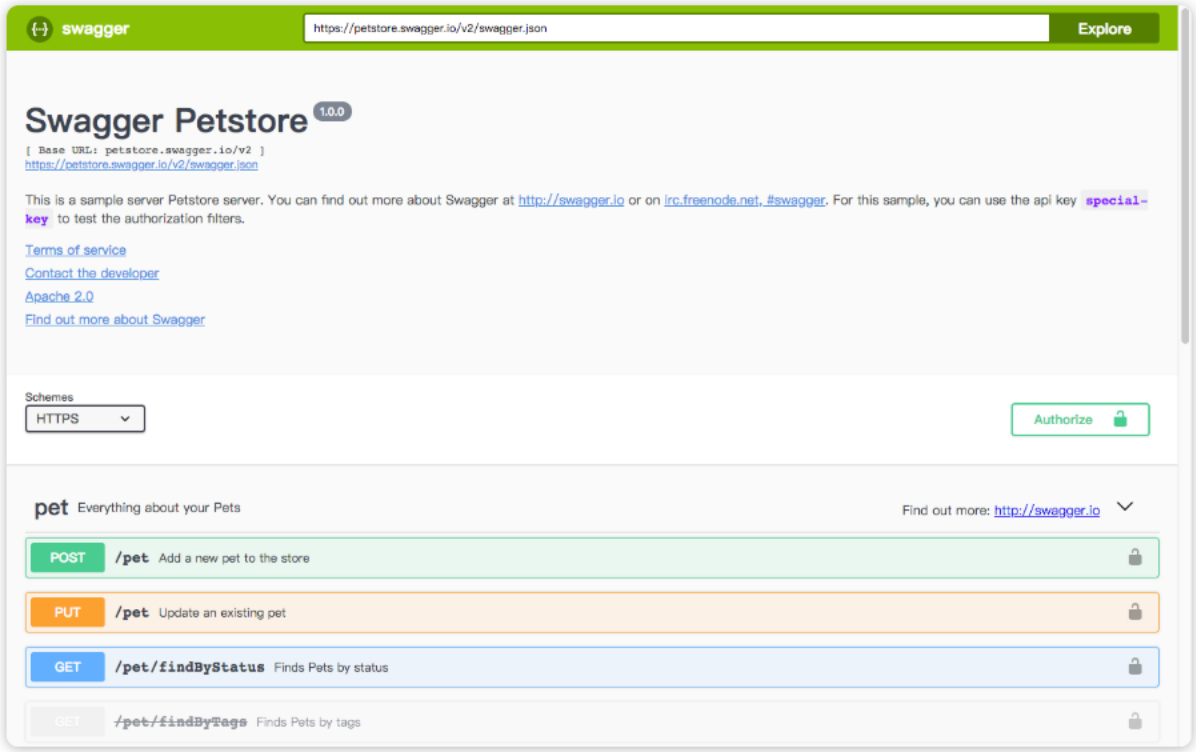
STARLETTE - 2



FastAPI is built right on top of Starlette. Think of Starlette as the engine and FastAPI as the car built around it: FastAPI adds handy features like automatic data checking and auto-generated docs, but the speed and async powers come from Starlette underneath.

© 2026 Innovation In Software Corporation

SWAGGER UI

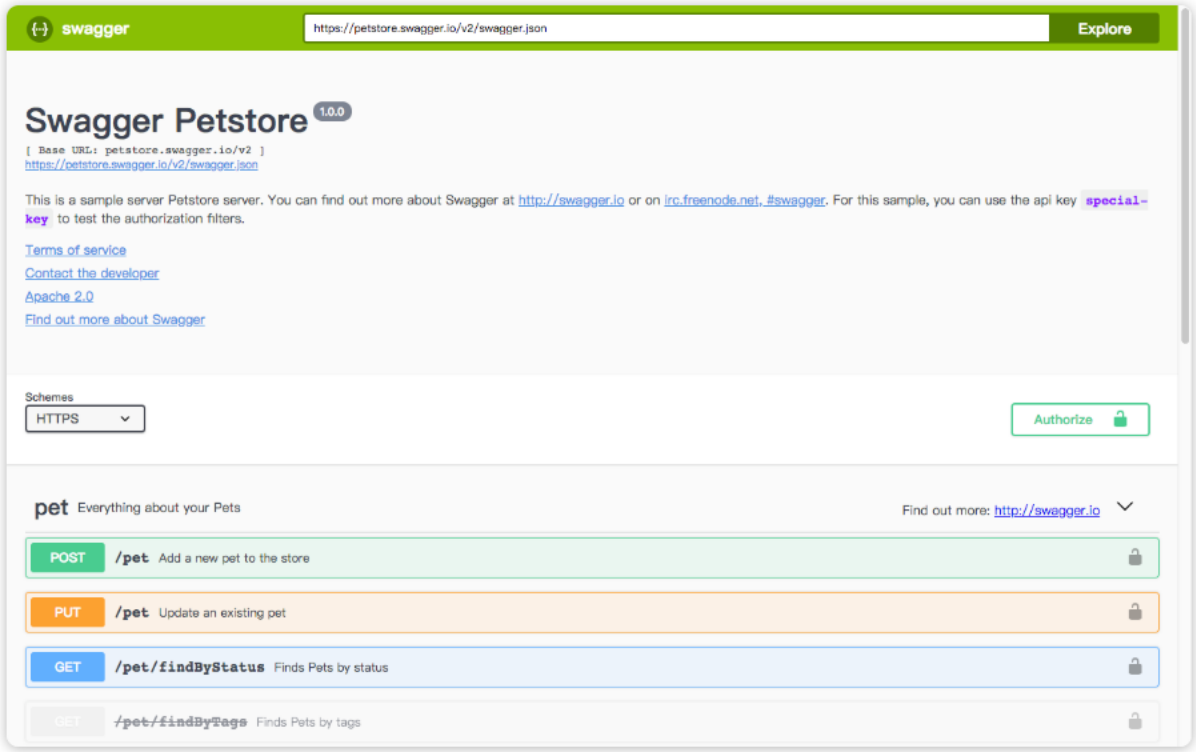


Swagger UI is a tool that turns your API's specification into an interactive web page you can click around in.

Instead of reading a static document that lists your API's endpoints, you get a live page in your browser where every endpoint is displayed with its details—what data it expects, what it returns, and what the possible responses are.

© 2026 Innovation In Software Corporation

SWAGGER UI - 2



With FastAPI, Swagger UI comes for free. FastAPI automatically generates an OpenAPI specification from your code and type hints, then serves the Swagger UI page—usually at the /docs URL of your running app. From the moment you write an endpoint, it shows up in interactive docs with no extra work.

© 2026 Innovation In Software Corporation

HELLO, WORLD!

```python
from fastapi import FastAPI          # import the framework

app = FastAPI()                      # creates the application object

@app.get("/")                        # handle GET requests to "/"
def read_root():
    return {"message": "Hello, World!"}     # dict → auto JSON

@app.get("/greet/{name}")            # {name} is a path parameter
def greet(name: str):                # typed & validated as str
    return {"message": f"Hello, {name}!"}   # returns JSON
```


© 2026 Innovation In Software Corporation

INSTALL FASTAPI

FastAPI can be installed on its own, but the recommended approach includes a few extra tools that make development easier.

```
pip install "fastapi[standard]“
```

Why [standard]? It installs FastAPI plus its recommended optional dependencies, including Uvicorn (the server) and the fastapi command-line tool. That gives you *fastapi dev main.py*, which finds your app in main.py and starts a Uvicorn server.

© 2026 Innovation In Software Corporation

FASTAPI - IMPORTS

The *from fastapi import* command imports only the app class — nothing else.

- FastAPI ships dozens of helper classes and functions, but each must be pulled in by name
- Two "levels" of imports to understand:
  - Top-level — the common request-handling tools, all under fastapi
  - Submodules — responses, security, middleware, and more, under deeper paths
- Knowing which is which saves you from confusing ImportErrors and NameErrors

© 2026 Innovation In Software Corporation

FASTAPI – IMPORTS - 2

The tell-tale difference: top-level names sit right after fastapi import, while submodule names require an extra dotted segment (fastapi.responses, fastapi.security, etc.) before import.

- Top-level — imported directly from fastapi:

```
from fastapi import FastAPI, Depends, HTTPException, Query
```

- Submodule — imported from a deeper path under fastapi:

```
from fastapi.responses import JSONResponse
```


© 2026 Innovation In Software Corporation

FASTAPI OBJECT

FastAPI() is the only way to create the app object. What varies is the arguments you pass in — they configure metadata, docs, startup logic, and more

No parameters:

```
app = FastAPI()
```

This is the simplest possible app — no configuration

- Perfectly valid; you get sensible defaults for everything
- Docs, OpenAPI schema, and routing all work out of the box
- Good for demos, prototypes, and learning
- Bottom line: Start with FastAPI() and add arguments only as your needs grow.

© 2026 Innovation In Software Corporation

FASTAPI OBJECT - METADATA

Bottom line: Metadata turns bare auto-docs into a polished, self-describing API page.

```
app = FastAPI(
    title="My API",
    description="Does useful things",
    version="1.0.0",
)
```

These values appear in the auto-generated documentation pages:

- title — the name shown at the top of your docs
- description — a summary of what the API does (supports Markdown)
- version — your API's version number, displayed alongside the title

Costs nothing and makes your docs look professional

© 2026 Innovation In Software Corporation

FASTAPI OBJECT - DOCUMENTATION

When creating the FastAPI object, you can control the various auto-documentation URLs:

- docs_url — moves Swagger UI to a custom path (here, /swagger instead of /docs)
- redoc_url=None — disables the ReDoc page entirely
- openapi_url=None — disables the raw OpenAPI schema (this also turns off *both* docs pages, since they depend on it)

```
app = FastAPI(docs_url="/swagger", redoc_url=None,
        openapi_url=None)
```

© 2026 Innovation In Software Corporation

DEFINE A ROUTE

Every FastAPI endpoint starts with a decorator that ties a URL and HTTP method to a function. When a GET request comes in for /, run the function underneath.

```
@app.get("/")           # handle GET requests to "/"
def read_root():
    ...
```

@app.get("/") — a decorator that registers the function below as a route handler:

- app — your FastAPI instance (app = FastAPI())
- .get — responds to the HTTP GET method
- "/" — the path; the root URL of your site, e.g. http://yourserver/

© 2026 Innovation In Software Corporation

ENDPOINT FUNCTION

The decorator sits on top of an ordinary Python function that does the actual work. The decorator connects the URL to this function; the function contains your logic.

```
@app.get("/")
def read_root():
    ...
```

def read_root(): — the function that handles the request:

- It's a normal Python function — nothing special about it
- The name (read_root) is arbitrary; FastAPI attaches it to the route and never calls it by name
- You could name it anything and the route would behave identically

© 2026 Innovation In Software Corporation

ENDPOINT FUNCTION - 2

Whatever the handler returns becomes the response — and FastAPI turns it into JSON for you. You work with normal Python objects (dicts, lists, Pydantic models) and FastAPI handles serializing them into a proper JSON HTTP response.

```
def read_root():
    return {"message": "Hello, World!"}
```

You return a plain Python dictionary

- FastAPI automatically converts it to JSON
- The client receives: {"message": "Hello, World!"}

© 2026 Innovation In Software Corporation

RETURN - NOTES

FastAPI serializes far more than the standard json module, but ORM objects, non-string dict keys, and response_model filtering are the three that trip people up most.

Non-serializable objects fail. JSON only understands simple data — text, numbers, true/false, lists, and dictionaries. Return a file handle, a NumPy array, or many custom classes, and FastAPI can't translate it, so you get an error. The fix: convert the object to a dictionary or Pydantic model first, then return that.

2. Dict keys get coerced to strings. JSON object keys must be strings. return {1: "a"} comes back as {"1": "a"}—the integer key silently becomes a string.

© 2026 Innovation In Software Corporation

RETURN – NOTES - 2

3. Float precision and special values. float('nan'), float('inf') aren't valid JSON. FastAPI handles these, but the result may not be what downstream consumers expect. Money as floats is the usual rounding trap—use Decimal or integer cents.

4. A declared response_model filters your output. If you set response_model=SomeSchema, FastAPI will *drop* any fields not in that schema—even if your function returned them. This surprises people ("where did my field go?"). It's a feature that prevents leaking data like password hashes.

5. Returning a Response object skips conversion entirely. If you return JSONResponse(...) or any Response yourself, FastAPI assumes you've handled serialization.

© 2026 Innovation In Software Corporation

RETURN – NOTES - 3

6. Large payloads and streaming. Returning a giant dict serializes it all in memory at once. For big or streaming data, use StreamingResponse instead.

7. Circular references (object A refers to B refers back to A) will raise an error—JSON has no way to represent them.

© 2026 Innovation In Software Corporation

RUN APP

Once you've saved the app, running it takes a single command — and FastAPI gives you a friendly one that handles the server setup for you.

From the same folder run:

```
fastapi dev main.py
```

That starts a development server (with auto-reload on code changes). Under the hood, fastapi dev runs Uvicorn — it's a convenient wrapper from the fastapi-cli package that imports your app and starts a Uvicorn server with development defaults (binds to 127.0.0.1:8000, reload enabled).

© 2026 Innovation In Software Corporation

RUN APP - 2

Either way the server runs at http://127.0.0.1:8000. Once it's up, you can hit these URLs in your browser to see it respond:

- http://127.0.0.1:8000/ → {"message": "Hello, World!"}
- http://127.0.0.1:8000/greet/Ada → {"message": "Hello, Ada!"}
- http://127.0.0.1:8000/docs → interactive Swagger UI to try the endpoints

© 2026 Innovation In Software Corporation

ITEMS

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True

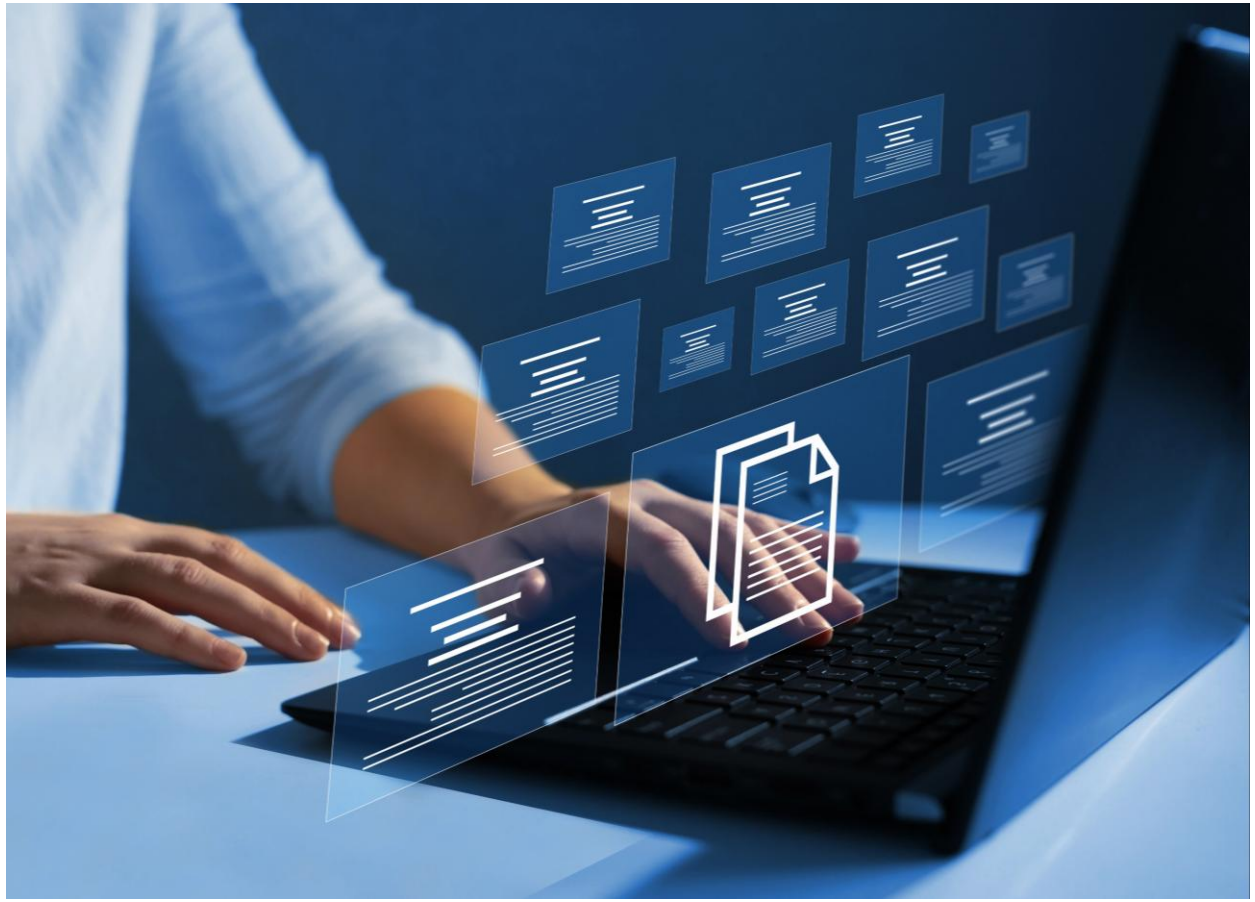
# In-memory "database"
items: list[Item] = []

# GET — return all items
@app.get("/items")
def list_items():
    return items
```

```python
# POST — add a new item
@app.post("/items")
def create_item(item: Item):
    items.append(item)
    return {"message": "Item added", "item": item}
```


© 2026 Innovation In Software Corporation

LIST OF ITEMS



A FastAPI app that manages a list of items in memory:

- Item (— defines the required shape of each item: a name (text), a price (number), and an optional in_stock flag that defaults to True.
- In-memory "database" — items: list[Item] = [] is just a Python list that holds the items while the app runs.
- GET /items — returns the full list of items as JSON.
- POST /items — accepts a new item in the request body, appends it to the list, and confirms it was added.

© 2026 Innovation In Software Corporation

PYDANTIC

BaseModel from Pydantic lets you define the shape of your data as a class (here Item, with a name, price, and an optional in_stock field), and FastAPI uses that model to automatically validate, parse, and document request data.

In this program it's used in the create_item POST handler — the item: Item parameter tells FastAPI to read the JSON request body, validate it against the Item model (rejecting bad input with a clear error), and hand you a ready-to-use Item object, which then also shapes the items stored in and returned from the list.

© 2026 Innovation In Software Corporation

CURL POST

```
curl -X POST http://127.0.0.1:8000/items -H "Content-Type:
    application/json" -d "{\"name\": \"Widget\",
    \"price\": 9.99}"
```

First, start the server using fastapi command:

fastapi dev second.py

The browser address bar only sends GET requests, so it works for reading items but can't test POST. Use curl from the terminal instead:

Important: enter as a continuous line of input – no line breaks.

© 2026 Innovation In Software Corporation

CURL GET

You can use the curl command to send a GET request that returns the full list of items as JSON — including any you've just added via POST — by hitting the /items route:

```
curl http://127.0.0.1:8000/items
```

Since this is a GET, you could also do the same thing straight from your browser — just visit http://127.0.0.1:8000/items in the address bar to see the list.

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you built a FastAPI application on Starlette and Pydantic, learned the import system, and configured the FastAPI object and its docs URLs. You mapped URLs to endpoint functions with route decorators.

You also saw how return values are serialized to JSON and its common pitfalls, ran the app with fastapi dev, and built an items API with a Pydantic model, testing its GET and POST routes with curl.

© 2026 Innovation In Software Corporation

LAB 3 – CHECKING ACCOUNT



GOAL



The objective of this lab is to build a web API that manages a single checking account balance.

By the end, your API will:

- Return the current balance
- Accept deposits that increase the balance
- Accept withdrawals that decrease the balance
- Reject any withdrawal that would overdraw the account

© 2026 Innovation In Software Corporation

TEMPLATE

You are given a working checking.py template using the Items example.

Study how it is structured:

- A FastAPI() app instance
- A Pydantic BaseModel defining the shape of incoming data
- Module-level data holding state
- GET and POST route handlers

You will replace the Items logic with checking account logic, keeping this same structure.

© 2026 Innovation In Software Corporation

REQUIREMENTS

Three routes:

- GET /balance — returns the current balance
- POST /deposit — increases the balance by the amount provided, returns the balance
- POST /withdrawal — decreases the balance by the amount provided, returns the balance

Overdraft rule:

- If a withdrawal would make the balance negative, do not change the balance
- Instead, return the message: *withdrawal would overdraw account*

© 2026 Innovation In Software Corporation

TRANSACTION MODEL

Replace the Item model with a model that describes a transaction.

- A deposit and a withdrawal both need one piece of data: an amount
- Decide the field name and its type
- This model will be the parameter type for both POST routes

Adapt the template's current Item model:

```
# Template starting point:
class Item(BaseModel):
    name: str
    price: float
    in_stock: bool = True
```


© 2026 Innovation In Software Corporation

TRANSACTION MODEL - 2

Change it into a transaction model:

- Rename Item to Transaction
- Replace the three item fields with a single amount field of type float
- This model becomes the parameter type for both /deposit and /withdrawal

```
class Transaction(BaseModel):
    amount: float
```


© 2026 Innovation In Software Corporation

BALANCE

Replace the in-memory items list with a single balance value.

- Balance is one number, not a collection
- Name the module-level variable balance, add a type hint, and set its starting value to 100
- Use a float type hint so the balance can hold cents: balance: float = 100
- Use this same variable name, balance, in every route that reads or changes it
- It lives at module level so every request sees the same account

© 2026 Innovation In Software Corporation

BALANCE ROUTE

Replace the template's GET /items handler with a GET /balance handler.

- Delete or replace the existing list_items GET route
- Your new route takes no input
- It returns the current balance
- This route only reads the value — it never changes it

© 2026 Innovation In Software Corporation

MANAGING GLOBAL STATE

Any route that *changes* the balance must reassign the module-level variable.

- Reading a global works without any declaration → GET /balance needs nothing
- Reassigning a global requires declaring global balance inside the function
- Without it, balance = … creates a local variable and raises an error

For this reason, deposit and withdrawal handler each need global balance; get_balance does not.

© 2026 Innovation In Software Corporation

DEPOSIT ROUTE

Replace the template's POST /items handler with a POST /deposit handler.

- Delete or replace the existing create_item POST route
- Accept the transaction model as input
- Increase the balance by the amount
- Return the updated balance
- Solution — adapt the template's create_item route:

© 2026 Innovation In Software Corporation

DEPOSIT ROUTE - SOLUTION

Here is the solution. Use this as a template for the remaining handlers.

```
@app.post("/deposit")
def deposit(transaction: Transaction):
        global balance
        balance += transaction.amount
        return balance
```


© 2026 Innovation In Software Corporation

WITHDRAWAL ROUTE - SOLUTION

Add a POST /withdrawal handler.

- Since the template has only one POST route, add this one as a new handler (or copy your deposit route and adapt it)
- Accept the transaction model as input
- Check whether subtracting the amount would push the balance below zero
- If it would: leave the balance unchanged and return the overdraft message
- Otherwise: reduce the balance and return the updated balance

© 2026 Innovation In Software Corporation

RUN THE SERVER

Start the app from the command prompt in the folder containing your file:

```
fastapi dev checking.py
```

From the command prompt, test each route in order, beginning with the balance:

```
curl http://127.0.0.1:8000/balance
```


© 2026 Innovation In Software Corporation

RUN THE SERVER - 2

From the command prompt, test the remaining routes:

Deposit:

curl -X POST http://127.0.0.1:8000/deposit -H "Content-Type: application/json" -d "{\"amount\": 100}“

Withdraw:

curl -X POST http://127.0.0.1:8000/withdrawal -H "Content-Type: application/json" -d "{\"amount\": 40}"

© 2026 Innovation In Software Corporation

BONUS

If you finish early, try:

- Reject deposits or withdrawals of zero or negative amounts
- Return the balance rounded to two decimal places
- Add a route that resets the balance to zero
- Track and return a running count of transactions

© 2026 Innovation In Software Corporation

Lab completed



CONTRACT FIRST

OPENAI



INTRODUCTION



In this module, you will learn the OpenAPI specification — a standardized, machine-readable description of a REST API — and the contract-first approach that treats the spec as the single source of truth. You will see how info, paths, components, and schemas describe an API in one structured document.

You will document request bodies, responses, parameters, and error codes, then generate the specification from a running app, save it, and explore the result in Swagger UI.

© 2026 Innovation In Software Corporation

OPENAPI



A standardized, machine-readable description of a REST API — written in JSON or YAML.

- Defines every endpoint, its inputs, its outputs, and its data shapes in one structured document
- Language-agnostic: describes *what* the API does, independent of *how* it's implemented and by whom
- Both humans and tools can read it — the foundation for docs, client generation, testing, and validation

© 2026 Innovation In Software Corporation

SOURCE OF TRUTH

One authoritative document that everything else derives from.

- Single reference — front-end, back-end, QA, and external consumers all work from the same contract
- No drift between teams — the spec, not tribal knowledge or a stale wiki, defines the agreement
- Tooling flows from it — interactive docs, client SDKs, mock servers, and contract tests are all generated from the spec
- Contract-first workflows — teams can agree on the spec *before* a line of implementation is written

© 2026 Innovation In Software Corporation

CONTRACT-FIRST



Design the interface first; build against it in parallel.

- Front-end and back-end teams start simultaneously, both coding to the agreed contract
- Consumers know exactly what to expect before the API is finished
- Breaking changes become visible as changes to the contract
- Reduces integration surprises late in the cycle

© 2026 Innovation In Software Corporation

WALKTHROUGH – CONTRACT FIRST (OPENAPI)



This walkthrough takes a minimal OpenAPI spec and turns it into a running FastAPI app. The idea is contract-first: you write the API definition first, generate the code stubs from it, then fill in the logic

© 2026 Innovation In Software Corporation

SETUP

Install: fastapi-code-generator

```
pip install fastapi "uvicorn[standard]" fastapi-code-generator
```

Generate app:

```
fastapi-codegen --input simple.yaml --output app
```


© 2026 Innovation In Software Corporation

SIMPLE.YAML

```yaml
openapi: 3.1.0
info:
  title: Simple API
  version: 1.0.0
paths:
  /items:
    get:
      responses:
        '200':
          description: OK
```


© 2026 Innovation In Software Corporation

SIMPLE.YAML - APP

```
# generated by fastapi-codegen:
#   filename:  simple.yaml
#   timestamp: 2026-08-08T07:58:29+00:00


from __future__ import annotations


from fastapi import FastAPI


app = FastAPI(
    title='Simple API',
    version='1.0.0',
)



@app.get('/items', response_model=None)
def get_items() -> None:
    pass
```


© 2026 Innovation In Software Corporation

MAIN.PY

Edit app/main.py to have the get_items handler return something:

```
#@app.get('/items', response_model=None)
# def get_items() -> None:
#     pass


@app.get('/items', response_model=None)
def get_items():
    return [{"id": 1, "name": "widget"}]
```


© 2026 Innovation In Software Corporation

VIEWING THE RESULTS

Run the app:

```
fastapi dev main.py
```

Afterwards, exercise the application – try the get_items handler.



© 2026 Innovation In Software Corporation

INFO: API METADATA

The info block describes your API's identity—the metadata that tells consumers what this API is, who owns it, and which version they're looking at.

- title — the human-readable name of the API
- version — the API's version (distinct from the OpenAPI spec version)
- description — a longer summary, supports Markdown
- Optional: contact, license, termsOfService

This is what appears at the top of your generated documentation page

© 2026 Innovation In Software Corporation

PATHS: ENDPOINTS AND OPERATIONS

The paths block is its functionality—the complete map of every URL the API exposes and what you can do at each one. It is where the actual routes are defined.

- Keyed by URL path (e.g., /items, /items/{id})
- Under each path, keyed by HTTP method (get, post, put, delete)
- Each operation defines: summary, parameters, requestBody, responses
- Path parameters like {id} are declared inline and typed

This is the heart of the spec — the map of everything the API can do.

© 2026 Innovation In Software Corporation

COMPONENTS AND SCHEMAS

The components block is a library of reusable pieces—define a data model or response once, then point to it from anywhere in the spec instead of copying it.

- components/schemas — data models (e.g., an Item with name, price, in_stock)
- Referenced with $ref: '#/components/schemas/Item' instead of repeating the definition
- Also holds reusable responses, parameters, securitySchemes, etc.
- Keeps the spec DRY (Don't Repeat Yourself) and consistent across endpoints

© 2026 Innovation In Software Corporation

DOCUMENTING REQUEST BODIES

The request body defines the data a client sends *to* your API — the payload carried inside a POST, PUT, or PATCH request, such as the JSON for a new user or updated order.

- Declared under an operation's requestBody
- Specifies the content type (e.g., application/json) and the schema of the payload
- Points to a reusable schema via $ref
- Marks whether the body is required

Let’s tools validate incoming data and generate example requests

© 2026 Innovation In Software Corporation

REQUESTBODY - EXAMPLE

```yaml
requestBody:
  # must send a body
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/Item'

components:
  schemas:
    Item:
      type: object
      properties:
        name:
          type: string
        price:
          type: number
      # body must contain these
      required:
        - name
        - price
```


© 2026 Innovation In Software Corporation

REQUEST BODIES – USE CASES

Case 1 — no body sent -> fails the outer check:

```
POST /items
(empty)
```

Result: 422 Unprocessable Entity — "field required: body"

Case 2 — body sent, but missing price -> passes outer, fails inner:

```
POST /items
{ "name": "widget" }
```

Result: 422 Unprocessable Entity — "field required: price"

© 2026 Innovation In Software Corporation

REQUEST BODIES – USE CASES - 2

Case 3 — body sent with both fields → passes both, request goes through:

POST /items

{ "name": "widget", "price": 9.99 }

The result is 200 Created

© 2026 Innovation In Software Corporation

API: RESPONSES

As discussed, requestBody describes what the client sends in, responses describes what comes back from the API server—one entry for each outcome the endpoint can produce.

- Keyed by HTTP status code (200, 201, 404, 422…)
- Each response has a description and, usually, a schema for its body
- Multiple statuses can be documented per endpoint (success *and* failure)
- Enables accurate "what to expect" docs and response validation

© 2026 Innovation In Software Corporation

API: RESPONSES - EXAMPLE

```yaml
paths:
  /items/{id}:
    get:
      responses:
        '200':
          description: The item was found
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Item'
        '404':
          description: No item with that ID exists
```

This shows the two most common outcomes side by side: a 200 success that returns an Item, and a 404 failure that just carries a description. A client reading this knows both what a successful response looks like *and* how failure is signaled

© 2026 Innovation In Software Corporation

API: RESPONSES – EXAMPLE - CODE

```
app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

items = {1: Item(name="widget", price=9.99)}


@app.get('/items/{id}')
def get_item(id: int) -> Item:
    if id not in items:
        raise HTTPException(status_code=404,
            detail="No item with that ID exists")
    return items[id]
```

Here is how the two responses map to the code:

200 — the return items[id] line. Returning the Item gives FastAPI a success response; it serializes the object to JSON and sends 200 automatically. You don't write the status code—returning normally *is* the 200 path.

404 — the raise HTTPException(...) line. When the ID isn't found, raising this stops the handler and sends back a 404 with your detail message, matching the spec's "No item with that ID exists."

© 2026 Innovation In Software Corporation

PREVIOUS CODE

What are the possible status codes?





5 MINUTES

© 2026 Innovation In Software Corporation

PARAMETERS / ERROR CODES

A thorough spec documents every input the API accepts and every response it can return — successes and failures alike.

- Parameters — path, query, header, and cookie inputs, each with a name, location (in), type, and required flag
- Error codes — document 4xx/5xx responses so consumers can handle failure gracefully
- A well-documented error contract is as valuable as the success contract
- Example: 422 Unprocessable Entity for validation failures, 404 for a missing resource

© 2026 Innovation In Software Corporation

PARAMETERS / ERROR CODES - EXAMPLE

```yaml
paths:
  /items:
    get:
      summary: List items
      parameters:
        - name: limit
          in: query
          required: false
          schema:
            type: integer
            default: 10
```

```yaml
responses:
  '200':
    description: Item found
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Item'
  '404':
    description: No item with that ID exists
  '422':
    description: Validation error (e.g., id was not an integer)
```


© 2026 Innovation In Software Corporation

GENERATING SPECIFICATIONS



The spec and the code can each produce the other—you can scaffold an app from a hand-written spec (contract-first), or write FastAPI code and have it generate the spec automatically. We have discussed the first option – now the second option.

© 2026 Innovation In Software Corporation

GENERATING SPECIFICATIONS - 2

The spec comes from your code — not a separate document you maintain.

- FastAPI reads your type hints and Pydantic models to infer schemas
- Path operations (@app.get, @app.post) become paths entries automatically
- Pydantic BaseModel classes become entries in components/schemas
- The generated document is served at /openapi.json
- Generates OpenAPI specifications

© 2026 Innovation In Software Corporation

WALKTHROUGH – GENERATE SPECIFICATIONS



This hands-on walkthrough shows how FastAPI turns your Python code into a standard OpenAPI spec—no spec written by hand. Starting from a minimal main.py, you'll run the app, view the auto-generated spec at /openapi.json.

© 2026 Innovation In Software Corporation

MAIN.PY

Turn FastAPI code into a saved OpenAPI spec file — no spec written by hand. The code is located at *generate_specs_wt\main.py*.

```
from fastapi import FastAPI


app = FastAPI()


@app.get('/items')
def get_items():
    return [{"id": 1, "name": "widget"}]
```


© 2026 Innovation In Software Corporation

SETUP

Set up the tools and move into the project folder.

- fastapi — the framework, uvicorn — the server
- Run all remaining commands from this folder
- main.py should be in this directory

```
pip install fastapi "uvicorn[standard]"
cd module4\generate_specs_wt
```


© 2026 Innovation In Software Corporation

RUN THE APP

Start the development server so FastAPI is live.

- main — the file (main.py)
- app — the FastAPI() object inside it
- Server starts at http://127.0.0.1:8000

```
fastapi dev main.py
```


© 2026 Innovation In Software Corporation

VIEW THE OPENAPI SPECIFICATION

See the spec FastAPI generates on the fly.

- FastAPI generates the full OpenAPI document live
- Nothing was written by hand

http://127.0.0.1:8000/openapi.json


```
{
  "openapi": "3.1.0",
  "info": {
    "title": "FastAPI",
    "version": "0.1.0"
  },
  "paths": {
    "/items": {
      "get": {
        "summary": "Get Items",
        "operationId": "get_items_items_get",
        "responses": {
          "200": {
            "description": "Successful Response",
```


© 2026 Innovation In Software Corporation

SAVE THE DOCUMENT

Write the spec out to a portable file — two ways.

- With the server running (second Command Prompt) — pull it with curl
- Without a running server — generate it straight from code

Either way you get a portable openapi.json

```
curl http://127.0.0.1:8000/openapi.json -o openapi.json
```



© 2026 Innovation In Software Corporation

SYNCHRONICITY

Because it's generated, the spec can't drift from the code.

- The spec is produced fresh from the live code on every run — no manual editing
- Change a model or add an endpoint → the spec updates automatically
- Enrich the docs with summary, description, response_model, tags, and Field(...) metadata
- The docs stay accurate as the API evolves, because they *are* the code
- In contrast, hand-written docs inevitably fall out of date

© 2026 Innovation In Software Corporation

SWAGGER UI

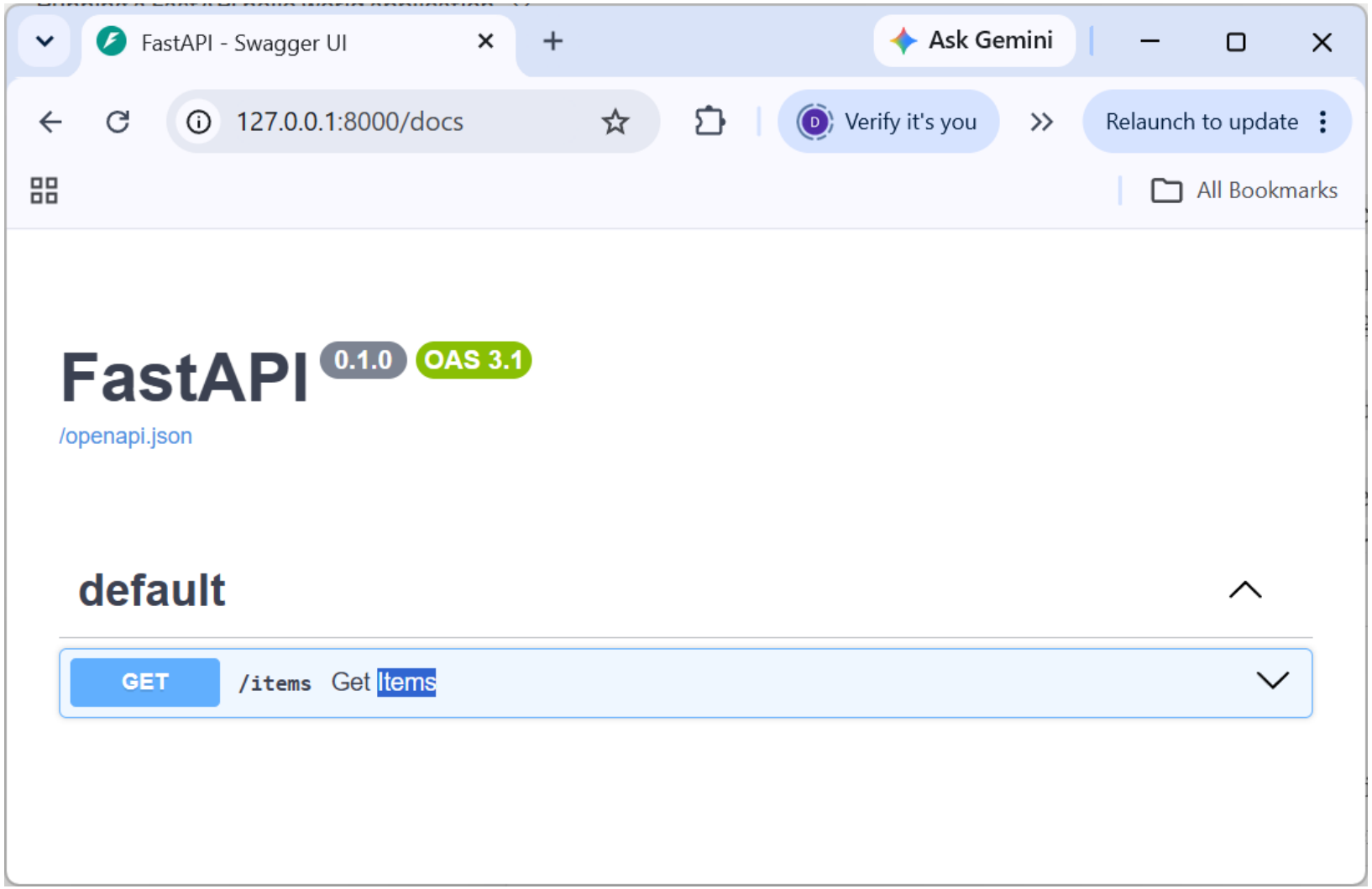
Swagger provides an interactive, in-browser documentation — generated from the spec.

- Reads your openapi.json and renders a live documentation page
- Sends real requests (including POST/PUT) straight from the browser, no curl needed
- Shows parameters, request bodies, response shapes, and status codes for every endpoint
- FastAPI serves it free at /docs

Turns the spec into something anyone on the team can explore and test

© 2026 Innovation In Software Corporation

SWAGGER UI - 2



© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned what the OpenAPI specification is, why it serves as a source of truth, and how contract-first development works from a YAML contract wired to an app. You saw how the spec is structured through its info, paths, components, and schemas sections.

You also documented request bodies, responses, parameters, and error codes, then generated a specification from code, saved the OpenAPI document, and reviewed the result in Swagger UI.

© 2026 Innovation In Software Corporation

LAB 4 – LOAN APPLICATION



GOAL



Turn a ready-made sample spec into a working Loan Applications API — no blank-page coding required.

- Edit the provided sample.yaml, following the [STEP N] markers, to reshape a Books API into a Loans API
- Write a short main.py with two GET routes and one POST route
- Run with fastapi dev main.py and test by clicking through Swagger UI

© 2026 Innovation In Software Corporation

SAMPLE.YAML

You are given you a file called sample.yaml that already describes a fully working "Books" API. Your whole job in the first part is to play find-and-replace: turn that Books API into a Loans API by swapping book words for loan words.

The file has little signposts in it like [STEP 1], [STEP 2], and so on — each one tells you exactly where to make a change. Work through them in order. Once the spec describes loans, you'll write some Python code to make it real, then open a friendly web page (Swagger) that lets you click buttons to test it.

© 2026 Innovation In Software Corporation

THE PLAN

What You're Doing and Why

- A "spec" is just a written description of what an API does — what you can ask it and what it sends back
- You'll start from a ready-made spec and reshape it, so the hard structural work is already done for you
- Think of it like editing a form letter: the layout is set, you're just changing the names and details
- By the end you'll have a real, running mini-application that handles loan applications

© 2026 Innovation In Software Corporation

THE STEPS

Open sample.yaml in your editor

- It currently describes a Books API
- Look for the [STEP N] comments; each marks one small change to make
- You're changing *words*, not *structure* — the shape of the file stays exactly the same
- Save often, and don't panic if it looks like a lot of text — you only touch the highlighted areas

© 2026 Innovation In Software Corporation

FILTER AND THE ID

Update the filter and the ID field to match loan data.

- [STEP 3] — Rename the availability filter to status, and change its allowed values to pending, approved, denied
- This is what lets someone list only the approved loans, for example
- [STEP 4] — Rename book_id to loan_id — this is the unique number used to look up one specific loan

Leave the type as integer (a whole number) — you're only changing the name

© 2026 Innovation In Software Corporation

THE RESPONSES

Confirm the response codes are in place and worded for loans.

- [STEP 5] — Check the numbered responses are still there: 201, 404, and 422
- 200 means "created successfully" — the loan was saved
- 404 means "not found" — someone asked for a loan ID that doesn't exist
- 422 means "you sent something invalid" — like text where a number belongs

You don't need to add these — just confirm they're present and the wording fits loans

© 2026 Innovation In Software Corporation

DATA MODEL

Reshape the data template so it describes a loan, not a book.

- [STEP 6] — Rename the Book template to LoanApplication
- This "template" describes what one loan looks like — its fields and their types
- Important: the name Book appears in three places (all marked [STEP 6]) — change all three
- Missing one is a common oversight, so double-check before moving on

© 2026 Innovation In Software Corporation

FIELDS

Define the loan's fields and mark which ones are mandatory.

[STEP 7] — Set the required fields (the ones that can't be left blank): applicant_name, amount, term_months

[STEP 8] — Rename each field to its loan version:

- title → applicant_name
- author → amount (and change its type to number, since money isn't a word)
- copies → term_months
- availability → status, with the default set to pending

New loans should start as pending automatically, so the person submitting doesn't have to set it

© 2026 Innovation In Software Corporation

CREATE FASTAPI APP

Turn your edited YAML into a working code skeleton.

- First install the tool (one time): pip install fastapi-code-generator
- Then generate the app from your spec:

```
fastapi-codegen --input sample.yaml --output app
```

- Look in the new app/ folder — you'll find main.py (the routes) and models.py (the LoanApplication model)

Hint: Notice the routes are already there, but their bodies (handlers) just say pass — that's your job next

© 2026 Innovation In Software Corporation

DATA

The generated app has routes but nowhere to keep data yet.

- Open app/main.py
- Near the top, add a simple in-memory list to act as your stand-in "database":

```
loans: list = []
```

- This holds loan applications while the app is running.

© 2026 Innovation In Software Corporation

IMPLEMENTATION

Replace each pass with real behavior — the one part only you can write.

- POST /loans — add the incoming application to the list and return it
- GET /loans — return the list; if a status was provided, return only the matching ones
- GET /loans/{loan_id} — find the loan by ID, or raise a 404 if it's not there

Hint: For the missing-ID case, use raise HTTPException(status_code=404) — FastAPI turns it into the 404 your spec promised

© 2026 Innovation In Software Corporation

IMPLEMENTATION - 2

Here are two routes with real logic worth showing.

```
...
@app.get("/loans")
def list_loans(status: Status | None = None):
    if status:
        return [l for l in loans if l.status == status]
    return loans


@app.get("/loans/{loan_id}")
def get_loan(loan_id: int):
    for l in loans:
        if l.loan_id == loan_id:
            return l
    raise HTTPException(status_code=404, detail="Loan not found")
```

The POST route is not completed— try it yourself first.

© 2026 Innovation In Software Corporation

TEST APPLICATION

Start the server and open the docs.

```
fastapi dev main.py
```

- Then open http://127.0.0.1:8000/docs — Swagger UI, generated automatically from your code and type hints.
- Test your app using the generated UI.

© 2026 Innovation In Software Corporation

Lab completed



FASTAPI CORE CONCEPTS

ASGI, FILTERING, AND MIDDLEWARE



INTRODUCTION



In this module, you will explore FastAPI's layered architecture, including ASGI in plain terms, the async event loop, and the worker thread pool that let it handle many requests at once.

You will then map URLs to functions, apply filtering, and learn what middleware is and how to implement it.

© 2026 Innovation In Software Corporation

FASTAPI – LAYERED ARCHITECTURE



FastAPI is a tool for building web APIs in Python — the behind-the-scenes services that apps and websites call to get data.

ASGI is the plumbing underneath it that lets FastAPI handle lots of requests at once without slowing down.

You write FastAPI. ASGI does the heavy lifting you never see.

© 2026 Innovation In Software Corporation

ASGI IN PLAIN ENGLISH

Think of a really good waiter at a busy restaurant.

- The old way: the waiter takes your order, then stands frozen at your table until the kitchen finishes cooking. Nobody else gets served while they wait.
- The ASGI way: the waiter takes your order, hands it to the kitchen, then serves other tables while your food cooks — and comes back when it's ready.

One great waiter keeps the whole room moving.

© 2026 Innovation In Software Corporation

ASGI IN PLAIN ENGLISH - 2

Waiter workflow

- The customer → the client (a browser or app asking for something)
- The order → the request ("give me this data")
- The waiter → the worker handling requests
- The kitchen → a slow step, like a database
- Waiting on food → the "await" — the worker helps others meanwhile
- Waiter delivers food → the response sent back

© 2026 Innovation In Software Corporation

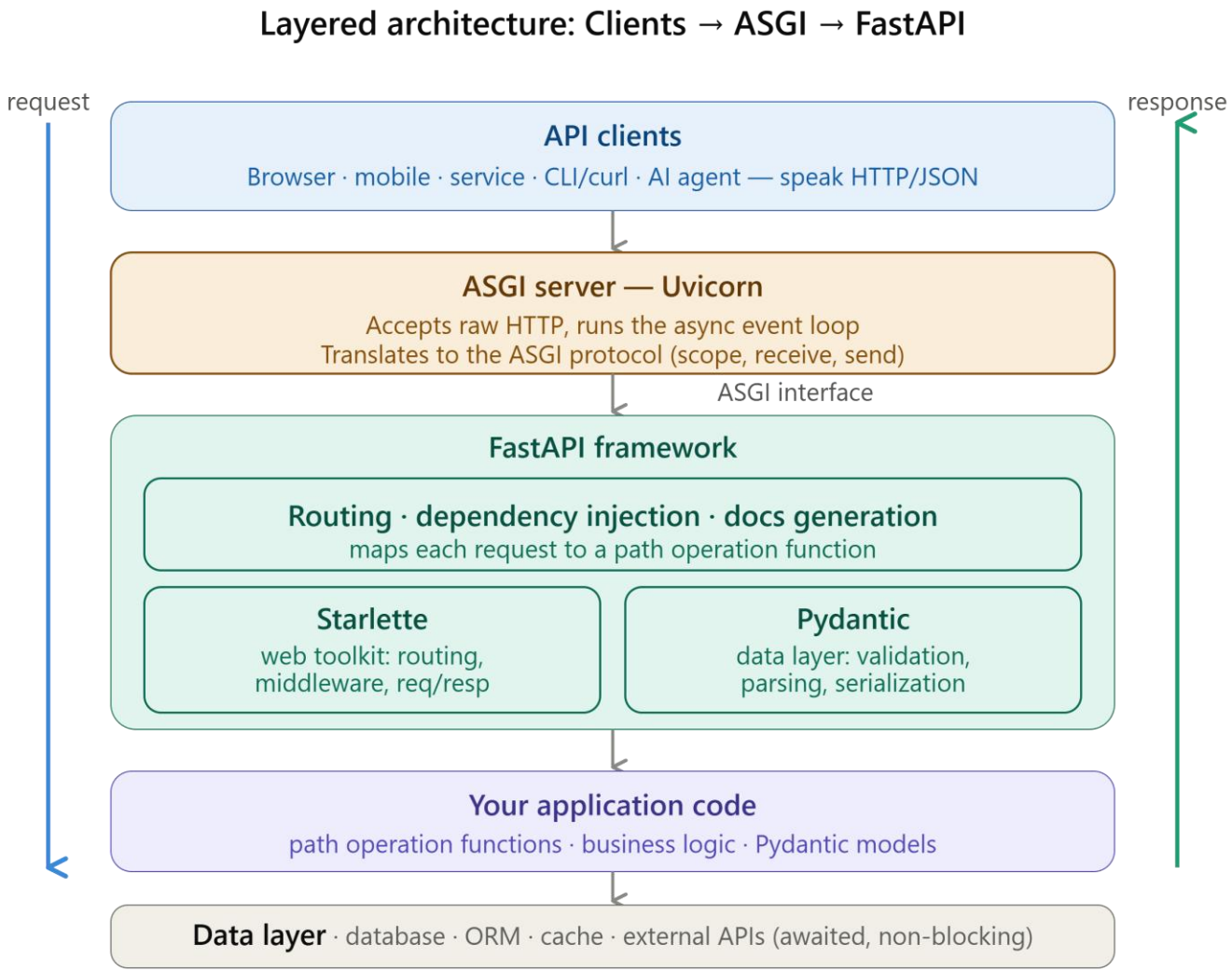
WHY IT MATTERS

Benefits to this framework:

- Serves more with less — one process handles thousands of requests, so you need fewer servers.
- Fast under load — waiting requests don't block everyone else, so the app stays snappy.
- Handles live features — chat, notifications, and live dashboards all work.
- One simple rule — use async def and await for slow work.

© 2026 Innovation In Software Corporation

FASTAPI – LAYERED ARCHITECTURE



© 2026 Innovation In Software Corporation

ASYNC EVENT LOOP

An event loop is a single worker that runs one task at a time but never sits idle. Whenever a task hits something slow, it steps aside and the worker picks up another task that's ready.

One worker, many tasks, no waiting around. When the slow thing finishes, the original task gets resumed right where it left off.

- async — marks a function as a coroutine: one that can pause and resume (but doesn't run anything by itself).
- await — the actual step-aside; frees the worker to do other things.
- task — a coroutine the loop is actively tracking and scheduling.

© 2026 Innovation In Software Corporation

WORKER THREAD POOL

When you write a FastAPI path operation with plain def instead of async def, FastAPI assumes it might *block* — calling something synchronous (an old database driver, a file read, the requests library) that freezes in place instead of stepping aside with await. Since the event loop runs on a single thread, letting that blocking call run directly would stall *every other request* until it finished.

So FastAPI automatically runs plain def handlers in a worker thread pool — a fixed set of helper threads kept ready in the background. The blocking call happens on a helper thread, off the event loop, leaving the loop free to serve everyone else. That's why plain def is a safe default: FastAPI quietly moves it off the main thread for you. To send a specific blocking call to the pool from inside an async function, use run_in_threadpool.

© 2026 Innovation In Software Corporation

WORKER THREAD POOL - 2

Two limits follow from how threads work.

First, the pool is finite — if many blocking requests arrive at once and all helpers are busy, new ones wait for one to free up; it's great for occasional blocking work but no substitute for proper async libraries.

Second, threads help with *waiting*, not *speed*: Python's Global Interpreter Lock (GIL) lets only one thread run code at a time, so the pool is perfect for blocking I/O but does not speed up heavy CPU work. For that you need a separate process — a ProcessPoolExecutor or a worker like Celery — which runs on its own core and sidesteps the GIL.

© 2026 Innovation In Software Corporation

WORKER THREAD POOL - 3

Event loop	Worker thread pool	Separate process
async def	**plain def**	**CPU-bound work**
Single thread, one core	Finite set of helper threads	Own interpreter, own core
Handlers that await database or API calls	Blocking calls run here, off the event loop	Number-crunching that would hold the GIL
Interleaves many requests at once	One slow call can't freeze the server	Runs in true parallel, past Python's GIL
Best for: async I/O	**Best for: blocking I/O**	Best for: heavy computation

© 2026 Innovation In Software Corporation

SAMPLE CODE

```python
import asyncio

from fastapi import FastAPI

app = FastAPI()

async def fetch_account_balance(account_id: int) -> float:
    await asyncio.sleep(2)
    return 1000.0 + account_id

@app.get("/balance/{account_id}")
async def get_balance(account_id: int):
    # await = step aside while waiting
    balance = await fetch_account_balance(account_id)
    return {"account_id": account_id, "balance": balance}
```

© 2026 Innovation In Software Corporation

MAPPING A URL TO A FUNCTION

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items")
async def list_items():
    return [{"id": 1}]
```

Routing is how FastAPI handles mapping a URL to a function. The decorator carries an HTTP method and a path that together form the address that triggers your function. With @app.get("/items"), .get answers GET requests and /items matches literally — no placeholder, so the function takes no arguments from the URL.

Example: a request to GET /items maps that URL to list_items().

URL: http://127.0.0.1:8000/items

© 2026 Innovation In Software Corporation

MAPPING A URL TO A FUNCTION

```python
from fastapi import FastAPI


app = FastAPI()


@app.get("/items/{item_id}")
async def get_item(item_id: int):
    return {"item_id": item_id}
```

Routing is how FastAPI handles mapping a URL to a function. The decorator carries an HTTP method and a path that trigger your function. With @app.get("/items/{item_id}"), .get answers GET requests, /items/ matches literally, and {item_id} captures whatever the client puts there — passed into your function by the matching argument name and converted to the declared type (int).

Because item_id sits inside the path, FastAPI reads it as a path parameter — part of the URL path itself, not a query string (no ?item_id=42).

URL: http://127.0.0.1:8000/items/42

© 2026 Innovation In Software Corporation

MAPPING A URL TO A FUNCTION

```python
from fastapi import FastAPI


app = FastAPI()


@app.get("/items")
async def list_items(
    skip: int = 0,
    limit: int = 10,
    q: str | None = None):
    ...
```

Routing is how FastAPI handles mapping a URL to a function. The decorator's method and path trigger your function: with @app.get("/items"), .get answers GET requests and /items matches literally. The function arguments — skip, limit, q — aren't in the path, so FastAPI reads them as query parameters: values after the ?. Each is converted to its declared type, and since all three have defaults, they're optional

URL: http://127.0.0.1:8000/items?skip=0&limit=5&q=book

© 2026 Innovation In Software Corporation

FILTERING

Filtering is what happens when you declare a response_model on a route. FastAPI runs whatever your function returns through that model on the way out, keeping only its declared fields and silently dropping the rest. In the example, save() returns id, name, price, and password, but because ItemOut lists only id and name, that's all the client receives — price and password never leave the server.

This makes filtering the safe way to hide sensitive data. You don't have to strip out passwords or internal fields by hand in every function — you declare the shape you want to send back, and FastAPI enforces it every time. No matter what your code accidentally returns, only the declared fields go out, so secrets can't leak through a forgotten line.

© 2026 Innovation In Software Corporation

FILTERING - EXAMPLE

```python
class Item(BaseModel):
    name: str
    price: float
    password: str

class ItemOut(BaseModel):
    id: int
    name: str
    # note: no password field

@app.post("/items", response_model=ItemOut)
async def create(item: Item):
    return save(item)
```

Filtering here comes from the deviation between the two models. Item is the input — the client sends name, price, and password. ItemOut is the output, listing only id and name. When create() returns the saved object (which still holds price and password), FastAPI runs it through ItemOut and drops everything not declared there — so the sensitive password never reaches the client, with no stripping code on your part.

© 2026 Innovation In Software Corporation

MIDDLEWARE



Middleware is a checkpoint that every request passes through on its way in and on its way out.

Picture your FastAPI app as a building. Your routes (the actual code that answers requests) are the offices inside. Middleware is the security desk at the front entrance. Every visitor must pass the desk on the way in, and pass it again on the way out. No one skips it.

© 2026 Innovation In Software Corporation

MIDDLEWARE - 2

That's the whole idea: middleware is code that runs on every single request automatically, so you don't have to copy the same logic into each route.

Why bother? Because some jobs apply to *all* requests, not just one. For example:

- Stamp each request with an ID (so you can trace it in logs)
- Time how long each request takes
- Check if the visitor is logged in
- Add security headers to every response

Instead of writing that logic into all fifty of your routes, you write it once in middleware. It sits in front of everything.

© 2026 Innovation In Software Corporation

MIDDLEWARE - IMPLEMENTATION

The salient fact to know is the call_next function. Middleware runs in two halves:

[middleware: stuff on the way IN]

↓ call_next(request)

[your actual route runs]

↓ (response comes back)

[middleware: stuff on the way OUT]

Everything before call_next happens as the request enters. call_next hands the request off to your route. Everything after call_next happens once the route has produced a response, before it goes back to the client.

© 2026 Innovation In Software Corporation

MIDDLEWARE - EXAMPLE

```
@app.middleware("http")
async def log_timing(request: Request,
                     call_next):
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start
    print(
        f"{request.method} {request.url.path} "
        f"took {duration:.3f}s"
    )
    return response
```

Explanation:

- @app.middleware("http") — Registers the function below as middleware for HTTP requests.
- async def log_timing(request, call_next): — The middleware function.
- start = time.time() — On the way *in*, record the current time. This is the stopwatch starting.

© 2026 Innovation In Software Corporation

MIDDLEWARE – EXAMPLE - 2

```python
@app.middleware("http")
async def log_timing(request: Request,
                     call_next):
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start
    print(
        f"{request.method} {request.url.path} "
        f"took {duration:.3f}s"
    )
    return response
```

response = await call_next(request) — Hand the request off to the route, wait for it to finish, and get the response back.

duration = time.time() - start — On the way *out*, check the time again and subtract the start time. That's how long the whole thing took.

return response — Send the response back out to the client.

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you explored FastAPI's layered architecture, ASGI, the async event loop, and the worker thread pool behind its concurrency. You saw how URLs map to functions and how filtering is applied.

You also learned what middleware is, how to implement it, and where it fits in the request pipeline.

© 2026 Innovation In Software Corporation

LAB 5 – MIDDLEWARE AND FILTERING



In this lab you'll add two pieces to a working FastAPI app: a middleware that counts requests, and a filter that keeps a secret token out of the response. By the end, /now returns the time plus a running count — but never the token.

GOAL



The starter route /now returns a TimeReading model with a timestamp and a secret token — and right now the whole thing goes out, token and all.

- Task 1 adds a middleware that increments and logs a counter on every request.
- Task 2 splits the model so only timestamp and a new count are returned.

The middleware produces the count, and the filtered model is what safely sends it out.

© 2026 Innovation In Software Corporation

RUN THE APPLICATION

Open a terminal in the folder with lab5\main.py.

Start the app: fastapi dev main.py

Visit http://127.0.0.1:8000/now and note that token shows up — that's the leak. Leave the server running; it auto-reloads on save.

```
{
  "timestamp": "2026-08-11T23:16:19.797539+00:00",
  "token": "super-secret-do-not-leak"
}
```


© 2026 Innovation In Software Corporation

COUNTING

The counter variable request_count = 0 is already in the starter code, just under app = FastAPI(). Below the route, complete this middleware:

```
@app.middleware("http")
async def count_requests(request, call_next):
    global request_count
    # your code here
```

Inside the function, you need to:

- On the way in: increment request_count by one, then log the new total (a print is fine).

© 2026 Innovation In Software Corporation

COUNTING - 2

- Hand the request to the route with call_next and capture the response it returns.
- On the way out: attach the total to the response as an X-Request-Count header.
- Return the response.

Save, refresh /now a few times, and watch the terminal total climb.

© 2026 Innovation In Software Corporation

FILTER

Add an output model with no token field:

```
class TimeReadingOut(BaseModel):
    timestamp: str
    count: int
```

Add the filter to the decorator: @app.get("/now", response_model=TimeReadingOut)

Update the route to include the current request_count as count when it builds the returned object. The internal object can still carry token — the filter drops it.

Save.

© 2026 Innovation In Software Corporation

VERIFY

Refresh /now a few times.

Confirm the response shows timestamp and count, token is gone, and count matches the terminal total.

© 2026 Innovation In Software Corporation

Lab completed



ERROR HANDLING

VALIDATION AND EXCEPTION HANDLING



INTRODUCTION



In this module, you will learn to validate input at the boundary of your app, so bad data is rejected before your function code runs, and build robust error handling in FastAPI.

You will raise a custom AppError, return specific error codes with a consistent structured shape, add a global exception handler, and trace requests through middleware.

© 2026 Innovation In Software Corporation

VALIDATION AT THE BOUNDARY

Validation is FastAPI's first line of defense — it runs *before* your function code executes. When a request arrives, FastAPI checks it against the shape you declared and rejects anything that doesn't fit, right at the boundary of your app. Your function only runs if the data is already valid, so bad input never reaches your logic.

This works because you declared the request body as a Pydantic model.

© 2026 Innovation In Software Corporation

VALIDATION AT THE BOUNDARY

In our example, price is typed as a float, so FastAPI expects a number. Let us assume, that the client sends a POST to /items with "price": "free" — text where a number is required. FastAPI catches the mismatch immediately and never calls your function. Instead it returns a 422 Unprocessable

Entity response with a structured error that pinpoints what went wrong:

- loc shows where the bad value is (body → price)
- msg explains the problem in plain language,
- type gives the machine-readable category (float_parsing).

© 2026 Innovation In Software Corporation

VALIDATION - EXAMPLE

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.post("/items")
async def create(item: Item):
    # This line only runs if validation passed
    return {"received": item}
```

The key takeaway is that you wrote no validation code to make this happen. You didn't check the type, craft the error message, or set the status code — declaring the model was enough.

FastAPI guarantees that by the time your function runs, the data is already the right shape.

© 2026 Innovation In Software Corporation

ROBUST ERROR HANDLING IN FASTAPI



Four patterns for predictable, debuggable APIs:

- Standard error envelope
- Stable error codes
- Global exception handler
- Per-request tracing

© 2026 Innovation In Software Corporation

ROBUST ERROR HANDLING IN FASTAPI - 2

Why This Matters?

- Clients need errors that are consistent and machine-readable — not a different shape every time.
- Support teams need a way to pinpoint the exact failure from a single ID.
- Unhandled exceptions should never leak stack traces or crash the response.
- Good error handling is an API contract, just like your success responses.

© 2026 Innovation In Software Corporation

ROBUST ERROR HANDLING IN FASTAPI - 2

Every error returns the same JSON shape, produced by an @app.exception_handler(AppError) handler. It is called whenever an AppError is raised anywhere in your route or dependency code), so clients write one parser instead of many.

```
{
  "error": {
    "code": "ACCOUNT_NOT_FOUND",
    "message": "No account exists with that ID.",
    "request_id": "a1b2c3d4-...",
    "details": []
  }
}
```

© 2026 Innovation In Software Corporation

RAISING APPERROR

When something goes wrong, you create an AppError and raise it. The three arguments bundle everything the response will need:

```
raise AppError(
    ErrorCode.ACCOUNT_NOT_FOUND,   # code     -> self.code
    "No account exists with that ID.",  # message  -> self.message
    status_code=404,               # status_code -> self.status_code
)
```

That __init__ just stores the three values on the instance (self.code, self.message, self.status_code) so they can be read back later. Because status_code has a default of 400, you can omit it for plain bad-request errors:

© 2026 Innovation In Software Corporation

RAISING APPERROR - 2

Raising the exception interrupts the route immediately — nothing after the raise runs. FastAPI then looks for a registered handler for that exception type and hands it the instance as exc. The handler reads the stored attributes back off it:

```
@app.exception_handler(AppError)
async def app_error_handler(request: Request, exc: AppError):
    return JSONResponse(
        status_code=exc.status_code,    # reads what you passed in
        content={"error": {
            "code": exc.code,           # reads what you passed in
            "message": exc.message,     # reads what you passed in
        }},
    )
```

© 2026 Innovation In Software Corporation

RAISING APPERROR - BENEFITS

- The class is the bridge between where an error happens (deep in a route or service function) and where it becomes an HTTP response (one central handler).
- Instead of every route building its own JSONResponse with the right status and JSON shape, each route just raises AppError with the facts, and the single handler guarantees they all come out in the identical envelope.
- You get one consistent error format across the whole API, defined in exactly one place.

© 2026 Innovation In Software Corporation

SPECIFIC ERROR CODES

Machine-readable strings that name exactly what went wrong.

- Specific: INSUFFICIENT_FUNDS, not a generic 400.
- Stable: once published, a code is never renamed — clients depend on it.
- Decoupled from HTTP status: many errors can share status 409 but have distinct codes.
- Centralize them (an Enum or constants module) so they're consistent across the app.

```
class ErrorCode(str, Enum):
    ACCOUNT_NOT_FOUND = "ACCOUNT_NOT_FOUND"
    INSUFFICIENT_FUNDS = "INSUFFICIENT_FUNDS"
    VALIDATION_ERROR = "VALIDATION_ERROR"
```

© 2026 Innovation In Software Corporation

STRUCTURED ERRORS

A custom exception carries the code, status, and message together.

```
class AppError(Exception):
    def __init__(self, code: ErrorCode, message: str,
                 status_code: int = 400):
        self.code = code
        self.message = message
        self.status_code = status_code
```

© 2026 Innovation In Software Corporation

GLOBAL EXCEPTION HANDLER

The @app.exception_handler(Exception) handler turns any raised error into the standard envelope. Anything unexpected becomes a safe 500 — no leaked internals.

```
@app.exception_handler(Exception)
async def unhandled_handler(request, exc):
    # log the real exception here
    return JSONResponse(
        status_code=500,
        content={"error": {
            "code": "INTERNAL_ERROR",
            "message": "An unexpected error occurred.",
            "request_id": request.state.request_id,
        }},
    )
```

© 2026 Innovation In Software Corporation

REQUEST TRACING

Request tracing with a unique ID solves a simple but painful problem: when something goes wrong, how do you connect what the *client* saw to what actually happened on the *server*?

A user says "my request failed at 2pm" — but your server handled thousands of requests around 2pm. Without



REQUEST TRACING - 2



The idea is to stamp every incoming request with a unique ID the moment it arrives, then make that same ID appear everywhere the request leaves a trace: every log line, the error response sent back to the client, and a readable response header.

That gives you one thread connecting all three — the client quotes the ID from their failed response, you search your logs for it, and you see exactly what their request did.

© 2026 Innovation In Software Corporation

REQUEST TRACING - MIDDLEWARE

Middleware is the right tool here because request tracing needs to apply to every request the same way — and middleware is FastAPI's mechanism for wrapping logic around the entire request/response cycle, running before your route handler and again after it, on every call automatically.

That "before and after, every time" shape is exactly what an ID needs. You want to mint the ID before the handler runs (so it's available while the handler does its work and writes logs), and attach it to the response after the handler finishes (so the client gets it back). A middleware function sits at precisely that seam.

© 2026 Innovation In Software Corporation

REQUEST TRACING - IMPLEMENTATION

That "before and after, every time" shape is exactly what an ID needs. You want to mint the ID *before* the handler runs (so it's available while the handler does its work and writes logs), and attach it to the response *after* the handler finishes (so the client gets it back). A middleware function sits at precisely that seam:

```
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())  # before: mint the ID
    request.state.request_id = request_id  # before: stash it where handlers can reach it
    response = await call_next(request)  # the handler runs here
    response.headers["X-Request-ID"] = request_id  # after: send it back
    return response
```


© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned how validation at the boundary rejects bad input before it reaches your logic, and how to handle errors robustly in FastAPI. You raised a custom AppError, returned specific codes, and produced structured error responses through a global exception handler.

You also added request tracing with middleware, making failures easy to correlate across a request.

© 2026 Innovation In Software Corporation

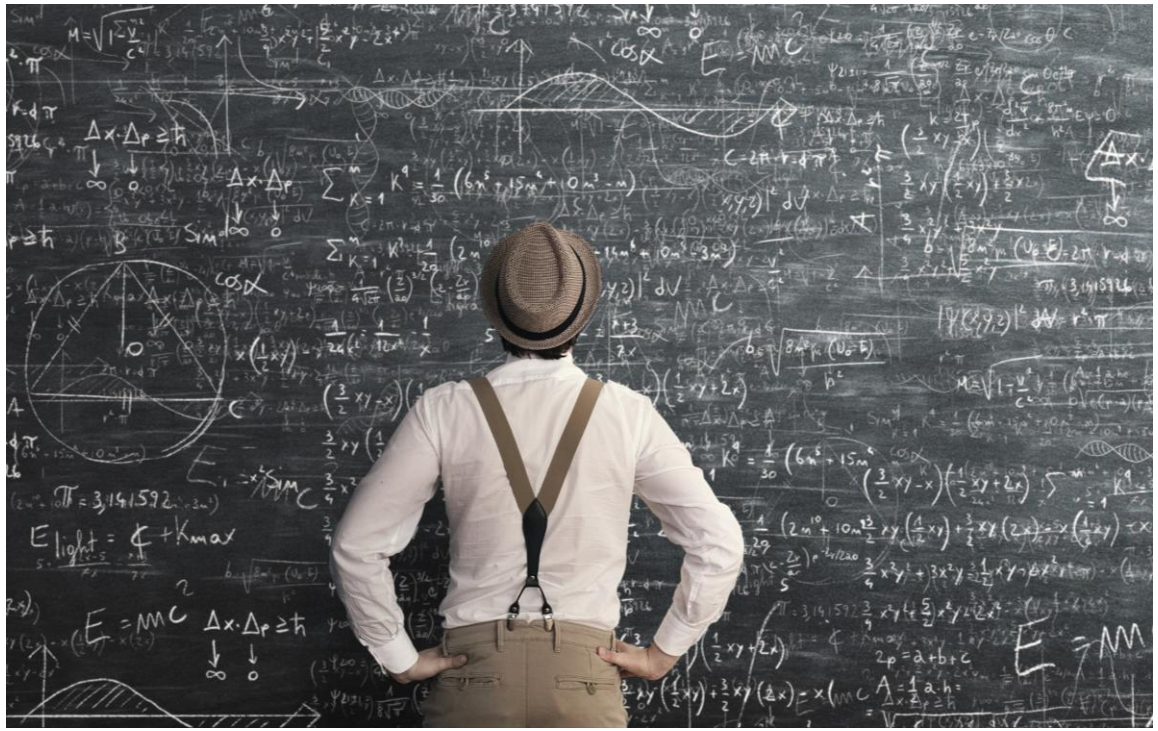
LAB 6 – MATH





The starter app has four routes — /add, /subtract, /multiply, and /divide — each taking query params a and b. They all work, but /divide?a=8&b=0 crashes with an unhandled ZeroDivisionError. The ErrorCode enum and AppError class are provided.

GOAL



This lab includes two tasks:

- Task 1 (tracing) is a middleware that mints a unique ID per request and returns it as the X-Request-ID header.
- Task 2 (errors) guards the divide route to raise AppError on a zero divisor, then registers a handler that turns it into the standard envelope.

These tasks are connected. The middleware produces the ID, and the handler reads it back so the client's error carries the same ID as your logs.

© 2026 Innovation In Software Corporation

RUN APPLICATION

Run it and see the problem

- Open a terminal in the folder with lab6\main.py.
- Start the app: fastapi dev main.py
- Confirm the happy path works: visit http://127.0.0.1:8000/add?a=4&b=2 — you should get a result of 6.
- Now trigger the bug: visit http://127.0.0.1:8000/divide?a=8&b=0. You'll get a 500 Internal Server Error and a stack trace in the terminal. That's what you'll fix. Leave the server running; it auto-reloads on save.

© 2026 Innovation In Software Corporation

TASK 1

For Task 1, complete the middleware so that, on every request, it:

- Before the handler: mints an ID with str(uuid.uuid4()).
- Before the handler: stashes it on request.state.request_id so routes and handlers can reach it.
- Hands the request to the route with call_next and captures the response.
- After the handler: attaches the ID to the response as the X-Request-ID header, then returns the response.

Save, then reload any route (e.g. /add?a=4&b=2) and check the response headers — you should see an X-Request-ID value that changes each request.

© 2026 Innovation In Software Corporation

TASK 2

Handle Divide-by-Zero with an Exception Handler

- In the divide route, check whether b is zero *before* dividing. If it is, raise a structured AppError instead of letting Python throw ZeroDivisionError.
- Register a handler with @app.exception_handler(AppError). FastAPI calls it automatically whenever an AppError is raised, and hands it the exception as exc.
- Inside the handler, build the standard error envelope: read code, message, and status_code off exc, and pull the correlated ID from request.state.request_id so the response matches the X-Request-ID header.

© 2026 Innovation In Software Corporation

CONFIRMATION

Verify the correlated ID. Visit http://127.0.0.1:8000/divide?a=8&b=0 again. Confirm you now get a clean 400 with a JSON error envelope — no stack trace:


```
{
  "error": {
    "code": "DIVIDE_BY_ZERO",
    "message": "Cannot divide by zero.",
    "request_id": "a1b2c3d4-..."
  }
}
```


Also confirm the correlation: the request_id in the body matches the X-Request-ID response header. That single ID is what a support team would use to find this exact request in the logs.

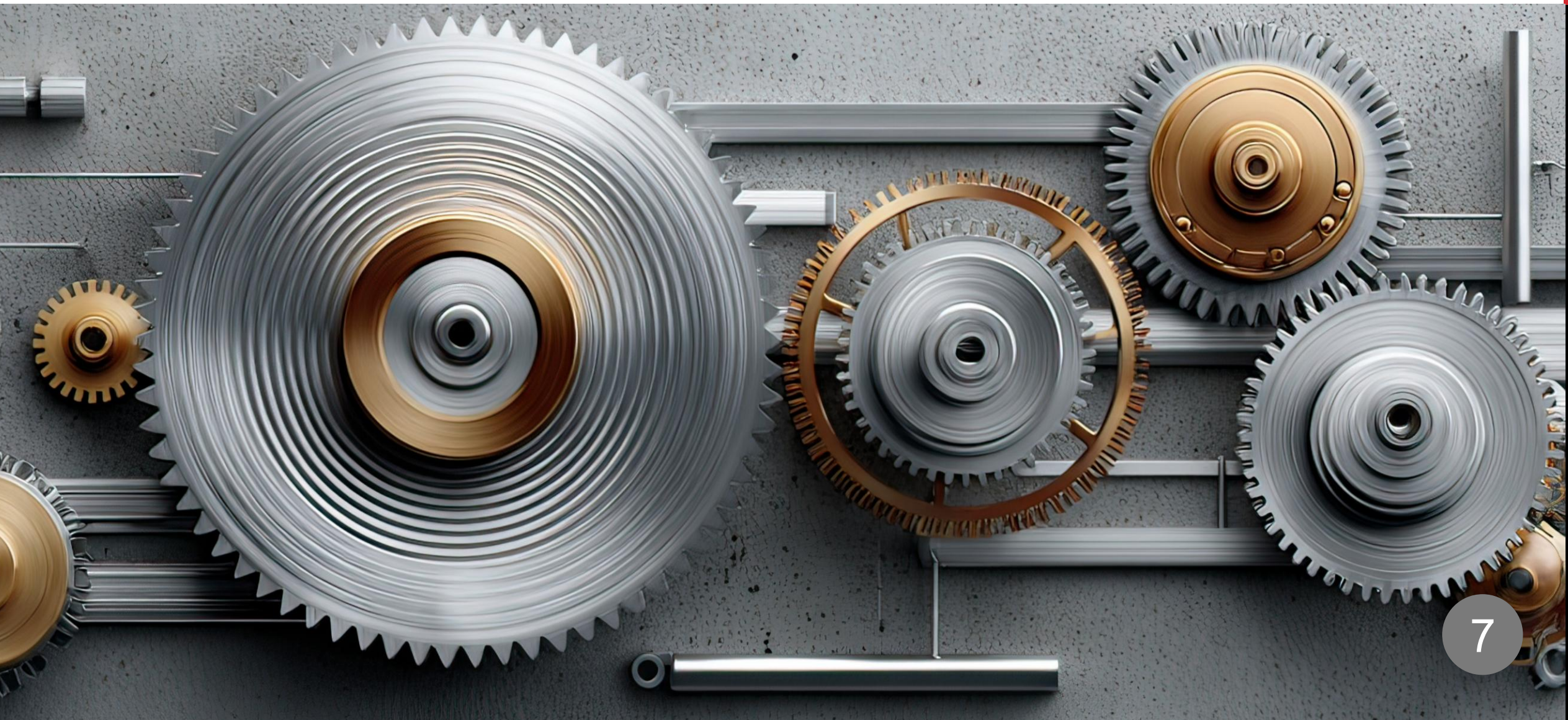
© 2026 Innovation In Software Corporation

Lab completed



SECURITY

JSON WEB TOKENS



INTRODUCTION



In this module, you will learn JSON Web Tokens — their structure, why they are signed rather than encrypted, and how HS256 and RS256 signatures work. You will follow the RS256 login flow, token verification, .well-known discovery, JWKS, key IDs, and key rotation.

You will then secure FastAPI with bearer tokens using Depends and appropriate status codes, and walk through a full JWT implementation.

© 2026 Innovation In Software Corporation

JSON WEB TOKEN (JWT)

Modern applications often need a safe way to let a user or application access an API. One common approach is to use a JSON Web Token (JWT). A JWT can be thought of as a temporary digital security badge. A trusted authorization server issues the token after the user or application has successfully authenticated.

The client carries the JWT and presents it to an API. The API does not simply trust the token because it exists. It verifies that the token was issued by a trusted system, has not been changed, has not expired, is intended for that API, and contains the required permissions.

© 2026 Innovation In Software Corporation

JSON WEB TOKEN (JWT)

Player	Plain-English Role
User	The person trying to use the application.
Client	The application the user is using. It receives and carries the JWT.
Authorization Server	The trusted security desk. It authenticates the user and issues the token.
API	The protected service. It verifies the token before allowing access.

Authorization server = security desk. JWT = temporary badge. Client = badge carrier. API = secured door.

© 2026 Innovation In Software Corporation

JWT STRUCTURE

In practical terms, it is a compact package of information that can be digitally signed. After login, the authorization server can issue a JWT that says things such as who the token is for, who issued it, what the holder is allowed to do, and when the token expires.

A JWT normally has three logical parts:

- Header — identifies information about the token, including the signing algorithm.
- Payload — contains claims, such as issuer, audience, expiration time, user or client identity, and permissions.
- Signature — allows the receiver to detect whether the token was changed and to verify that it was signed with the expected key.

© 2026 Innovation In Software Corporation

NOT ENCRYPTED



A signed JWT is usually not encrypted. Its contents may be readable. The signature protects integrity and authenticity, not secrecy.

© 2026 Innovation In Software Corporation

SIGNATURE

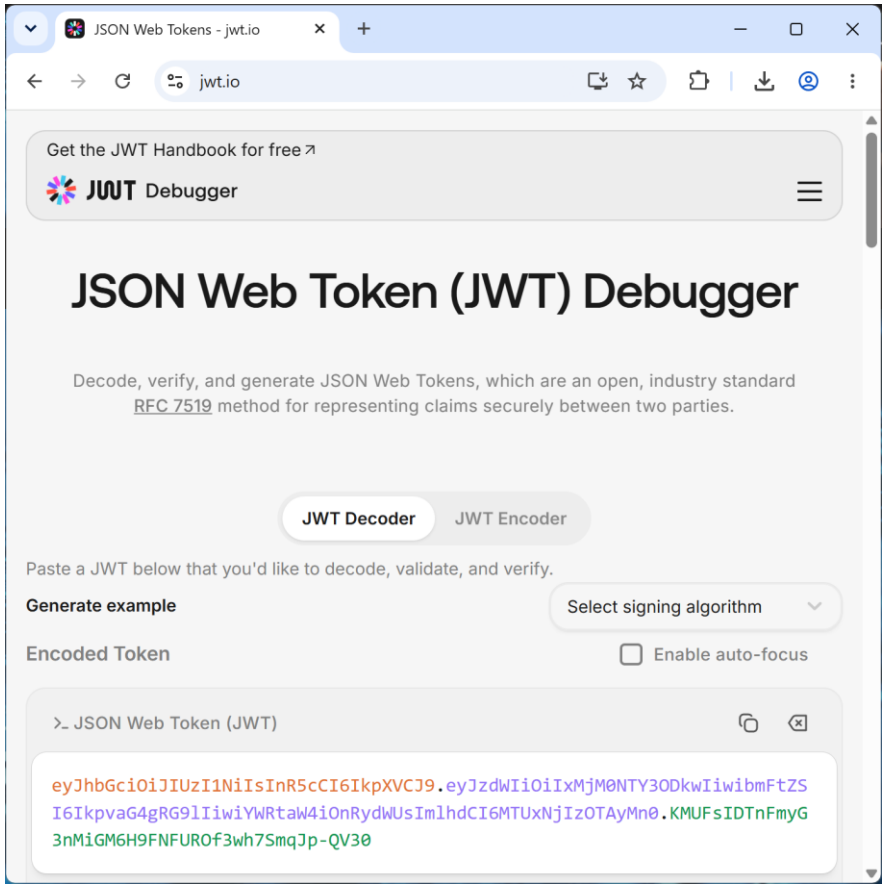


A signature is a mathematical proof attached to the JWT. It lets the API check that the token was created by a trusted party that possessed the correct signing key and that the signed contents have not been altered.

If someone changes a claim from “role = user” to “role = administrator,” the original signature will no longer match. The API should reject the modified token.

© 2026 Innovation In Software Corporation

JWT.IO



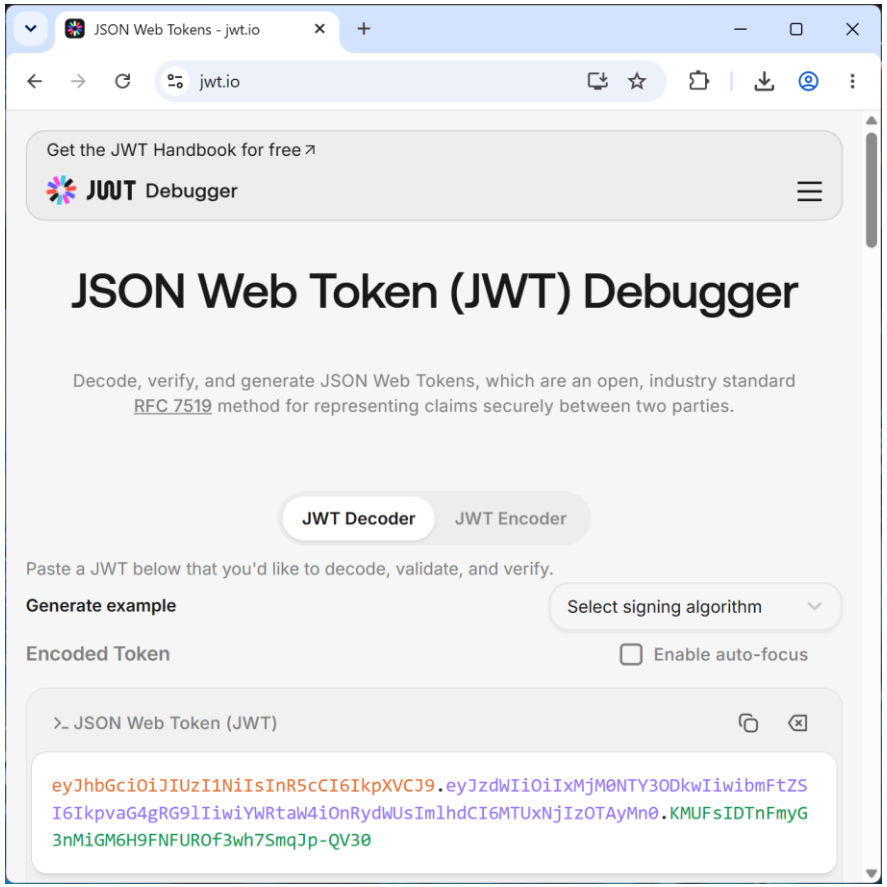
JWT.io is a tool for managing tokens.

Paste any JWT and it instantly decodes the header, payload, and signature into readable JSON.

You can also build and sign your own tokens right in the browser. It's the fastest way to understand what's actually inside a token — no code required.

© 2026 Innovation In Software Corporation

JWT.IO



Never put sensitive data in the payload — it's encoded, not encrypted. Anyone can decode it.

© 2026 Innovation In Software Corporation

HS256

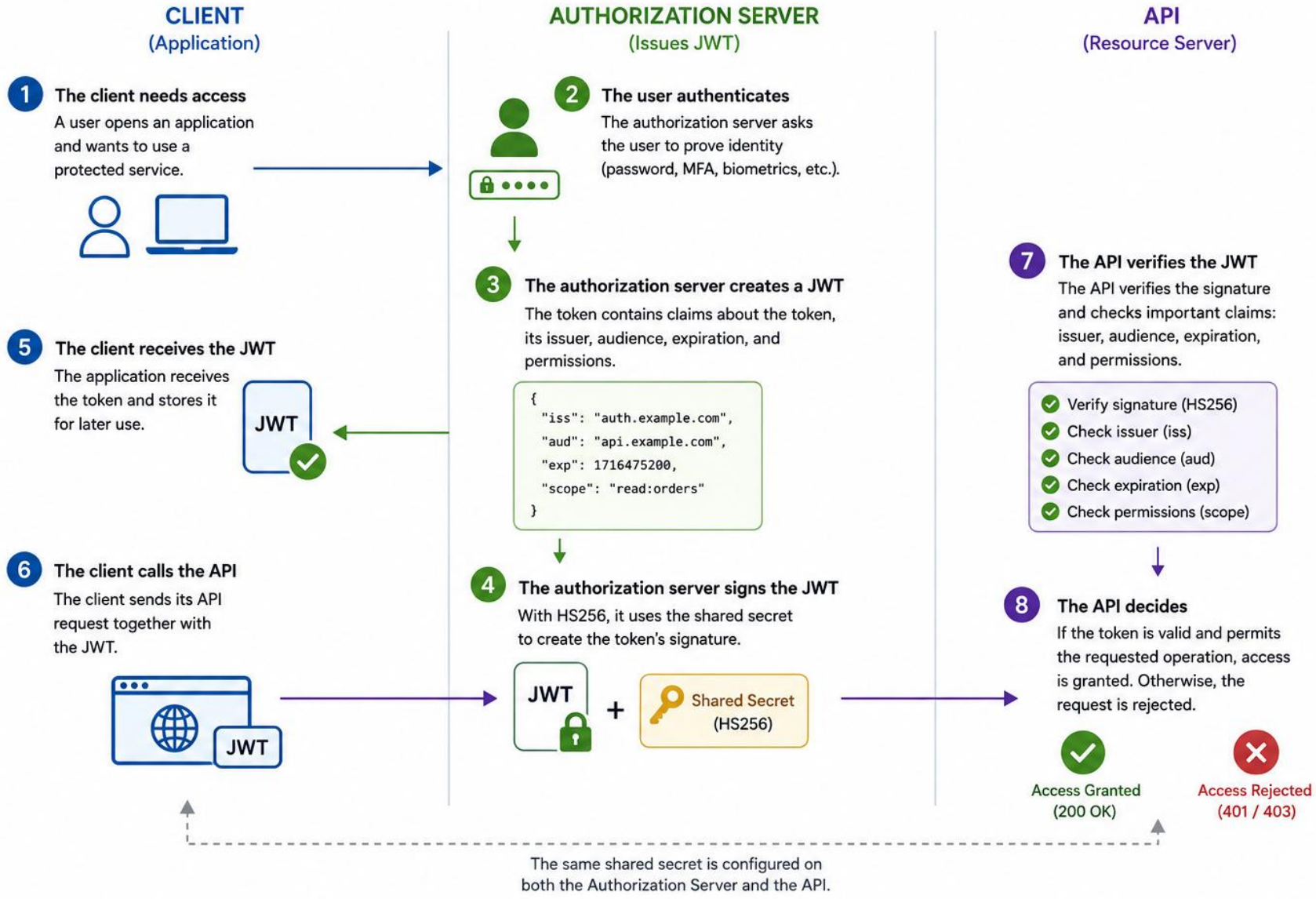
HS256 is one way to sign a JWT. It uses one shared secret. The authorization server uses that secret to create the signature, and the API uses the same secret to verify the signature.



Think of the shared secret as a password known to both systems. If the secret is stolen, an attacker may be able to create tokens with signatures that appear valid. For that reason, the secret must be strongly protected.

© 2026 Innovation In Software Corporation

HS256



© 2026 Innovation In Software Corporation

RS256

RS256 is a way to prove that a JWT really came from a trusted Authorization Server and was not changed afterward.

The important idea is that RS256 uses two keys:

- Private key — kept secret by the Authorization Server.
- Public key — given to APIs so they can verify JWTs.

Think of the private key as a special stamp that only the Authorization Server owns.

The public key lets anyone check the stamp, but it cannot be used to create the stamp.

© 2026 Innovation In Software Corporation

RS256 – USER LOGIN

Before the API can verify a JWT, the process begins with the user requesting access and the Authorization Server creating and signing the JWT.

- User needs access — The user opens an application and tries to use a protected service.
- User logs in — The Authorization Server verifies the user's identity using a password, MFA, biometrics, or another method.
- JWT is created — The Authorization Server creates a JWT containing information such as who issued it, who it is for, when it expires, and what the user can do.
- JWT is signed — The Authorization Server signs the JWT using its private key.

The private key stays secret with the Authorization Server.

© 2026 Innovation In Software Corporation

RS256 – USER LOGIN

Once the Authorization Server has created and signed the JWT, the application can use that JWT to request access to a protected API.

Client sends the JWT to the API:

- Client receives the JWT — After login, the application receives the signed JWT.
- Client calls the API — The application sends the JWT along with its API request.
- API needs to verify it — The API must make sure the JWT really came from the trusted Authorization Server and was not changed.

© 2026 Innovation In Software Corporation

JWT VERIFIED

Assuming that the API has the correct public key, it can verify the JWT and decide whether the requested access should be allowed.

- Verify the signature — The API uses the public key to verify the JWT's signature.
- Check the JWT — The API checks important information such as the issuer, audience, expiration, and permissions.
- Make a decision — If the JWT is valid and allows the requested operation, the API grants access.
- Reject invalid requests — If verification fails or permission is missing, access is denied.

© 2026 Innovation In Software Corporation

.WELL-KNOWN DISCOVERY

.well-known helps an application or API find important information about the Authorization Server.

Think of it as an information page published by the Authorization Server. Instead of the API being told where everything is located, it can look at this page.

For example, .well-known can tell the API where to find the public keys used to verify JWTs.

A common address ends with: .well-known/openid-configuration

© 2026 Innovation In Software Corporation

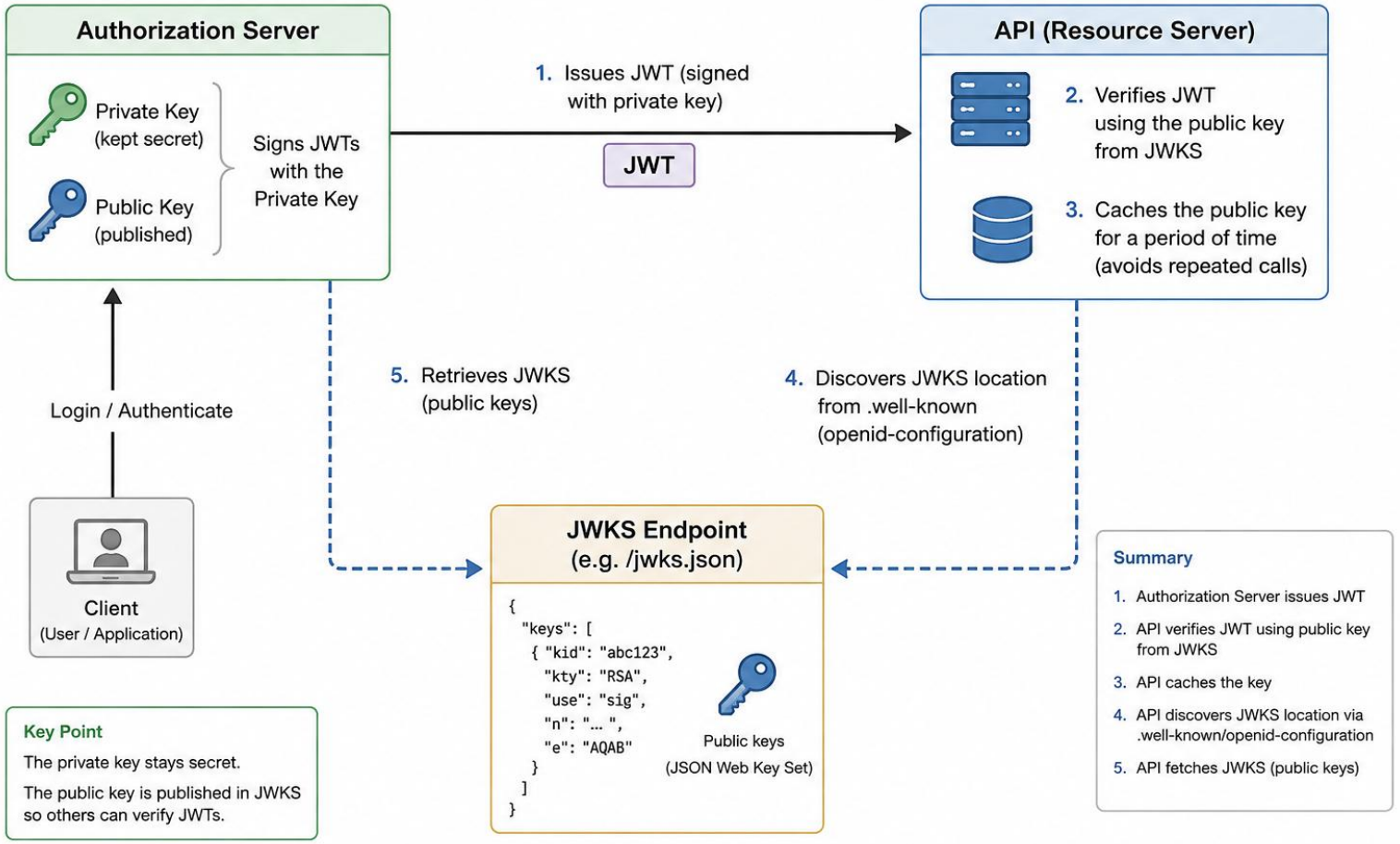
JSON WEB KEY SET

JWKS stands for JSON Web Key Set. It is a standardized way for an authorization server to publish public verification keys. This is especially relevant when public/private key signing algorithms are used.

The discovery document can point the API to the JWKS location. The API can obtain the appropriate public key there and cache it for verification.

© 2026 Innovation In Software Corporation

JSON WEB KEY SET - 2



© 2026 Innovation In Software Corporation

KIDS



An Authorization Server may publish multiple public keys in its JWKS. The API needs to know which one was used to sign a particular JWT.

The JWT header contains a kid (Key ID) that identifies the correct key.

Think of kid as a key number.

1. The API reads the kid from the JWT.
2. The API gets the list of public keys from JWKS.
3. It finds the public key with the matching kid.
4. It uses that public key to verify the JWT signature.

© 2026 Innovation In Software Corporation

KIDS - 2



The primary reason for kid (Key ID) is to tell the API which public key to use to verify the JWT.

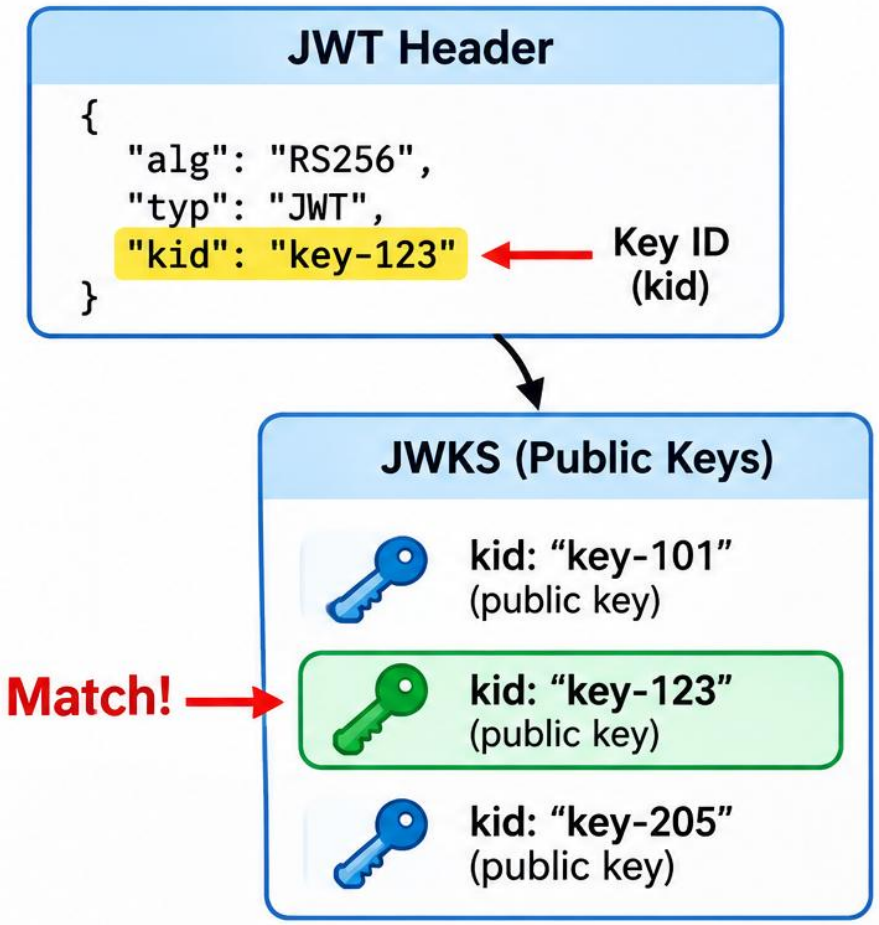
An Authorization Server may have multiple public keys in its JWKS, especially during key rotation.

Without kid, the API might have to try multiple keys to determine which one works.

© 2026 Innovation In Software Corporation

HOW KIDS WORKS?

The Key ID tells the API which public key to use.



1. Read the `kid` from the JWT header.
2. Find the public key in the JWKS whose `kid` matches.
3. Use that public key to verify the JWT signature.

Result: The API uses the matching public key (key-123) to verify the JWT.

© 2026 Innovation In Software Corporation

KEY ROTATION

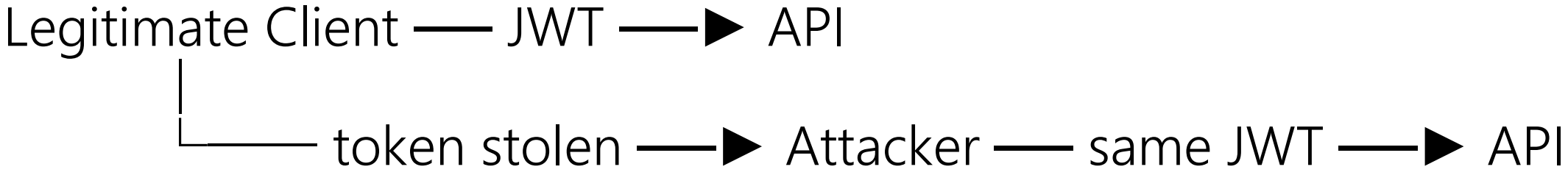
Key rotation is performed for several security and maintenance reasons:

- Improve security — Avoid using the same key forever.
- Key compromised — Replace a key that may have been exposed.
- Limit damage — Reduce how long a stolen key can be useful.
- Meet security policies — Some organizations require regular key changes.
- Retire old keys — Replace outdated or unwanted keys.
- Change algorithms — Move to a newer or stronger signing method.

© 2026 Innovation In Software Corporation

BEARER TOKEN - STOLEN

A critical point is that many access tokens are bearer tokens. Bearer means that possession of the token is normally enough to present it. The API generally does not know whether the request came from the original person or from someone who stole that token.



If the stolen token still has a valid signature, has not expired, is intended for the API, and contains sufficient permissions, the API may accept it. The digital signature prevents unauthorized modification or forgery; it does not by itself prevent theft and replay.

© 2026 Innovation In Software Corporation

BEARER TOKEN – STOLEN - 2

How to reduce bearer-token risk:

- HTTPS/TLS protects the token while it travels across the network.
- Short token lifetimes reduce how long a stolen token remains useful.
- Limited scopes or permissions reduce what a stolen token can do.
- Audience checking helps prevent a token intended for one API from being accepted by another.
- Secure token storage reduces the chance that the token is stolen from the client.
- Refresh tokens are protected more carefully because they may be used to obtain new access tokens.

© 2026 Innovation In Software Corporation

DEPENDS

Attach the dependency to any endpoint that needs a caller identity.

- One dependency, reused everywhere — consistency by construction
- Handlers stay focused on business logic, not token parsing

```
@app.get("/accounts")
async def list_accounts(claims: dict = Depends(verify_token)):
    user_id = claims["sub"]
    return get_accounts_for(user_id)
```


© 2026 Innovation In Software Corporation

STATUS CODES



Three different questions map to three different status codes:

401 Unauthorized — authentication failed: no/invalid/expired token ("who are you?")

403 Forbidden — authorization failed: valid token, insufficient permissions ("you can't do that")

404 Not Found — the resource doesn't exist

© 2026 Innovation In Software Corporation

LIBRARIES

Depends — FastAPI's dependency-injection marker, imported from fastapi. Wrap a callable in Depends(...) and attach it to a route parameter; FastAPI runs it before your handler and passes the return value in. It's how one verify_token gets reused across every protected route. Ships with FastAPI.

HTTPBearer — A security scheme class from fastapi.security. It pulls the token from the Authorization: Bearer <token> header and registers the scheme in your Swagger docs. It only extracts — it does *not* verify the signature or claims. It raises 403 on a missing header by default, so use HTTPBearer(auto_error=False) to return a correct 401 yourself. Ships with FastAPI.

© 2026 Innovation In Software Corporation

LIBRARIES - 2

HTTPAuthorizationCredentials — A small Pydantic model from fastapi.security that HTTPBearer returns. It has two fields: .scheme ("Bearer") and .credentials (the raw token). You read creds.credentials to get the token you'll decode. Part of FastAPI — no install.

jwt (PyJWT) — The library that actually decodes and verifies tokens: jwt.decode(token, key, algorithms=[...]) checks signature, expiry, issuer, and audience. This one must be installed: pip install "pyjwt[crypto]" (the [crypto] extra enables RS256/ES256). Note the import is import jwt.

© 2026 Innovation In Software Corporation

HTTPBEARER

HTTPBearer is a security-scheme class from fastapi.security. It inspects each request, finds the Authorization header, confirms it uses the Bearer scheme, and hands you back the raw token. It's purely an extraction tool — it does not decode the token, check the signature, verify expiry, or look at claims. That real validation is your job (typically with PyJWT). Think of HTTPBearer as the doorman who checks you're holding something that looks like a ticket; deciding whether it's genuine happens later.

```
HTTPBearer()
```

Beyond extraction, HTTPBearer registers the bearer scheme in your OpenAPI spec, which is what makes the "Authorize" button appear in the /docs Swagger UI so you can paste a token and test protected routes.

© 2026 Innovation In Software Corporation

WALKTHROUGH – FASTAPI / JWT IMPLEMENTATION



This FastAPI app shows the minimal JWT pattern: /login issues a token signed with a secret (HS256) carrying the username and a one-hour expiration, HTTPBearer pulls it from the Authorization header, and verify_token decodes it—checking signature and expiry—returning 401 on failure. Any route using Depends(verify_token) is guarded.

© 2026 Innovation In Software Corporation

FASTAPI – JWT - IMPLEMENTATION

```
from datetime import datetime, timedelta, timezone


# PyJWT library - handles encoding (creating) and decoding (verifying) JWTs.
# The import name is "jwt" even though the package installed is "pyjwt".
import jwt


# Depends       -> tells FastAPI "run this function first and inject its result"
# FastAPI       -> the app class
# HTTPException -> the way to return an error response (like 401 Unauthorized)
from fastapi import Depends, FastAPI, HTTPException
```


© 2026 Innovation In Software Corporation

FASTAPI – JWT – IMPLEMENTATION - 2

```
# HTTPBearer                    -> a security scheme that reads the
#     "Authorization: Bearer <token>" header
# HTTPAuthorizationCredentials -> the object HTTPBearer hands back;
#      holds the scheme + the token string
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer


# The secret key used to SIGN and VERIFY tokens. Only the server knows it.
# If someone knows this, they can forge valid tokens. Use a long random value
# from an environment variable in real code - never hardcode it.
SECRET = "change-me"


# The signing algorithm. HS256 = HMAC + SHA-256 (symmetric:
#      same secret signs and verifies).
ALGORITHM = "HS256"
```


© 2026 Innovation In Software Corporation

FASTAPI – JWT – IMPLEMENTATION - 3

```
# Create the web application.
app = FastAPI()


# Create the security scheme object. This one instance does two jobs:
#   1. Pulls the token out of the Authorization header on incoming requests.
#   2. Adds the "Authorize" button to the /docs Swagger UI.
security = HTTPBearer()
```

© 2026 Innovation In Software Corporation

FASTAPI – JWT – IMPLEMENTATION - 4

```python
def create_token(username: str) -> str:
    """Build and sign a JWT for the given user."""
    payload = {
        # "sub" (subject) = who the token is about. A registered JWT claim.
        "sub": username,
        # "exp" (expiration) = when the token stops being valid.
        # PyJWT reads this automatically and rejects the token after this time.
        # timezone.utc keeps the time unambiguous across servers/regions.
        "exp": datetime.now(timezone.utc) + timedelta(hours=1),
    }
    # Sign the payload with the secret. Returns a compact string:
    # header.payload.signature  (three base64 parts joined by dots).
    return jwt.encode(payload, SECRET, algorithm=ALGORITHM)
```

© 2026 Innovation In Software Corporation

FASTAPI – JWT – IMPLEMENTATION - 5

```
def verify_token(creds: HTTPAuthorizationCredentials = Depends(security)) ->
dict:
    """
    Dependency that protects an endpoint.


    Depends(security) means: before this function runs, FastAPI calls the
    HTTPBearer scheme, which reads the Authorization header. If the header is
    missing or malformed, HTTPBearer itself raises 401 - so by the time we get
    here, `creds` is guaranteed to hold a token string in creds.credentials.
    """
    try:
        # Verify the signature AND the expiration in one step.
        # If either fails, PyJWT raises an exception. On success we get the
        # decoded payload dict back (e.g. {"sub": "alice", "exp": ...}).
        return jwt.decode(creds.credentials, SECRET, algorithms=[ALGORITHM])
```

© 2026 Innovation In Software Corporation

FASTAPI – JWT – IMPLEMENTATION - 6

```
    except jwt.PyJWTError:
        # PyJWTError is the base class for all PyJWT errors (bad signature,
        # expired token, malformed token, etc.),
        #       so this one catch covers them all.
        # Returning 401 tells the client the credentials were rejected.
        raise HTTPException(status_code=401, detail="Invalid or expired token")


@app.get("/login")
def login():
    """
    Demo endpoint that hands out a token. In a real app you'd first check a
    username/password here, and only issue a token if they're correct.
    """
    return {"token": create_token("alice")}
```

© 2026 Innovation In Software Corporation

FASTAPI – JWT – IMPLEMENTATION - 7

```python
@app.get("/protected")
# The magic line: payload = Depends(verify_token).
# FastAPI runs verify_token first. If it raises 401, this handler never runs.
# If it succeeds, its return value (the decoded payload)
#      is injected as `payload`.
def protected(payload: dict = Depends(verify_token)):
    """Only reachable with a valid, unexpired token."""
    # payload["sub"] is the username we stored when creating the token.
    return {"message": f"Hello {payload['sub']}"}
```

© 2026 Innovation In Software Corporation

TEST WITH CURL

On Windows CMD, run each step. First get a token:

```
curl http://127.0.0.1:8000/login
```

Copy the token string, then call the protected route with it in the header:

```
curl http://127.0.0.1:8000/protected -H "Authorization: Bearer PASTE_TOKEN_HERE“
```

Test rejection by omitting the header or sending a garbage token. Confi e command returns {"detail":"Invalid or expired token"} with status 401.



© 2026 Innovation In Software Corporation

LIVENESS

Two signals clients depend on — they mean different things.

- Liveness (/health/live) — "is the process alive?" Fail → restart the container
- Readiness (/health/ready) — "can it serve traffic?" Checks dependencies (DB, cache). Fail → stop routing traffic, don't restart

```
@app.get("/health/live")
async def live() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/health/ready")
async def ready() -> dict[str, str]:
    await check_db()
    return {"status": "ready"}
```


© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned how JWTs are structured and signed, the difference between HS256 and RS256, and the RS256 login and verification flow. You covered discovery, JWKS, key IDs, key rotation, and the risk of stolen bearer tokens.

You also implemented JWT authentication in FastAPI using Depends and proper status codes, then verified it with curl.

© 2026 Innovation In Software Corporation

LAB 7 – VAULT



In this lab, build a small "Vault API" to practice FastAPI's Depends and liveness/readiness health checks. It fakes a database (a FakeDB on/off flag, toggled via /admin/toggle-db) so readiness has something to check, and fakes authentication to demonstrate the Depends gate.

GOAL



In this lab you'll build a small "Vault API" that puts Depends and health checks to work together — no JWT required. You'll implement seven TODOs in main_starter.py, then verify each scenario with curl against an expected-results table.

© 2026 Innovation In Software Corporation

REUSABLE CHECK

A dependency lets you write a check once and run it before any route that needs it.

- TODO 1a — write the if condition that detects a wrong/missing key → 401
- TODO 1b — return the value handed back on success (hint: the key passed in)
- TODO 2 — attach the check to /vault (hint: uses Depends(...))
- TODO 3 — attach the same check to /vault/status
- Save and watch the window reload

© 2026 Innovation In Software Corporation

TEST AND VERIFY

Sending the same route three different keys shows the gate accepting and rejecting callers.

```
curl .../vault
curl .../vault -H "X-API-Key: nope"
curl .../vault -H "X-API-Key: let-me-in"
curl .../vault/status -H "X-API-Key: let-me-in
```

No security code inside the routes — the Depends line did it all.

© 2026 Innovation In Software Corporation

HEALTH CHECKS

Liveness and readiness answer different questions, so they fail in different ways.

- /health/live = "is the app running?" → fail = restart
- /health/ready = "can it serve traffic?" → fail = stop routing, don't restart

The live() check has no dependency: a running app should stay 200 even when the DB is down — restarting won't fix a database outage

The lab fakes a DB with an on/off switch you can toggle.

© 2026 Innovation In Software Corporation

HEALTH CHECKS - 2

You'll write the DB check once, then share it between a health probe and a real route.

- TODO 4a — if condition that detects a disconnected DB → 503 (hint: check connected on fake_db)
- TODO 4b — return the value when the DB is fine
- TODO 5 — live() returns a simple "alive" signal, with no Depends()
- TODO 6 — make /health/ready run the DB check (hint: same Depends(...) pattern)
- TODO 7 — make /data run the same DB check

Save!

© 2026 Innovation In Software Corporation

TEST AND VERIFY

Toggling the fake database off reveals which routes depend on it and which don't.

curl …/health/live

curl …/health/ready

curl …/data

curl -X POST …/admin/toggle-db (turn DB off, re-check, then on again)

If DB off: /health/live → 200 · /health/ready → 503 · /data → 503

© 2026 Innovation In Software Corporation

SEVERAL DEPENDENCIES

Real routes often need more than one check before they run — and one check can lean on another.

- A single route can require several dependencies at the same time
- A dependency can even use another dependency: require_admin_key first runs your require_api_key (must have a valid key), then adds one more rule — the key must be the admin's

The /reports route will use three checks together: pagination + admin + database

© 2026 Innovation In Software Corporation

SEVERAL DEPENDENCIES

Stacking three dependencies keeps all the guarding out of the handler body.

- TODO 8 — if condition that detects a non-admin → 403
- TODO 9 — attach all three deps to /reports (page, admin, db; each uses Depends(...))

Test:

```
curl .../reports

curl .../reports -H "X-API-Key: let-me-in"

curl ".../reports?skip=5&limit=2" -H "X-API-Key: let-me-in"
```


© 2026 Innovation In Software Corporation

RESULTS

Here's what you should see when you test the /reports route:

- Calling it without an API key returns a 401 error. The key check runs first and blocks the request before you ever reach the route.
- Calling it with the correct key succeeds (200), using the defaults skip 0 and limit 10 — it starts at the beginning and returns up to 10 results.
- Adding ?skip=5&limit=2 to the URL also succeeds, this time using your numbers: it skips the first 5 results and returns just 2.

Together these confirm two things: the key check is guarding the route, and pagination works — defaults when you leave it out, your own numbers when you include it.

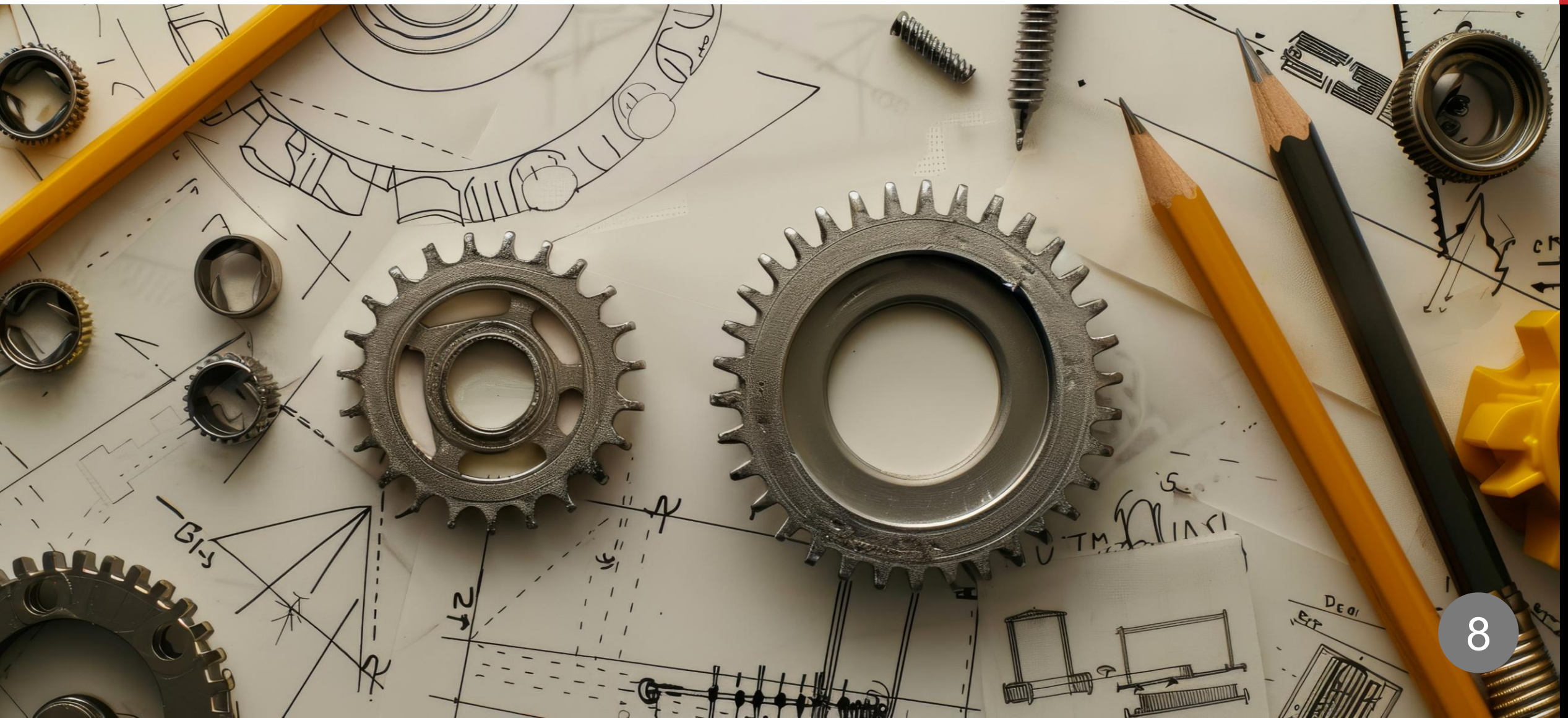
© 2026 Innovation In Software Corporation

Lab completed



BEST PRACTICES

RESOURCES, HTTP STATUS CODES, AND MORE



INTRODUCTION



In this module, you will learn conventions for predictable resource URLs, the HTTP methods and status codes that go with them, and best practices for errors and API security. You will add logging in FastAPI and explore observability through logs, metrics, percentiles, correlation IDs, and dashboards.

You will then look at Splunk and learn to version APIs through the URL path, manage major and silent changes, and deprecate endpoints.

© 2026 Innovation In Software Corporation

RESOURCE URLS

A few core conventions keep your resource URLs predictable and easy to consume:

- Model nouns, not verbs — resources are things (/accounts, /loans), operations are HTTP methods
- Use plural collection names consistently (/customers, not /customer)
- Prefer flat over deeply nested — nest only to express true ownership (/accounts/{id}/transactions)
- One canonical URL per resource; avoid multiple paths that return the same thing

© 2026 Innovation In Software Corporation

RESOURCE URLS

Consistent path and query-string conventions make your API predictable and keep implementation details from leaking to clients:

- Lowercase, hyphen-separated paths (/loan-applications, never /loanApplications or /loan_applications)
- No trailing slashes; no file extensions in the path (use Accept headers for format)
- Nouns in the path, filters in the query string (/accounts?status=active&type=checking)
- Reserve query params for pagination, sorting, filtering, and field selection (?page=2&sort=-created&fields=id,name)
- Keep IDs opaque — don't leak sequential integers that expose volume or ordering

© 2026 Innovation In Software Corporation

HTTP

What are the common HTTP methods?





5 MINUTES

© 2026 Innovation In Software Corporation

RESOURCE URLS

HTTP methods tell an API what action you want to perform on a resource.

- GET — retrieve; safe and idempotent; never mutates state
- POST — create a new resource (server assigns ID); not idempotent
- PUT — full replacement of a resource; idempotent
- PATCH — partial update; send only changed fields
- DELETE — remove a resource; idempotent
- Match method to intent — never tunnel writes through GET

© 2026 Innovation In Software Corporation

HTTP STATUS CODE

The HTTP status code tells the client what kind of thing went wrong, so they can respond correctly instead of guessing:

400 Bad Request — malformed syntax or invalid payload

401 Unauthorized — missing or invalid authentication

403 Forbidden — authenticated but not permitted

404 Not Found — resource doesn't exist

409 Conflict — state conflict (e.g., duplicate, version mismatch)

422 Unprocessable Entity — well-formed but semantically invalid

429 Too Many Requests — rate limit exceeded

500 / 503 — server fault / temporarily unavailable

© 2026 Innovation In Software Corporation

ERROR – BEST PRACTICES

Do

- Return the same envelope for every error, every endpoint
- Map internal exceptions to safe, typed error codes via a global handler
- Include enough context for the client to act (which field, what rule)

Don't

- Leak stack traces, SQL, file paths, or internal hostnames
- Reflect raw input back into messages unescaped
- Return 200 with an error buried in the body

© 2026 Innovation In Software Corporation

API SECURITY – BEST PRACTICES

API security should be built into every request and every endpoint, not added as an afterthought.

- TLS everywhere — Reject plaintext HTTP. TLS encrypts data as it travels between the client and API.
- OAuth 2.1 / JWT — Validate signature, issuer, audience, and expiry. Never trust a token until it has been properly verified.
- Authorization per resource — Check permissions and ownership, not just the endpoint. JWT claims, such as roles, permissions, scopes, or user IDs, can help determine what resources and actions the user is authorized to access.
- Validate all input — Allowlist expected data and reject unexpected fields. Never assume data received from a client is safe.

© 2026 Innovation In Software Corporation

API SECURITY – BEST PRACTICES - 2

- Rate limiting and size limits — Control request frequency and payload size. This helps protect the API from abuse and DoS attacks.
- OWASP security guidelines — Use the OWASP Top 10 and OWASP API Security Top 10 as baseline security checklists. They identify many of the most common security risks facing applications and APIs.
  - OWASP Top 10 — 2025
  - OWASP API Security Project

© 2026 Innovation In Software Corporation

LOGGING IN FASTAPI

Logging records what is happening inside your API while it is running. Logging helps developers understand, troubleshoot, and monitor a FastAPI application.

- Track requests — Record important API activity, such as incoming requests and completed responses.
- Find errors — Log exceptions and failures so problems are easier to diagnose.
- Monitor behavior — Record important events, warnings, and unexpected conditions.
- Use Python logging — FastAPI applications commonly use Python's built-in logging module.
- Choose log levels — Use levels such as DEBUG, INFO, WARNING, ERROR, and CRITICAL.
- Avoid sensitive data — Do not log passwords, tokens, secrets, or other confidential information.

© 2026 Innovation In Software Corporation

LOGGING SAMPLE CODE

```
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)



@app.middleware("http")
async def log_requests(request: Request, call_next):
    logger.info(f"{request.method} {request.url.path}")
    response = await call_next(request)
    return response



@app.get("/")
def hello():
    return {"message": "Hello"}
```

© 2026 Innovation In Software Corporation

LOGGING IN FASTAPI – BEST PRACTICES

Logs are easy to leak and easy to forget about, so treat them as a security surface — what you write today may sit readable for months:

- Never log secrets — passwords, tokens, API keys, full card numbers, SSNs
- Mask or redact PII at the logging boundary (e.g., ****1234)
- Log the traceId, not the sensitive payload that produced the error
- Beware accidental capture — full request/response dumps, verbose exception context
- Treat logs as a data store subject to the same access controls and retention rules

© 2026 Innovation In Software Corporation

OBSERVABILITY



Observability is understanding a running system from the signals it emits, without shipping new code to investigate. Unlike traditional monitoring ("is the server up?"), it lets you ask new questions about unexpected behavior.

This matters for a production REST API because distributed failures are rarely clean crashes—slow endpoints, intermittent errors, and cascading timeouts that only make sense when you correlate evidence across the three pillars: logs, metrics, and traces.

© 2026 Innovation In Software Corporation

LOGS

Application logs should be emitted, as shown previously, as structured data rather than free-form text, with JSON being the de facto standard because it's machine-parseable by log aggregators like Splunk, Elasticsearch, or Loki. Each entry carries a severity level (DEBUG, INFO, WARN, ERROR) so operators can filter noise from signal, and critically, every log line includes a `traceId` field.

That correlation key is what lets you pull every log statement generated by a single request—across threads, async tasks, and service boundaries—into one coherent timeline instead of guessing which lines belong together.

© 2026 Innovation In Software Corporation

METRICS

Where logs describe individual events, metrics are aggregated numerical measurements sampled over time, ideal for dashboards and alerting.

The RED method gives you a focused starting set per endpoint:

- Rate: requests per second
- Errors: the count or percentage of failed requests
- Duration: tracked as percentiles like p50, p95, and p99 rather than averages, since averages hide outliers.

Tracking these three per route quickly surfaces which endpoint is slow, which is failing, and which is under unexpected load.

© 2026 Innovation In Software Corporation

PERCENTILE

A percentile tells you the value below which a given percentile of your measurements fall. If p95 latency is 200ms, that means 95% of requests completed in 200ms or less, and the slowest 5% took longer.

```
from fastapi import FastAPI, Request
from statistics import quantiles
import time

app = FastAPI()
durations = []  # stored in memory for demo purposes only

@app.middleware("http")
async def track_latency(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    elapsed_ms = (time.perf_counter() - start) * 1000
    durations.append(elapsed_ms)
    return response
```

© 2026 Innovation In Software Corporation

PERCENTILE- 2

```python
@app.get("/")
async def root():
    return {"message": "hello"}


@app.get("/metrics")
async def metrics():
    if len(durations) < 2:
        return {"count": len(durations)}
    # quantiles(n=100) splits data into 100 buckets, giving p1..p99
    p = quantiles(durations, n=100)
    return {
        "count": len(durations),
        "p50_ms": round(p[49], 2),
        "p95_ms": round(p[94], 2),
        "p99_ms": round(p[98], 2),
    }
```

© 2026 Innovation In Software Corporation

CORRELATION ID



The mechanism that ties all three pillars together is a correlation ID generated the moment a request enters your system, typically in edge middleware. That ID must be attached to every log line, tagged onto metrics, and forwarded in the headers of every downstream call.

If any API service fails to propagate it, the trace breaks and you lose the ability to stitch the request's full path back together.

© 2026 Innovation In Software Corporation

OBSERVABILITY – BEST PRACTICES

These practices turn observability from theory into a running system operators can actually depend on:

- Every request carries a traceId — generated at the edge, returned in responses and errors
- Instrument latency percentiles (p50/p95/p99), not just averages
- Standard health endpoints: /health (liveness) and /ready (readiness)
- Alert on symptoms users feel (error rate, latency) over raw resource counters
- Make dashboards answer: is it up, is it fast, is it failing?

© 2026 Innovation In Software Corporation

DASHBOARDS

What is the best tool to create dashboards from the observability metrics?



© 2026 Innovation In Software Corporation

5 MINUTES

SPLUNK



Splunk is a solid, widely-used choice for building dashboards from observability data.

Splunk ingests your structured JSON logs, and its Search Processing Language (SPL) computes rate, error rate, and latency percentiles at search time, then pins those searches to dashboard panels. Since your logs already carry a traceId and are leveled, they slot right in—you'd use stats and timechart to aggregate over them and build the "is it up, is it fast, is it failing?" views.

© 2026 Innovation In Software Corporation

VERSIONING: URL PATH

Put the version number right in the URL so it's impossible to miss. When a client calls /v1/accounts, anyone can see which version is in use just by looking at the address.
It's cache-friendly because caches treat each full URL as its own thing. So /v1/accounts and /v2/accounts are stored separately and never get mixed up.

It's easy to route because your server can send /v1/ traffic to one set of code and /v2/ traffic to another, just by matching the path.

This approach makes the versioning explicit, cache-friendly, easy to route.

© 2026 Innovation In Software Corporation

VERSIONING: MAJOR VERSION CHANGE

A major version change occurs when there is a breaking change – a breaking change is anything that could make an existing client's code stop working.

Examples: deleting a field the client reads, changing a field from a number to a string, or changing what a value means (say balance used to be in dollars and now it's in cents).

When you make a change like this, move to a new version—/v1/ becomes /v2/. Old clients keep using /v1/ and stay working. New clients opt into /v2/.

© 2026 Innovation In Software Corporation

VERSIONING: SILENT CHANGES

Never silently change behavior within a version — clients depend on the contract.

Once /v1/ is published, it's a promise. Clients wrote their code assuming /v1/ behaves a certain way. For that reason, don't *quietly* change how it works underneath them. Even a small tweak—like rounding a number differently or reordering results—can break someone in ways that are painful to debug.

If the behavior truly needs to change, that's what a new version is for. Keep each version stable and predictable so clients can trust it.

© 2026 Innovation In Software Corporation

ENDPOINT DEPRECATION

Retiring an old API version should be a gradual, well-communicated process—not a sudden shutoff—so clients have time to migrate:

- Announce — publish the deprecation and the replacement path with a clear timeline
- Signal — send Deprecation and Sunset response headers on affected endpoints
- Overlap — run old and new versions in parallel through a defined support window
- Monitor — track remaining consumers of the old version before shutdown
- Retire — remove only after the sunset date; return *410 Gone* - *more* for the removed version

© 2026 Innovation In Software Corporation

EXAMPLE CODE

Here's the FastAPI implementation for the **overlap window**, when the old version still works but is announced as deprecated through the Deprecation and Sunset signaling headers:

```
# ---------------------------------------------------------------------------
# v1 — DEPRECATED but still running during the overlap window
# ---------------------------------------------------------------------------
v1 = APIRouter(prefix="/v1", tags=["v1 (deprecated)"])


@v1.get("/hello")
async def hello_v1(response: Response):
    # Deprecation: true  -> "this endpoint is deprecated"
    # Sunset: <date>      -> "it will be removed on this date"
    # Link                -> points clients to the replacement
    response.headers["Deprecation"] = "true"
    response.headers["Sunset"] = format_datetime(SUNSET, usegmt=True)
    response.headers["Link"] = '</v2/hello>; rel="successor-version"'
    return {"message": "Hello from v1", "version": "v1"}
```


© 2026 Innovation In Software Corporation

EXAMPLE CODE - 2

```
# ------------------------------------------------------------------------
# v2 — CURRENT
# ------------------------------------------------------------------------
v2 = APIRouter(prefix="/v2", tags=["v2 (current)"])

@v2.get("/hello")
async def hello_v2():
    return {"message": "Hello from v2", "version": "v2"}


app.include_router(v1)
app.include_router(v2)
```

The final two lines register the routers with your main FastAPI app. Until now, v1 and v2 were standalone APIRouter objects the app didn't know about.

Calling app.include_router() wires each one in, making their paths (/v1/hello and /v2/hello) live endpoints the app will actually serve.

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned conventions for resource URLs, HTTP status codes, and best practices for errors and API security. You added logging and explored observability with metrics, percentiles, correlation IDs, and dashboards.

You also examined Splunk and learned to version APIs through the URL path, manage major and silent changes, and deprecate endpoints cleanly.

© 2026 Innovation In Software Corporation

LAB 8 – IT WAS THE BEST OF TIMES…



In this lab you'll run two versions of an endpoint side by side and signal the old one's retirement the way real APIs do.

GOAL

APIs can't change overnight — clients need time to migrate. During that "overlap window," the old version keeps working but signals it's going away with three response headers: Deprecation (it's deprecated), Sunset (when it's removed), and Link (where the replacement lives). You'll build a Quote API with a deprecated v1 route and a current v2 route that replaces it.

You start from api_starter.py, where a bare v1/quote route works but doesn't yet warn callers it's retiring. Your job: add the three headers to v1 (sunset 01 Jan 2027) and build the header-free v2/quote route. Fill in the 5 hinted TODOs, then verify with curl — check api_answer_key.py only if stuck.

© 2026 Innovation In Software Corporation

SETTING UP THE HEADERS

First, make the old route politely announce that it's retiring — it keeps working, but now it warns callers.

- TODO 1 — set the sunset date. Create the date v1 stops working: 01 Jan 2027. This is just a value you store once and reuse in TODO 3. (Hint: datetime(2027, 1, 1, tzinfo=timezone.utc).)
- TODO 2 — add the Deprecation header. This flag tells callers "this endpoint is deprecated." Set the header's value to the text "true".
- TODO 3 — add the Sunset header. This tells callers when the route disappears. Use your date from TODO 1, formatted as a proper web date so clients can read it.

© 2026 Innovation In Software Corporation

SETTING UP THE HEADERS - 2

- TODO 4 — add the Link header. This points callers to the replacement so they know where to go next — it references the /v2/quote path and marks it as the successor.

All three headers go on the existing v1/quote route. The route already returns the quote; you're only adding these three lines.

© 2026 Innovation In Software Corporation

V2 ROUTING

Next, create the route that replaces v1 — updated content, no retirement warnings.

- TODO 5 — build v2/quote. A new GET route on the v2 router that returns the updated quote, "It is the best of times. It is the worst of times.", with none of the deprecation headers. (Hint: copy v1, delete the three header lines, swap in the new quote, and change the version label to "v2".)
- v1 and v2 return different quotes so the version difference is obvious in the response.

© 2026 Innovation In Software Corporation

VERIFICATION

Start the app, then test in a second window (-i shows the headers):

```
curl -i http://127.0.0.1:8000/v1/quote
curl -i http://127.0.0.1:8000/v2/quote
```

You did it right if:

- /v1/quote → 200, and its headers include Deprecation: true, a Sunset date of 01 Jan 2027, and a Link to /v2/quote
- /v2/quote → 200 with the same quote and no deprecation headers

© 2026 Innovation In Software Corporation

Lab completed



TRUNK BASED DEVELOPMENT (TBD)

SOURCE CONTROL



INTRODUCTION



In this module, you will learn trunk-based development and the Git fundamentals behind it: distributed version control, software forges, commit hashes, the staging area, branches, and releases.

You will then walk the TBD workflow — a typical day locally and on GitHub, plus a hotfix — and cover code reviews, pull requests, and feature flags.

© 2026 Innovation In Software Corporation

TRUNK-BASED DEVELOPMENT

Trunk-based development is a software development approach where developers collaborate on a main trunk as the single source of truth. Instead of maintaining multiple long-lived branches for different features or bug fixes, developers make small, frequent commits directly to the trunk.

This approach encourages continuous integration, where changes are integrated into the main branch frequently. Trunk-based development also promotes collaboration, reduces the risk of merge conflicts, and allows for faster feedback. Integration practices, automated testing, and deployment pipelines are typically used to ensure the stability of the mainline code.



TRUNK-BASED DEVELOPMENT - 2

Trunk-based development adheres to many of the same principles and practices of agile processes. This includes maintaining a releasable increment, which increases the flexibility and ultimately confidence in the project for stakeholders and developers.



DISTRIBUTED CONTROL SYSTEMS



A distributed version control system (DVCS) tracks changes to files where every contributor keeps a complete copy of the repository—including its full history—on their local machine, rather than relying on a single central server. Each developer can commit, view history, create branches, and work offline, then sync with others by pushing and pulling changes.

Examples include Git, Mercurial, and Bazaar.

This module will use Git for all examples.

© 2026 Innovation In Software Corporation

SOFTWARE FORGE



A software forge (or code hosting platform) is a web-based service that hosts software repositories and layers collaboration tools on top of version control—things like pull requests, issue tracking, code review, and CI/CD pipelines. These platforms let distributed teams share code, coordinate changes, and manage projects in one place. Common examples include GitHub, GitLab, and Bitbucket.

This course will use GitHub for all examples.

© 2026 Innovation In Software Corporation

GIT-BASH



Git Bash is a command-line interface (CLI) tool that provides a Unix-like environment to interact with Git and other tools directly from the command line. It combines the power of Git version control with the familiarity of Unix commands.

Git Bash provides Git commands for repository, branch manipulation, and version control operations, along with a set of Unix utilities for file manipulation, text processing, and system administration tasks.

The Git Bash CLI provides a consistent command interface, regardless of the platform environment.

© 2026 Innovation In Software Corporation

COMMIT HASH

```
f2a4c6e8b0d2f4a6c8e0b2d4f6a8c0e2b4d6f8a0
8d0f2a4c6e8b0d2f4a6c8e0b2d4f6a8c0e2b4d6f
0b2d4f6a8c0e2b4d6f8a0c2e4b6d8f0a2c4e6b8d
3c5e7a9b1d3f5c7e9a1b3d5f7c9e1a3b5d7f9c1e
9f1b3d5a7c9e1b3d5f7a9c1e3b5d7f9a1c3e5b7d
5a7c9e1f3b5d7a9c1e3f5b7d9a1c3e5f7b9d1a3c
e7b9d1f3a5c7e9b1d3f5a7c9e1b3d5f7a9c1e3b5
2d4f6a8c0e2b4d6f8a0c2e4b6d8f0a2c4e6b8d0f
b1d3f5a7c9e1b3d5f7a9c1e3f5b7d9a1c3e5f7b9
7f9a1c3e5b7d9f1a3c5e7b9d1f3a5c7e9b1d3f5a
c3e5f7b9d1a3c5e7f9b1d3a5c7e9f1b3d5a7c9e1
4d6f8a0c2e4b6d8f0a2c4e6b8d0f2a4c6e8b0d2f
a5c7e9b1d3f5a7c9e1b3d5f7a9c1e3b5d7f9a1c3
f7b9d1a3c5e7f9b1d3a5c7e9f1b3d5a7c9e1f3b5
1c3e5f7b9d1a3c5e7f9b1d3a5c7e9f1b3d5a7c9e
8a0c2e4b6d8f0a2c4e6b8d0f2a4c6e8b0d2f4a6c
d1f3a5c7e9b1d3f5a7c9e1f3b5d7a9c1e3f5b7d9
```

Each commit is a snapshot of the entire project at the moment it's created. The snapshot becomes an object placed in the Git repository and assigned a unique identifier called a "commit hash" or "commit ID."

This hash is a 40-character hexadecimal string, generated by applying the SHA-1 algorithm to the contents of the commit—including the commit message, author information, timestamp, and most importantly, the state of the project's files at that specific point in time.

STAGING AREA



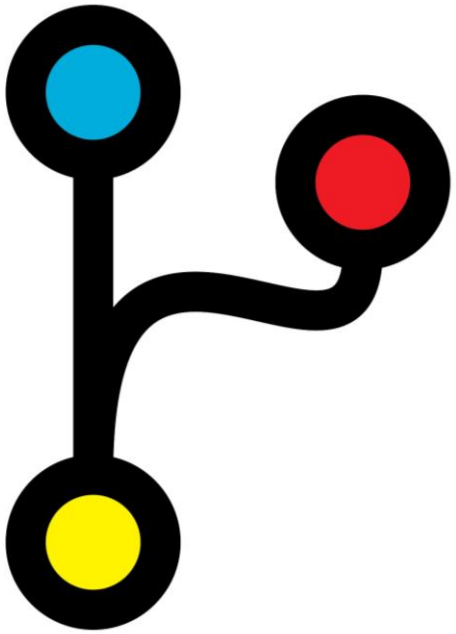
No, not that kind of stage!

The staging area is an intermediary where changes are prepared before a commit. It's backed by a data structure called the "index," which records the current state of the working directory and what will be included in the next commit. Rather than storing differences between versions, the index holds a snapshot of the content staged for the next commit, along with metadata such as file paths and timestamps.

The git add command stages changes in the index. The git commit command permanently saves those staged changes to the repository as a new version.

© 2026 Innovation In Software Corporation

BRANCHES



In trunk-based development, short-lived branches are temporary branches created for a specific task, feature, or bug fix. Unlike GitFlow, short-lived branches are created and merged back into the main/trunk branch relatively quickly, often within a day or two. (In this class, we assume a single development cycle is an Agile sprint.)

Once the task is complete or the feature is implemented, the changes in the short-lived branch are merged back into the main/trunk branch, and the branch is deleted.

© 2026 Innovation In Software Corporation

RELEASE

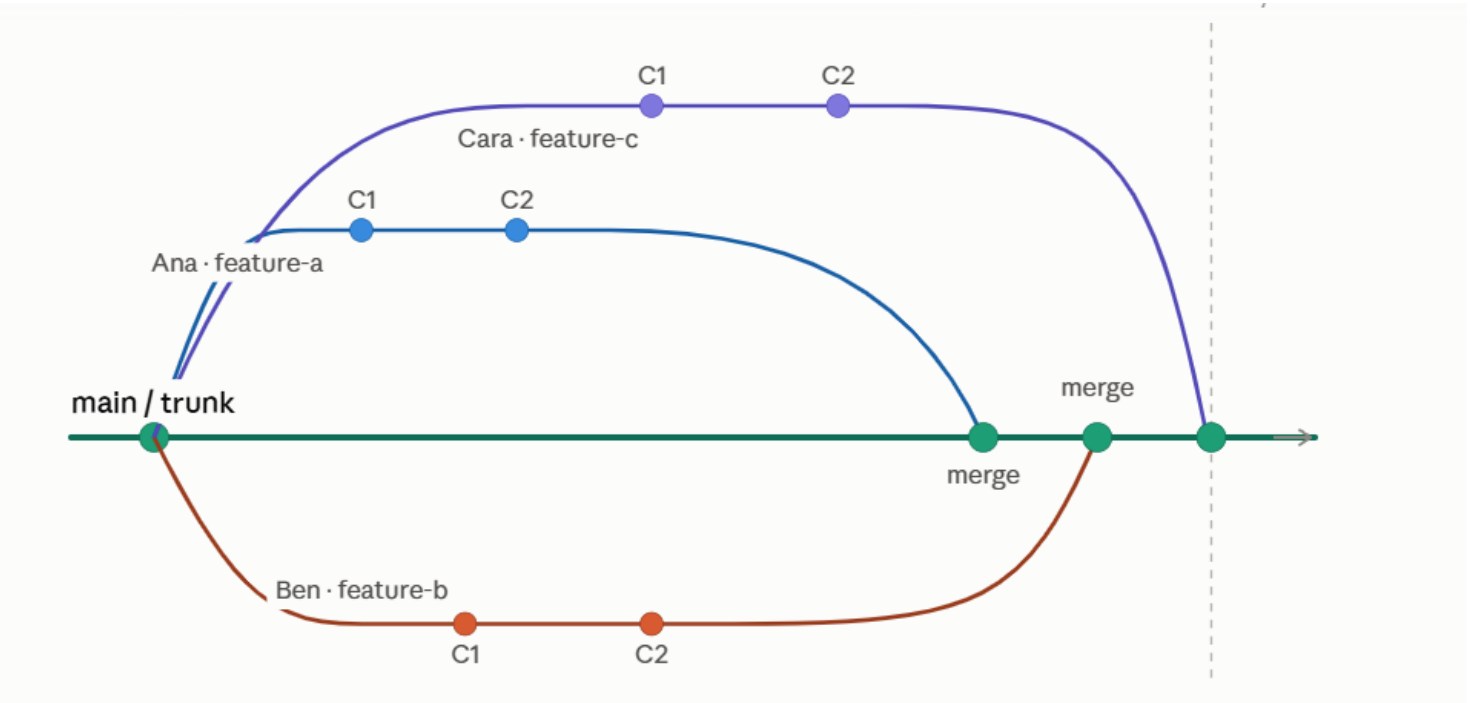
With trunk-based development, the main trunk is kept in a releasable state at all times. This aligns with Agile methodology and the idea of a perpetual increment—you can release from trunk at any time. Specific releases, including hotfixes, are marked with tags (e.g., v1.2.0) that pin the exact commit released, giving you traceability without having a long-lived release branch.



© 2026 Innovation In Software Corporation

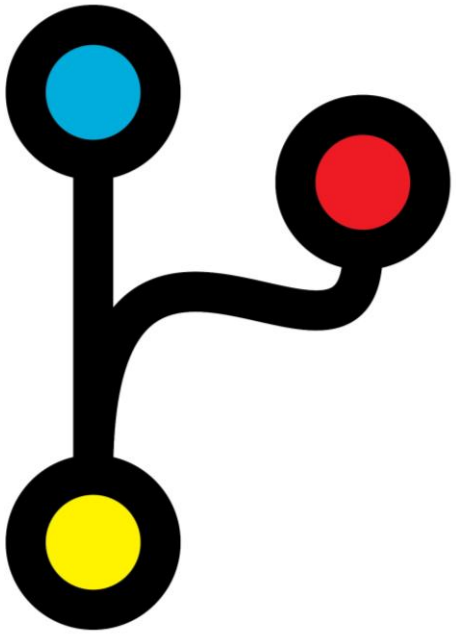
TBD WORKFLOW

The dots on each branch are commits made throughout the day—Ana, Ben, and Cara commit locally on their own short-lived branches as often as they like (small, frequent commits like C1, C2 are encouraged). Near the end of the day, each branch merges into main and is deleted. In the diagram, every branch reconnects to the trunk at or before the end-of-day line, so the trunk always carries everyone's integrated work forward.



© 2026 Innovation In Software Corporation

GETTING STARTED



The trunk-based development lifecycle begins with a one-time setup: cloning the repository. The git clone command creates a copy of a remote repository on your local machine and establishes a connection between your local and remote repositories. After running it, you'll have a complete copy—all branches, commit history, and files.

git clone [options] <repository_url> [<directory>]

Once cloned, the day-to-day cycle starts by pulling the latest changes from the main/trunk branch before beginning any new work, so you're always building on the most current code

© 2026 Innovation In Software Corporation

TYPICAL DAY

The following walkthrough traces a single day's work in trunk-based development—from getting the latest code to integrating your changes back into main—so you can see how the individual Git commands fit together into one repeatable cycle.

- Set up (once). git clone downloads the whole project—including its history—to your machine. You only do this the first time.
- Start fresh. pull the latest main so you're building on current code, then create a short-lived branch—a private workspace off main where you can work without disturbing the trunk.
- Work and save. Make your change, then add (stage what goes in the snapshot) and commit (record it with a message). Repeat often—small, frequent commits give you save points and a clear history.

© 2026 Innovation In Software Corporation

TYPICAL DAY - 2

- Integrate and clean up. Pull the latest main again and merge it into your branch to resolve conflicts while they're small. Then merge into main, push to the remote, and delete the branch—its work now lives in main.

There is a rhythm starting with the latest trunk → work in a short-lived branch → commit small and often → re-sync, merge, push → delete. Doing this at least once a day keeps the trunk healthy and conflicts tiny.

© 2026 Innovation In Software Corporation

TYPICAL DAY - GITHUB

```
# 1. One-time setup: clone the repo
$ git clone git@github.com:InnovativeIo/1_hello.git
$ cd 1_hello

# 2. Start of a task: get the latest trunk
$ git checkout main
$ git pull origin main

# 3. Create a short-lived branch for your PBI
$ git checkout -b feature/hello-update

# 4. Do the work
$ echo "more changes" >> readme.md
```

© 2026 Innovation In Software Corporation

TYPICAL DAY – GITHUB - 2

```
# 5. Stage and commit (repeat often as you work)
$ git add .
$ git commit -m "Add hello update to readme"

# 6. Before integrating: pull latest trunk and merge in
$ git checkout main
$ git pull origin main
$ git checkout feature/hello-update
$ git merge main          # resolve any conflicts here, while they're small
```

Notice that you ran git pull origin main while on main and then git push origin main, but the commit was made without ever creating a branch—so it committed straight onto main.

© 2026 Innovation In Software Corporation

TYPICAL DAY – GITHUB - 2

```
# 7. Integrate into trunk and push
$ git checkout main
$ git merge feature/hello-update
$ git push origin main

# 8. Clean up: delete the short-lived branch
$ git branch -d feature/hello-update
$ git push origin --delete feature/hello-update   # if it was
pushed remotely
```


© 2026 Innovation In Software Corporation

HOTFIX - SCENARIO



Let's walk through a scenario with two developers, Alice and Bob, using trunk-based development (TBD) in a remote repository.

Setup

Alice and Bob are both developers on a software project using TBD. They both have access to the main trunk of the repository.

Normal Development

Alice and Bob are working on different features or tasks, integrating their changes into the main trunk frequently as they complete them.

© 2026 Innovation In Software Corporation

HOTFIX – SCENARIO - 2



Hot Fix

Suddenly, a critical issue arises that needs immediate attention. Alice identifies it and creates a short-lived branch to address it quickly without disturbing the main trunk's stability. She names the branch "hotfix-123" off the main trunk and makes the necessary changes there to fix the issue.

Merging the Hot Fix

Once Alice is confident her fix is correct, she pulls the latest main, merges it into her branch to catch any conflicts, then merges "hotfix-123" back into the main trunk. The trunk now includes the fix for the critical issue.

© 2026 Innovation In Software Corporation

CODE REVIEWS

Code reviews serve as a crucial quality control measure, ensuring that code adheres to coding standards, follows best practices, and is free from errors and potential bugs. Even small changes can introduce issues that may be missed without proper review.

- Gain insights into the changes made by their colleagues, learn new techniques, and understand different approaches to solving problems.
- Identify potential problems early in the development process minimizing the risk of larger problems down the line.
- Ensure consistency in coding style, architecture, and design across the codebase.
- Adherence to compliance and governance requirements, especially in the area of security.



© 2026 Innovation In Software Corporation

NO PULL REQUEST



To move faster, teams practicing trunk-based development may forgo formal pull requests (PRs) in favor of continuous review—most often pair programming, where code is reviewed in real time as it's written. So "no code review" here usually means no separate PR gate, not that review is skipped; it simply happens live. This works because check-ins are small and frequent, often just a few lines.

It depends on comprehensive automated testing to keep the main trunk stable, since there's no separate review step to catch regressions. It also requires high trust and is best suited to experienced teams with senior developers.

© 2026 Innovation In Software Corporation

PULL REQUEST

A pull request (PR) is a request to merge the changes on your branch into another branch (usually main). The typical flow:

1. Create a branch and do the work. Branch off main, make your commits.

2. Push the branch to the remote. git push origin your-branch so the branch exists on the server (GitHub, GitLab, Bitbucket).

3. Open the pull request. On the hosting platform, you open a PR from your branch into main, with a title and description explaining what changed and why.

© 2026 Innovation In Software Corporation

PULL REQUEST - 2

4. Review and discussion. Teammates review the diff, leave comments, request changes, or approve. Automated checks (CI tests, linters) usually run automatically and must pass.

5. Address feedback. You push additional commits to the same branch to respond to comments; the PR updates automatically.

6. Approve and merge. Once approved and checks pass, the PR is merged into main (via merge commit, squash, or rebase, depending on team policy).

7. Delete the branch. After merging, the short-lived branch is deleted—its work now lives in main.

© 2026 Innovation In Software Corporation

LONG RUNNING PRS



One note tying this back to trunk-based development: PRs fit TBD when they're small and short-lived—opened and merged within a day or so. Long-running PRs with big diffs work against the frequent-integration goal. Some TBD teams skip PRs entirely in favor of pair programming, as we discussed.

© 2026 Innovation In Software Corporation

FEATURE FLAGS

Imagine you've built a new checkout button, but it's not quite ready for customers. In older workflows, you'd keep that unfinished code on a separate branch until it was done—and then face a big, messy merge later.

A feature flag lets you do the opposite: you merge the unfinished code into the main trunk right away, but wrap it in a simple on/off switch. The switch is "off," so customers never see the new button, even though its code is live in the application. When the feature is ready, you flip the switch "on"—no new merge, no scramble. It's like wiring a new light into your house but leaving the switch off until the room is finished.



otherwise filtered using feature toggles or other method.

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned trunk-based development along with the core Git concepts behind it: commit hashes, the staging area, branches, and releases. You followed the TBD workflow through a typical day and a hotfix scenario.

You also compared review approaches with and without pull requests, and saw how feature flags let you integrate and ship safely.

© 2026 Innovation In Software Corporation

SCENARIOS / QUESTIONS



For each scenario, decide what the developer should do (and why), based on trunk-based development practices. Notate your answer before checking the key.

QUESTIONS

Question 1 — The long-lived branch

Priya is new to the team and comes from a GitFlow background. She creates a branch called feature/new-dashboard and plans to keep working on it for the next three weeks until the entire dashboard is finished, only merging back to main at the very end. A teammate notices and raises a concern. What is the problem with Priya's plan under trunk-based development, and what should she do instead?

Question 2 — The unfinished checkout button

Marcus has built about half of a new checkout button. The code compiles and doesn't break anything, but the feature isn't ready for customers to see. His manager wants all work integrated into the trunk daily, but Marcus is worried that merging half-finished code will expose an incomplete feature to users. How can Marcus satisfy both the daily-integration requirement and the need to hide the unfinished feature?

© 2026 Innovation In Software Corporation

QUESTIONS

Question 3 — Starting the day

Dana sits down Tuesday morning to start a new task. Eager to begin, she immediately creates a short-lived branch off her local copy of main — which she last updated four days ago — and starts coding. By afternoon she's hitting confusing merge conflicts. What step did Dana skip, and what's the correct start-of-day sequence?

Question 4 — The critical production bug

While Alice and Bob are each working on separate features, a critical issue surfaces in production that needs an immediate fix. Alice volunteers to handle it. Walk through how Alice should address this hotfix using trunk-based development, from creating the fix to getting it into the trunk safely.

© 2026 Innovation In Software Corporation

QUESTIONS

Question 5 — "We don't do code review"

A new hire hears a senior developer say the team "doesn't do pull requests or formal code review." The new hire concludes that code on this team ships without anyone else ever looking at it. Is that conclusion correct? Explain what's actually happening and what conditions make this approach workable.

Question 6 — The giant pull request

Sam's team does use pull requests. Sam opens a PR containing two weeks of accumulated work — 47 files changed, thousands of lines. It sits open for another five days while reviewers slowly work through it, during which main moves on and the PR develops conflicts. How does Sam's PR conflict with trunk-based development principles, and what would a TBD-friendly PR look like instead?

© 2026 Innovation In Software Corporation

QUESTIONS

Question 7 — Marking a release

The team wants to release the current state of the trunk as version 1.2.0, and they need to be able to point back to exactly what was shipped if a problem turns up later. One developer suggests creating a long-lived release-1.2 branch to hold the released code. Under trunk-based development, is a long-lived release branch the right tool here? What should they do instead?

Question 8 — Commit rhythm

Two developers describe their habits. Tomas makes one large commit at the end of each day containing everything he did. Lena makes many small commits throughout the day (C1, C2, C3…) as she completes each small piece, then integrates at day's end. Which pattern better fits trunk-based development, and what concrete benefits does it provide?

© 2026 Innovation In Software Corporation

QUESTIONS

Question 9 — Committed straight to main

Reviewing the team's history, a junior developer notices that on one occasion a teammate ran git pull origin main while on the main branch, made a change, committed it, and ran git push origin main — without ever creating a short-lived branch. The commit landed directly on main. The junior developer asks whether this is "allowed" in trunk-based development, since everything they've learned emphasized short-lived branches. How would you explain what happened and when this is or isn't appropriate?

Question 10 — Two copies of the repository

Nadia works partly from a train with no internet connection. She's worried that without a constant connection to the server, she won't be able to commit her work, view the project's history, or create branches until she's back online. A teammate tells her not to worry. Why is Nadia able to keep working offline, and what does she still need a connection for?

© 2026 Innovation In Software Corporation

ANSWERS

Answer 1 — The long-lived branch

A three-week branch is a long-lived branch, which trunk-based development specifically avoids. Long-lived branches drift far from main, accumulate large diffs, and produce big, painful merges with a high risk of conflicts — the opposite of TBD's goal of frequent integration. Priya should break the dashboard into small tasks, work on short-lived branches (created and merged back within a day or two, within the sprint), and commit small and often. If part of the dashboard isn't ready to be visible, she can merge it behind a feature flag rather than holding it on a branch.

Answer 2 — The unfinished checkout button

Marcus should use a feature flag. He merges the unfinished code into the trunk right away but wraps it in an on/off switch that's set to "off," so customers never see the incomplete button even though the code is live in the application. When the feature is ready, he flips the switch "on" — no new merge and no big-bang integration later. This satisfies daily integration while keeping the trunk releasable and the feature hidden.

© 2026 Innovation In Software Corporation

ANSWERS

Answer 3 — Starting the day

Dana skipped pulling the latest main before starting. Her local main was four days stale, so she branched off outdated code and drifted further out of sync, which caused the conflicts. The correct start-of-day sequence: check out main, pull the latest main from the remote, then create a fresh short-lived branch off that up-to-date trunk before doing any work. Building on current code keeps conflicts small.

Answer 4 — The critical production bug

Alice creates a short-lived branch off the main trunk (e.g., hotfix-123) so she can fix the issue without disturbing the trunk's stability. She makes the necessary changes there. Once she's confident the fix is correct, she pulls the latest main and merges it into her branch to catch any conflicts while they're small, then merges hotfix-123 back into the main trunk and pushes. The trunk now contains the fix. Afterward she deletes the short-lived branch.

© 2026 Innovation In Software Corporation

ANSWERS

Answer 5 — "We don't do code review."

The conclusion is incorrect. "No pull requests / no formal code review" usually means no separate PR gate — not that review is skipped. Review still happens, most often live through pair programming, where code is reviewed in real time as it's written. This works because check-ins are small and frequent (often just a few lines), and it depends on comprehensive automated testing to keep the trunk stable since there's no separate review step to catch regressions. It also requires high trust and is best suited to experienced teams with senior developers.

Answer 6 — The giant pull request

A large, long-running PR works against TBD's core goal of frequent integration. Two weeks of accumulated changes and a five-day-open PR let the branch drift from main, grow a big diff, and develop conflicts — exactly what TBD tries to prevent. A TBD-friendly PR is small and short-lived: opened and merged within a day or so, containing a small set of changes. Sam should break the work into small increments, integrate each frequently, and keep any PR quick to review and merge.

© 2026 Innovation In Software Corporation

ANSWERS

Answer 7 — Marking a release

A long-lived release branch is not the right tool. In TBD the trunk is kept releasable at all times, and you release from the trunk itself. To capture exactly what was shipped, mark the release with a tag (e.g., v1.2.0) that pins the exact commit released. This gives full traceability — including for hotfixes — without maintaining a long-lived release branch that would work against frequent integration.

Answer 8 — Commit rhythm

Lena's pattern of small, frequent commits fits trunk-based development better. Small, frequent commits are explicitly encouraged: they create save points, produce a clear and granular history, make it easier to isolate and undo problems, and keep integration small when branches merge into main at day's end. Tomas's single large end-of-day commit bundles unrelated changes together, obscures history, and makes problems harder to trace and integrate. Both should still integrate into main at least once a day to keep the trunk healthy and conflicts tiny.

© 2026 Innovation In Software Corporation

ANSWERS

Answer 9 — Committed straight to main

The developer pulled the latest main, changed a file, committed, and pushed directly to the trunk with no short-lived branch. This is legitimate in TBD: very small, low-risk changes can go straight onto an up-to-date main, as long as main was pulled first and the trunk stays releasable (backed by automated testing). Anything larger, riskier, or needing review still belongs on a short-lived branch — so it's a judgment call on size and risk, not a license to skip branches routinely.

Answer 10 — Two copies of the repository

Nadia can keep working because Git is a distributed version control system: every contributor keeps a complete copy of the repository, including its full history, locally rather than relying on a central server. She can commit, view history, and create branches entirely offline. A connection is only needed to sync — pushing her commits and pulling teammates' changes. So her work continues on the train; the network matters only when she wants to share or integrate.

© 2026 Innovation In Software Corporation

Lab completed



DEFINITION OF DONE

TIERS



INTRODUCTION



In this module, you will learn why "done" means different things to different people and how a shared, written definition removes that ambiguity, using everyday analogies to sort readiness requirements into levels of importance.

You will then work through the three-tier framework of Must, Should, and Nice, connect it to the MoSCoW method, distinguish a gate from a ladder, and learn to run an honest self-assessment.

© 2026 Innovation In Software Corporation

DONE



"Done" means different things to different people.

- A developer might mean "the code runs on my machine"
- A manager might mean "the feature works"
- A customer means "it works reliably, safely, and I can trust it"

Without a shared definition, teams ship things that *look* finished but aren't truly ready.

© 2026 Innovation In Software Corporation

THE BATH!

When my son was four, I asked him to take a bath. Twenty minutes later he came out of the bathroom, beaming, and announced he was done.

By his definition, he was done. To a four-year-old, "take a bath" means *get in the tub and spend some time there* — a duty he had fulfilled completely, and happily. Getting wet was, in his view, entirely optional; soap didn't enter into it at all. To me, "take a bath" meant *get wet, wash with soap, come out cleaner than you went in*. Neither of us was lying. He had genuinely done exactly what he understood the job to be — he'd sat in the tub, he'd had a lovely time, and by his own honest accounting the bath was complete.

I, of course, was picturing a clean, damp child. What emerged instead was a perfectly dry one, radiant with the pride of a job well done.

© 2026 Innovation In Software Corporation

THE LESSON



"Done" felt obvious to each of us, so neither thought to define it. A shared definition of done — "a bath means *wet, washed with soap, and actually cleaner*" — would have turned two confident finishes into one that was truly complete. This is exactly why teams write "done" down instead of assuming everyone pictures the same thing.

© 2026 Innovation In Software Corporation

TIERS

Think of finishing a project like a house passing inspection before people move in.

- Must-haves: running water, electricity, a locking front door — no one moves in without these
- Should-haves: insulation, working appliances — makes the house livable and comfortable
- Nice-to-haves: smart thermostat, landscaping — polish that adds real value

You'd never rank a smart thermostat above a locking door. Tiers keep priorities straight.

© 2026 Innovation In Software Corporation

TIERS - 2

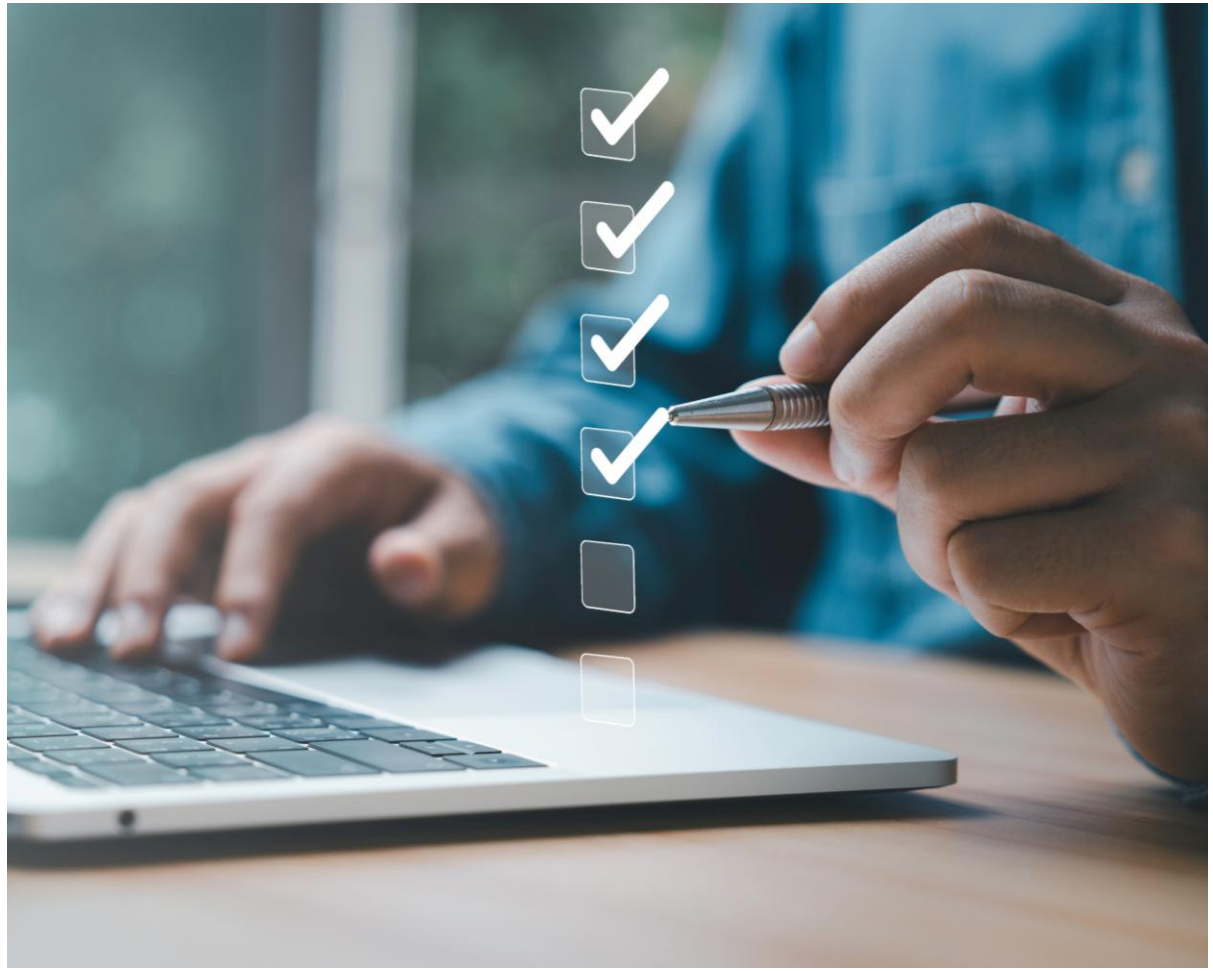
Think of finishing a project like getting a car ready to drive a family across the country.

- Must-haves: working brakes, inflated tires, functioning headlights — no one leaves the driveway without these
- Should-haves: a full tank, a spare tire, a serviced engine — makes the trip reliable and safe over the long haul
- Nice-to-haves: heated seats, a premium sound system, a phone mount — comfort that makes the drive genuinely better

You'd never rank a sound system above working brakes. Tiers keep priorities straight.

© 2026 Innovation In Software Corporation

TIERED DEFINITION OF DONE



A tiered definition of done is a checklist of readiness requirements, sorted into levels of importance. Instead of one flat list where a trivial nicety sits next to a hard blocker, the requirements are organized so that everyone can see at a glance what must be true before release versus what merely should be true.

© 2026 Innovation In Software Corporation

CLEAR REQUIREMENTS

Each requirement is something you can check as met or not met. The value of a definition of done comes from removing ambiguity, so every item is written as a binary, verifiable condition rather than a judgment call. "All unit tests pass," "the API returns a 401 for missing tokens," and "the endpoint is documented in the OpenAPI spec" are checkable — two people looking at the same code will agree on whether each is satisfied.

Vague aspirations like "the code is clean" don't belong, because they can't be objectively confirmed and they reopen the same debate every time. If you can't point to evidence that a requirement is met, it isn't written sharply enough yet.

© 2026 Innovation In Software Corporation

CLEAR REQUIREMENTS - 2

Requirements are grouped into tiers by how essential they are. Not every item carries the same weight, and pretending they do is what makes flat checklists frustrating — a failing security test and a missing changelog entry are not equal, yet a single list treats them the same.

Tiering sorts each requirement by consequence: what breaks or who is harmed if this is skipped? A broken authentication check endangers users and blocks release; an un-updated diagram is cosmetic. Placing each item into a tier forces that conversation up front, when it's calm, rather than during a release crunch when pressure distorts judgment

© 2026 Innovation In Software Corporation

THREE TIERS (OVERVIEW)

Use three tiers. It's the number that maps cleanly to how people actually think about work — must-haves, should-haves, and nice-to-haves — and it gives you exactly enough separation to prioritize without so many levels that the boundaries blur:

Tier 1 — Must: correctness, security, anything that causes real harm if skipped. This is the release gate. Nothing ships until every item here is met, no exceptions.

Tier 2 — Should: strongly expected quality — solid test coverage, documentation, error handling. These can be deferred, but only with a deliberate, visible decision, never silently.

Tier 3 — Nice: polish and maturity — refactoring, extra observability, cosmetic cleanup. These are postponed freely when time is short.

© 2026 Innovation In Software Corporation

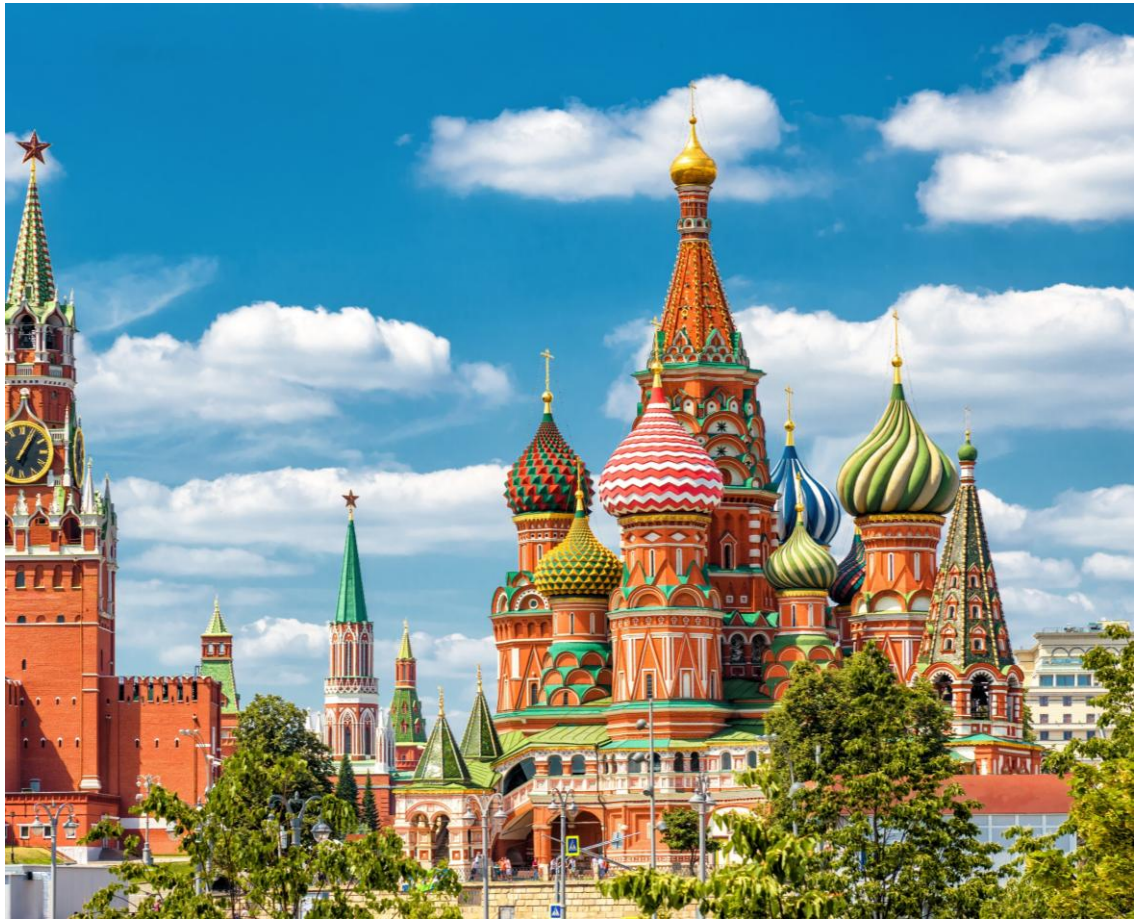
THREE TIERS (OVERVIEW) - 2

Each tier carries has a specific rule if items within the tier are unmet:

- Tier 1 blocks the release
- Tier 2 requires a conscious sign-off to skip
- Tier 3 can be dropped without ceremony.

© 2026 Innovation In Software Corporation

MOSCOW METHOD



MoSCoW is a simple prioritization method that sorts requirements into four categories by necessity: Must have, Should have, Could have, and Won't have (the capital letters spell the name, with the o's added to make it pronounceable).

It's widely used in agile and project planning precisely because it forces the same conversation a tiered definition of done does — separating what's essential from what's optional before deadline pressure makes the call for you.

© 2026 Innovation In Software Corporation

MOSCOW METHOD

The alignment is nearly one-to-one:

- Must → Tier 1. The non-negotiable release gate — correctness and security. Ship-blockers in both models.
- Should → Tier 2. Strongly expected but deferrable with a deliberate, visible decision.
- Could → Tier 3. Nice-to-have polish, dropped freely when time is short.
- Won't → out of scope. Not a tier at all, but the useful habit of writing down what you're deliberately *not* doing this round, so deferred items don't quietly creep back in as assumed work.

The three tiers map straight onto Must/Should/Could, and MoSCoW's fourth category gives you vocabulary for the scope boundary around the whole checklist.

https://en.wikipedia.org/wiki/MoSCoW_method

© 2026 Innovation In Software Corporation

GATE VERSUS LADDER

"Gate" versus "ladder" is the key distinction. A gate you must pass through — no partial credit for going live. A ladder you climb over time; you don't need to be at the top today, but you should know which rung you're on.

- Tier 1 — Production Gate: the must-pass basics. Fail any of these and you do not go live.
- Tier 2 — Baseline Maturity: the marks of a solid, professional, maintainable result.
- Tier 3 — Advanced Patterns: the sophisticated extras that set the best work apart.

© 2026 Innovation In Software Corporation

TIER 1 - TECHNICAL

Tier 1 answers: "Is it safe and functional enough to be used by real people right now?"

Typical Tier 1 items:

- It actually works for its core purpose
- It's secure — no obvious way to break in or leak sensitive data
- It handles errors without crashing or corrupting things
- Someone can tell whether it's up or down

If any Tier 1 item fails, the answer to "are we ready?" is simply no. Yes, one hard Tier 1 failure stops the launch, even if everything else is perfect.

© 2026 Innovation In Software Corporation

TIER 1 – TECHNICAL - FAILURES

These are examples of Tier 1 failures — any one is a stop sign:

- The login can be bypassed
- A common user action makes the whole system crash
- Sensitive data is exposed to the wrong people
- No one can tell the service has gone down until customers complain

Important: these aren't "nice to fix later" problems.

© 2026 Innovation In Software Corporation

TIER 2 - TECHNICAL

Tier 2 answers: "Is this built well enough to maintain, trust, and grow?"

Typical Tier 2 items:

- Clear documentation so others can understand it
- Consistent, predictable behavior across situations
- Reasonable performance under normal load
- Tests that catch problems before customers do

You can live having missed some Tier 2 items — but they're what separate "barely working" from "professionally built." Missing Tier 2 items become tomorrow's headaches.

© 2026 Innovation In Software Corporation

TIER 2

Is there a common technical term that can be applied to unmet Tier 2 items?



5 MINUTES

© 2026 Innovation In Software Corporation

TECHNICAL DEBT



Deferred Tier 2 items are one of the clearest and most defensible examples of *technical debt*. In fact they're the healthy kind, and the tiered model is largely a mechanism for making that debt visible and deliberate rather than accidental.

© 2026 Innovation In Software Corporation

TIER 3 - TECHNICAL

Tier 3 answers: "Does this reflect advanced, best-in-class practice?"

Typical Tier 3 items:

- Sophisticated monitoring that predicts problems before they happen
- Graceful handling of rare, extreme situations
- Automation that reduces manual effort and human error
- Design choices that make future scaling easy

Mature organizations invest here because it compounds — quicker problem resolution, faster growth, and lower long-term cost.

Tier 3 is where good becomes excellent.

© 2026 Innovation In Software Corporation

PERFORM A SELF ASSESSMENT

A self-assessment is simply you walking the checklist and rating each item honestly.

- Get the tiered checklist for your project
- Go through each item, one at a time
- Gather evidence — don't rely on memory or hope
- Record a rating for each item (next slides)
- Total up: is every Tier 1 item passed?

You're grading your own work, which takes discipline. The goal isn't a good-looking score; it's an *accurate* one.

A self-assessment that hides problems only guarantees you'll meet them later at a worse time.

© 2026 Innovation In Software Corporation

RATING

For each checklist item, record one of four ratings:

- Pass — fully meets the requirement, with evidence
- Fail — does not meet the requirement
- Partial — partly met; some work still remains
- N/A — genuinely doesn't apply to this project

These four labels cover every situation without forcing a false yes-or-no.

"Partial" prevents you from marking something done when it's half-finished, and "N/A" prevents you from being penalized for a requirement that legitimately doesn't apply.

© 2026 Innovation In Software Corporation

RATING - 2

Guidance for honest labeling:

- Use Pass only with real evidence — when in doubt, it's Partial
- A single Fail in Tier 1 stops the launch, full stop
- Partial must name what's left to do, so it becomes an action item
- Use N/A sparingly, and write down *why* it doesn't apply

© 2026 Innovation In Software Corporation

TIER OVERVIEW TABLE

The tier overview table is the "here's the whole framework on one slide" reference. It lays out all three tiers side by side across the same set of columns, so a learner can see at a glance how the tiers differ and — most importantly — which one actually stops a launch.

Tier	Name	Question it answers	Treatment	Blocks launch?
1	Production Gate	Is it safe and functional enough for real users now?	Gate — pass/fail	Yes
2	Baseline Maturity	Is it built well enough to maintain and trust?	Ladder — climb over time	No
3	Advanced Patterns	Does it reflect best-in-class practice?	Ladder — climb over time	No

© 2026 Innovation In Software Corporation

SELF-ASSESSMENT SCORECARD

Here's the self-assessment scorecard completed for the coffee-shop opening, trimmed to seven representative items — spanning all three tiers, every row rated and backed by evidence:

Item	Tier	Rating	Evidence / Note
Health & food-safety permit approved	1	**Pass**	Certificate #4471 received, posted by register
Register can take payment	1	**Pass**	Ran 5 test sales (card + cash), all cleared
Fire exits clear, extinguisher present	1	**Pass**	Inspected morning of; extinguisher tag current
Staff trained on menu & register	2	**Partial**	2 of 3 staff signed off; new hire trains Thu
Reliable supplier for beans & milk	2	**Pass**	Weekly standing order confirmed with roaster
Cleaning schedule written & assigned	2	**Fail**	Not yet written — owner to draft before Fri
Loyalty app with demand prediction	3	**N/A**	Deferred to Q4; not in scope for opening

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned why "done" means different things to different people, and how a shared, written definition removes that ambiguity. You sorted readiness requirements into tiers by consequence, each written as a verifiable condition.

You worked through the three-tier framework of Must, Should, and Nice, connected it to MoSCoW, and distinguished a gate from a ladder. You also learned to run an honest self-assessment against real evidence.

© 2026 Innovation In Software Corporation

LAB 10 – SCENARIO



In this lab you'll act as the release reviewer for a payment-checkout API due to launch tomorrow, deciding whether it's actually ready to ship.

GOAL

This lab moves the tiered definition of done from concept to practice. A tiered checklist replaces vague confidence with a shared standard: Tier 1 (Must) is the production gate, Tier 2 (Should) is baseline maturity deferred only with a conscious sign-off, and Tier 3 (Nice) is polish you can drop freely. Here you'll apply it to a realistic project where requirements aren't a tidy list — they're embedded in a narrative, mixed together, some met and some not.

You'll practice three skills in sequence. First, determine the requirements by reading the situation and pulling out each distinct, verifiable readiness condition.

© 2026 Innovation In Software Corporation

GOAL - 2

Next, assign each a tier and a rating — Tier 1, 2, or 3, rated Pass, Fail, or Partial on the evidence. Finally, make the release decision using the gate rule: any unmet Tier 1 blocks launch, unmet Tier 2 ships only with a recorded sign-off (deliberate technical debt), and Tier 3 can wait. The point isn't a good-looking score — it's an accurate one, and defending the ship/no-ship call as you would in a real release review.

© 2026 Innovation In Software Corporation

THE SCENARIO – CHECKOUT API

Your team has built a payment-checkout API for an online store, and the launch date is tomorrow. Walking the code and talking to the developers, you gather the following.

The endpoint correctly processes a valid order and charges the card — this has been demonstrated repeatedly and works reliably. Login and the main order endpoints are protected by proper token authentication. There is a health endpoint that accurately reports whether the service is up or down. A fire-drill test confirmed the on-call alert fires when the service goes down.

© 2026 Innovation In Software Corporation

THE SCENARIO – CHECKOUT API

However, one developer mentions that the "apply coupon" endpoint doesn't check the auth token — anyone with the URL can call it directly. When the payment processor times out, the service currently returns a 500 and leaves the order in a half-charged state, with the customer's card charged but no order recorded.

On the quality side, unit tests exist and pass for the happy path, but there are none for declined cards or network timeouts, and the OpenAPI documentation is missing three of the eight endpoints. Finally, the team had planned an optional feature that would automatically retry a payment when the processor is briefly unavailable, but they never built it.

© 2026 Innovation In Software Corporation

YOUR TASKS

Here are your tasks:

- Determine the requirements — pull each distinct readiness requirement out of the narrative.
- Assign a tier and rating to each: Tier 1 (Must), Tier 2 (Should), or Tier 3 (Nice), rated Pass, Fail, or Partial.
- Decide releasability — can this ship tomorrow? Name what blocks the launch, what could be deferred with a sign-off, and what can be dropped.

When you're finished, check your work against the answer key in README.md.

© 2026 Innovation In Software Corporation

Lab completed



SPLUNK

INFORMATION EXHAUST



INTRODUCTION



In this module, you will learn what Splunk is and how it helps you identify, analyze, and present machine data through events, timelines, and timestamps, along with its documentation and Splunkbase.

You will then focus on indexing — hosts, source types, and metadata — and bring in data by monitoring files, directories, and the network.

© 2026 Innovation In Software Corporation

WHAT IS SPLUNK



Splunk is a software platform that collects, indexes, searches, analyzes, and visualizes machine-generated data. It helps organizations turn massive amounts of logs, events, and metrics from IT systems, applications, networks, cloud services, and security devices into valuable operational and business insights.

© 2026 Innovation In Software Corporation

JUST TO REITERATE

Learning Splunk can feel overwhelming at first because it includes hundreds of features, settings, apps, and administrative options. Even experienced Splunk professionals rarely know every capability of the platform. The key is to focus on the fundamentals that provide the greatest value: understanding events, learning how data is organized, and becoming comfortable with the Search Processing Language (SPL).

Once you can confidently search, filter, transform, and visualize data, you will have the skills needed for the vast majority of real-world tasks. As your experience grows, you can gradually explore more advanced features when your projects require them.

© 2026 Innovation In Software Corporation

IDENTIFY, ANALYZE, PRESENT

Using Splunk generally involves three simple phases. You begin with a question, locate the relevant machine data, analyze it, and then present the results in a meaningful way.

The three phases are:

- Identify the machine data that can answer your question.
- Analyze the data to uncover patterns, trends, or the root cause of a problem.
- Present the results using reports, dashboards, charts, or graphs that are easy to understand.

© 2026 Innovation In Software Corporation

How Splunk Works: 3 Simple Phases

Using Splunk generally involves three simple phases. You begin with a question, locate the relevant machine data, analyze it, and then present the results in a meaningful way.



1 IDENTIFY

Find the machine data that can answer your question.



2 ANALYZE

Analyze the data to uncover patterns, trends, or the root cause of a problem.



Start with a question

"Why is the website slow?"
"Why did the server fail?"
"What's causing the errors?"

Locate relevant machine data

- Servers & Applications
- Networks & Devices
- Security Logs
- Cloud Services
- Databases

Search & Explore

Find events and patterns in your data.

Discover Insights

Identify trends, outliers, and correlations.

Find Root Cause

Drill down to determine what's really happening.

Dashboards

Real-time visibility at a glance.

Reports

Detailed analysis and summaries.

Charts & Graphs

Visualize trends and compare data.

© 2026 Innovation In Software Corporation

WORKFLOW

Step 1 – A Problem Occurs
A server, application, or network device begins to fail or perform poorly. Monitoring software detects the issue and generates an alert. However, the logs needed to diagnose the problem are scattered across many systems and may contain millions of unrelated events.

Step 2 – Splunk Collects and Indexes the Data
Splunk gathers machine data from servers, applications, cloud services, databases, and network devices. It indexes this data into a central repository, allowing administrators to search all systems from one place instead of logging into dozens of individual machines.

© 2026 Innovation In Software Corporation

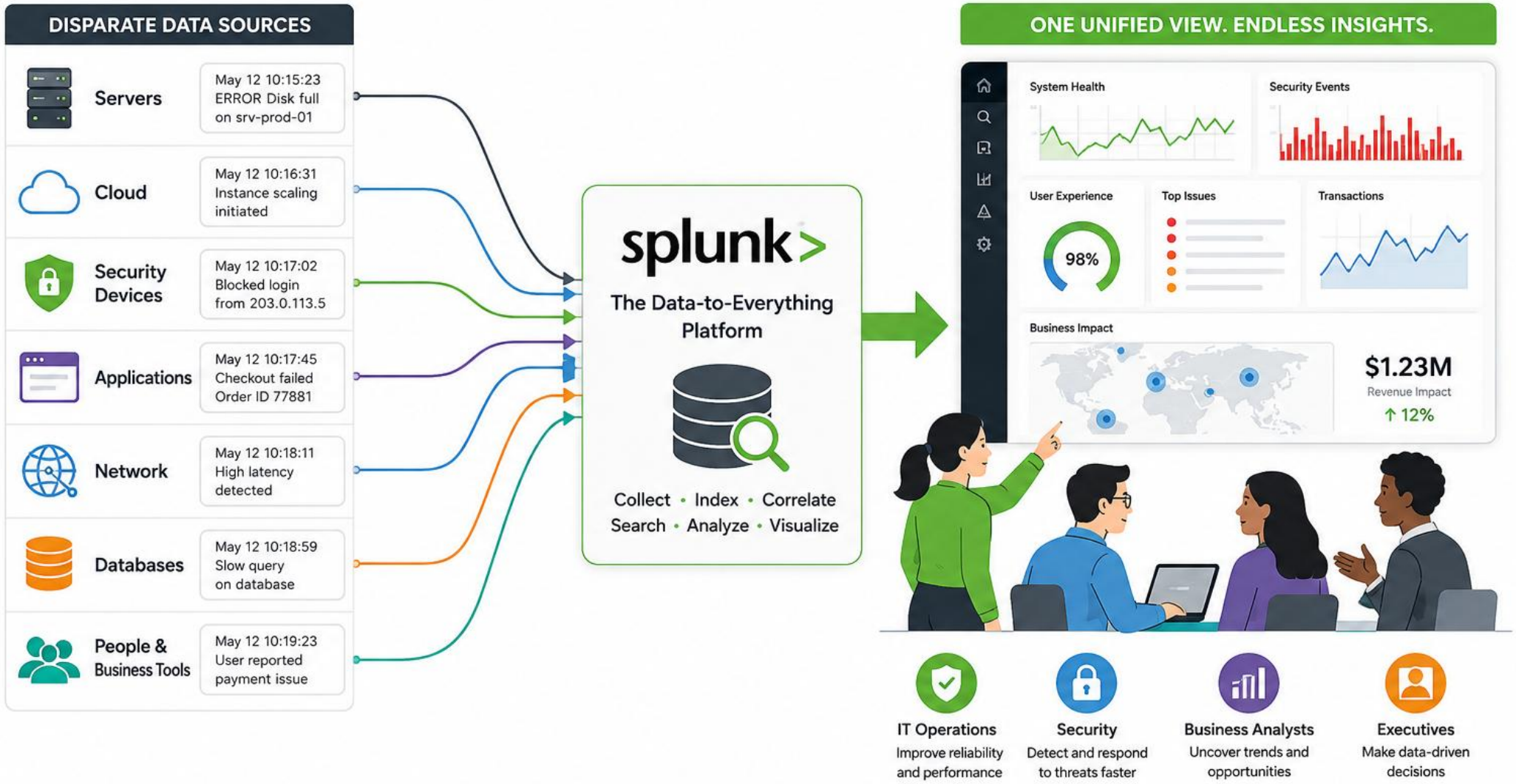
WORKFLOW - 2

Step 3 – Search, Investigate, and Find the Root Cause
Using the indexed data, Splunk quickly searches across all logs to identify when the problem first occurred. Analysts can narrow the search to the relevant time period, correlate related events across multiple systems, and determine the root cause of the failure.

Step 4 – Resolve and Prevent Future Problems
After identifying the cause, the issue can be corrected. Splunk dashboards and reports document what happened, while alerts can be created to automatically detect the same problem in the future, reducing downtime and improving system reliability.

© 2026 Innovation In Software Corporation

Splunk's ability to combine and collate disparate data from any source gives you one unified view to drive faster insights and better decisions.



© 2026 Innovation In Software Corporation

EVENTS

Events are the core building blocks of Splunk. Every piece of machine data that Splunk collects—such as a log entry, application message, security alert, network event, or system error—is stored as an event. Each event typically includes a timestamp, the source of the data, and the event's contents.

Think of an event as a single line in a log file. By itself, one event provides only a small amount of information. However, when Splunk collects millions of events from many systems, it can search, correlate, and analyze them to reveal patterns, detect anomalies, troubleshoot problems, and identify security threats.

© 2026 Innovation In Software Corporation

EVENTS - 2

Almost everything you do in Splunk revolves around events. Searches retrieve events, dashboards summarize events, reports analyze events, alerts monitor events, and visualizations display event data. Simply put, events are the fundamental unit of data upon which every Splunk feature is built.

© 2026 Innovation In Software Corporation

TIMELINES

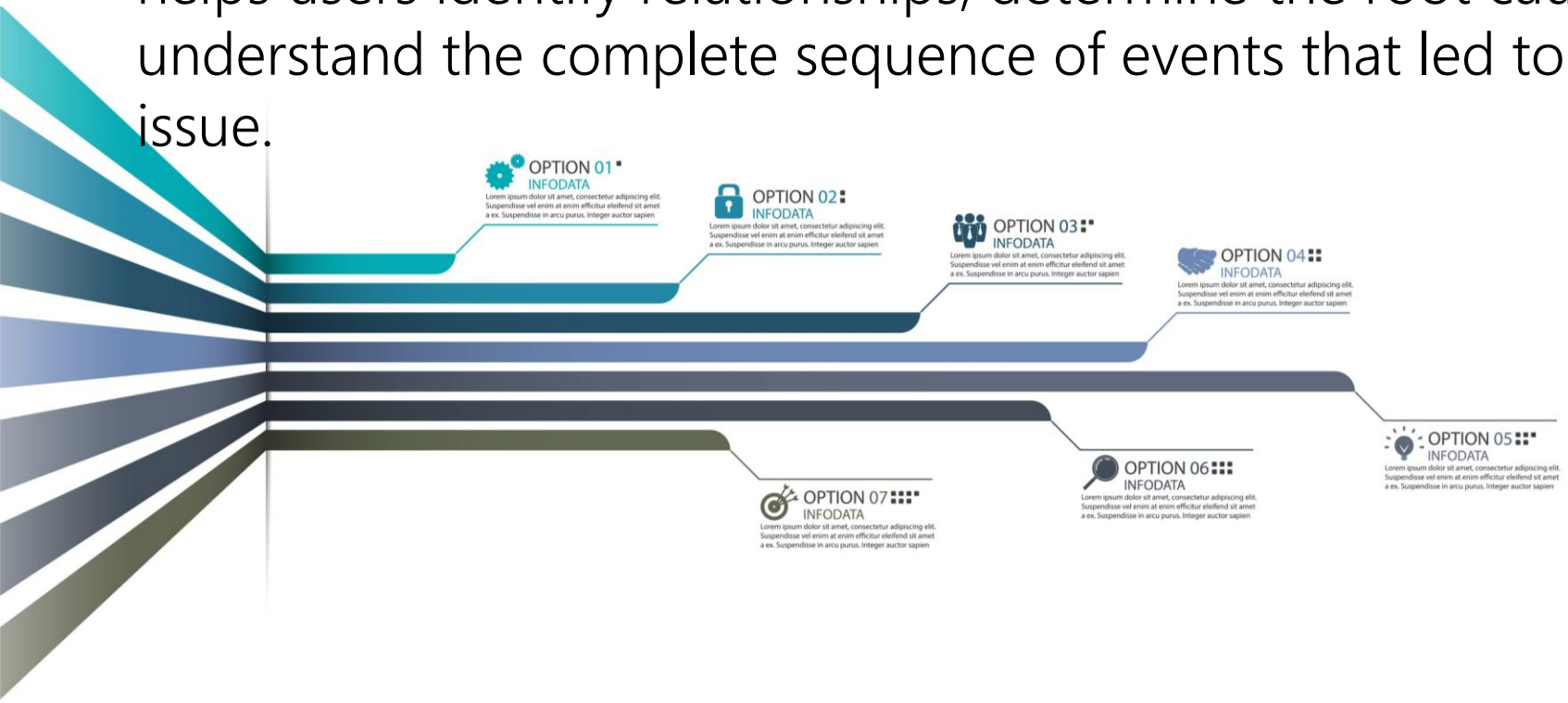
Time is one of the most important pieces of information in Splunk. Every event that Splunk indexes is automatically assigned a timestamp, allowing it to organize data into a chronological timeline. Because virtually every machine event occurs at a specific moment, time provides the common thread that links data from different systems together.

These automatic timelines make it easy to answer questions such as "When did the problem begin?", "What happened just before the failure?", and "Which events occurred afterward?" Splunk can quickly narrow a search to a specific second, minute, hour, or day, allowing administrators to focus on only the events that matter instead of searching through millions of log entries.

© 2026 Innovation In Software Corporation

TIMELINES - 2

Timelines also enable Splunk to correlate events across multiple systems. For example, a database slowdown, an application error, and a network failure may all occur within the same 30-second window. By automatically aligning events by time, Splunk helps users identify relationships, determine the root cause, and understand the complete sequence of events that led to an issue.



© 2026 Innovation In Software Corporation

TIMESTAMP

When an event does not have an explicit time occurrence, how is the timestamp determined?





5 MINUTES

© 2026 Innovation In Software Corporation

PROACTIVE



Although Splunk is often associated with troubleshooting problems after they occur, it is equally valuable for preventing problems before they impact users. By continuously analyzing machine data in real time, Splunk gives organizations visibility into the health of applications, servers, networks, and cloud services.

Using predictive analytics and machine learning, Splunk can forecast potential problems, detect anomalies, and generate early alerts. This helps organizations move from reactive troubleshooting to proactive operations, reducing downtime and improving reliability.

© 2026 Innovation In Software Corporation

FLAVORS

In a Splunk Fundamentals class, “Splunk flavors” usually means the main ways Splunk is packaged or used.

Splunk Enterprise

This is the self-managed version of Splunk. You install it on your own server, laptop, VM, or company infrastructure. It is commonly used for labs, learning, on-premises environments, and companies that want more control over their deployment.

Splunk Cloud Platform

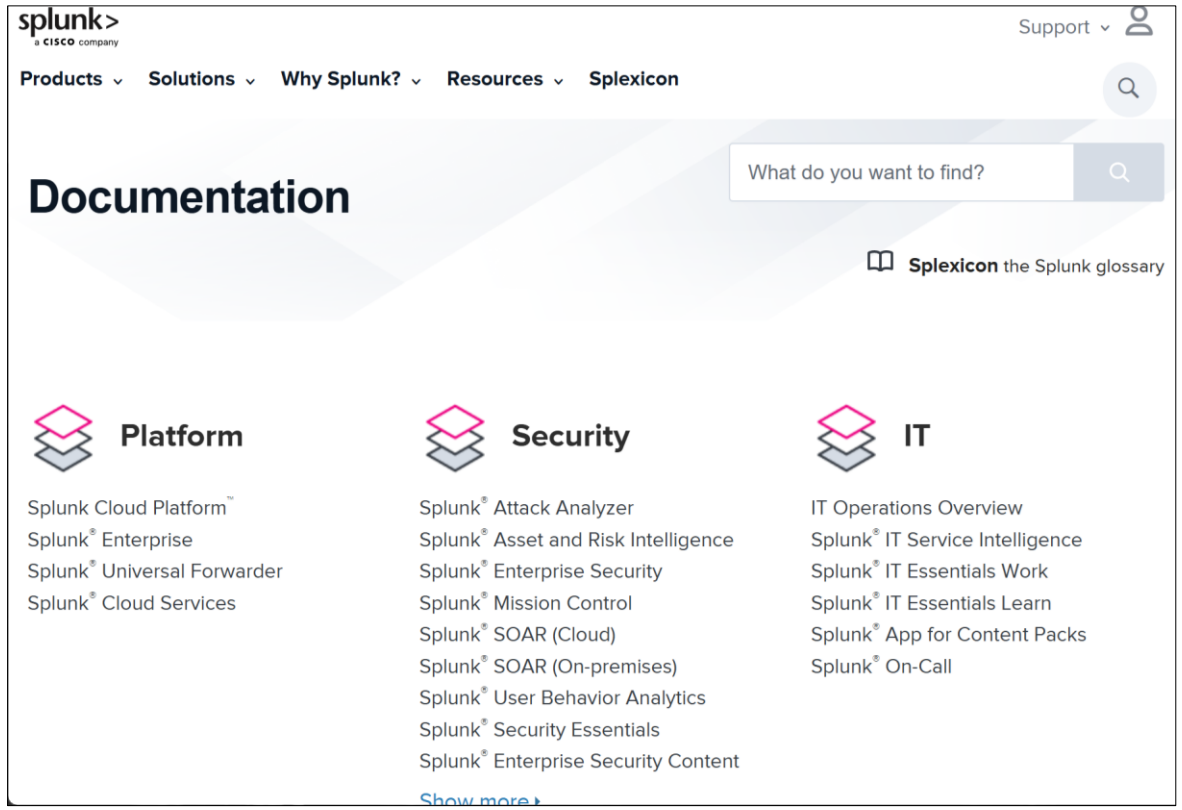
This is Splunk hosted and managed in the cloud by Splunk. You still use Splunk to search, report, alert, and build dashboards, but Splunk handles much of the infrastructure management. Splunk lists both Splunk Enterprise and Splunk Cloud Platform as trial/download options.

Splunk Universal Forwarder

This is a lightweight agent installed on machines that generate data. Its job is to collect logs and send them to Splunk. It is not where you normally search data.

© 2026 Innovation In Software Corporation

SPLUNK DOCUMENTATION



Splunk Documentation – The official documentation for Splunk Enterprise, Splunk Cloud, SPL, administration, installation, and tutorials. This should be your primary technical reference throughout the course.

https://docs.splunk.com

© 2026 Innovation In Software Corporation

SPLUNKBASE



Splunkbase – The official marketplace for Splunk apps, add-ons, integrations, dashboards, and extensions developed by Splunk and the community. Many organizations use Splunkbase to add support for new technologies without writing custom code.

https://splunkbase.splunk.com

© 2026 Innovation In Software Corporation

ERIK SWAN AND ROB DAS

Splunk was founded in 2003 after co-founders Erik Swan and Rob Das recognized that troubleshooting IT systems was slow and frustrating. IT professionals often spent hours manually searching log files to diagnose problems.

Many compared the process to crawling through dark caves with outdated tools. This inspired a simple question: "Why can't searching IT data be as easy as using Google?"

That question became Splunk's original vision—to make machine data fast, searchable, and easy to understand, helping organizations quickly find answers hidden in their data.

© 2026 Innovation In Software Corporation

INDEXING



Indexing is the process of reading machine data and storing it in a format optimized for fast searching.

During indexing Splunk:

- Reads incoming machine data
- Breaks data into events
- Assigns timestamps
- Extracts metadata
- Compresses and stores data in indexes

© 2026 Innovation In Software Corporation

INDEXING - 2



Once indexed, Splunk can search millions or even billions of events in seconds. This enables users to quickly troubleshoot problems, detect security threats, identify trends, create dashboards, and generate alerts, making Splunk fast, scalable, and highly effective.

© 2026 Innovation In Software Corporation

Splunk Indexing Workflow

From raw machine data to searchable, time-ordered events.



1. Machine Data

Raw data is generated by systems, applications, devices, and users.

2. Input

Splunk collects data from files, directories, network ports, scripts, APIs, and forwarders.

3. Parsing

Splunk reads the raw data and extracts metadata such as host, source, and source type.

Host: web01
Source: /var/log/access.log
Source Type: access_combined

4. Timestamp Recognition

Splunk detects the correct timestamp and normalizes the time.

Event Time:
2026-06-29 10:15:23 -0700

5. Event Breaking

Splunk splits the data into individual events based on line breaks, delimiters, or custom rules.

Lines of raw data → Individual events

6. Indexing

Events are compressed and stored in indexes for fast and efficient searching.

Indexes
- main
- web
- security
- ...

7. Searchable Events

Events are now available for search, analysis, visualization, alerting, and reporting.

Search & Analyze

Dashboards & Alerts

© 2026 Innovation In Software Corporation

HOST

The host field identifies the computer, server, or device that generated an event. Every event in Splunk has exactly one host value, which is automatically assigned during data ingestion. Although the host is often the system's hostname, it can also be a custom identifier, such as a firewall, router, cloud instance, or application server. Because the host is indexed metadata, it is one of the fastest and most commonly used fields for searching and filtering events.

Key points:

- Automatically generated metadata field.
- Every event has exactly one host value.
- Host names are case-sensitive (SERVER01 and server01 are treated as different hosts).
- The host value can be overridden during data input if needed.
- Changing a host name affects only newly indexed events; previously indexed events retain their original host value.

© 2026 Innovation In Software Corporation

SOURCE TYPES

A Source Type tells Splunk what kind of data it is reading.

Source Type	Description	Typical Source
access_combined	Apache web server access logs with fields such as IP, URI, and status.	Apache web servers
syslog	Standard Linux and network device log format.	Linux, firewalls, routers
WinEventLog	Windows system, application, and security events.	Windows Event Viewer
csv	Structured comma-separated data; each row becomes an event.	Spreadsheets, database exports
json	Structured key-value data with automatic field extraction.	APIs, cloud services, applications

© 2026 Innovation In Software Corporation

SPLUNK METADATA

Every event automatically receives metadata. Splunk automatically assigns the timestamp, host, and source,

Common metadata includes:

- host
- source
- sourcetype
- index
- timestamp

These fields are used in almost every search.

© 2026 Innovation In Software Corporation

DATA SOURCES

A data source is simply the location where Splunk obtains machine-generated data. This data may come from a log file, an operating system, a network device, a cloud service, an application, or almost any other system that produces information. Splunk continuously collects this data, processes it, and stores it so it can be searched and analyzed.

One of Splunk's greatest strengths is that it can combine data from many different sources into a single searchable index. For example, a security investigation might correlate Windows Event Logs, Linux syslog messages, firewall logs, web server logs, and cloud activity—all in one search. This flexibility gives organizations a unified view of their IT environment, making it easier to troubleshoot problems, detect security threats, monitor system health, and identify trends without switching between multiple tools.

© 2026 Innovation In Software Corporation

DATA SOURCES - 2

Data Source	Description
Log Files	Collects application, web server, and system log files.
Directories	Monitors entire folders and automatically indexes new files.
Windows Event Logs	Collects security, application, and system events from Windows.
Linux Syslog	Captures operating system and application messages from Linux/Unix systems.
TCP Ports	Receives streaming data from applications over TCP connections.

© 2026 Innovation In Software Corporation

DATA SOURCES - 3

Data Source	Description
UDP Ports	Collects network data sent using the UDP protocol.
Scripts	Runs scripts to gather custom or generated data.
APIs	Retrieves data from applications and web services using APIs.
Cloud Services	Ingests logs and events from cloud platforms such as AWS, Azure, and Google Cloud.

© 2026 Innovation In Software Corporation

MONITORING FILES AND DIRECTORIES

One of Splunk's most common ways of collecting data is by monitoring files and directories. Instead of manually importing data, you tell Splunk where the data is located. Splunk continuously watches those locations and automatically reads any new information as it is written.

Splunk can monitor:

- Individual files (such as web.log)
- Rotating log files (when applications create new log files as older ones fill up)
- Entire directories (folders) containing many log files

As new events are added to these files, Splunk immediately indexes them.

© 2026 Innovation In Software Corporation

MONITORING FILES AND DIRECTORIES - 2

Instead of configuring each log file individually, you can simply tell Splunk to monitor an entire folder, such as:

C:\Logs\

Splunk automatically discovers and monitors every log file in that folder, including:

app1.log

app2.log

web.log

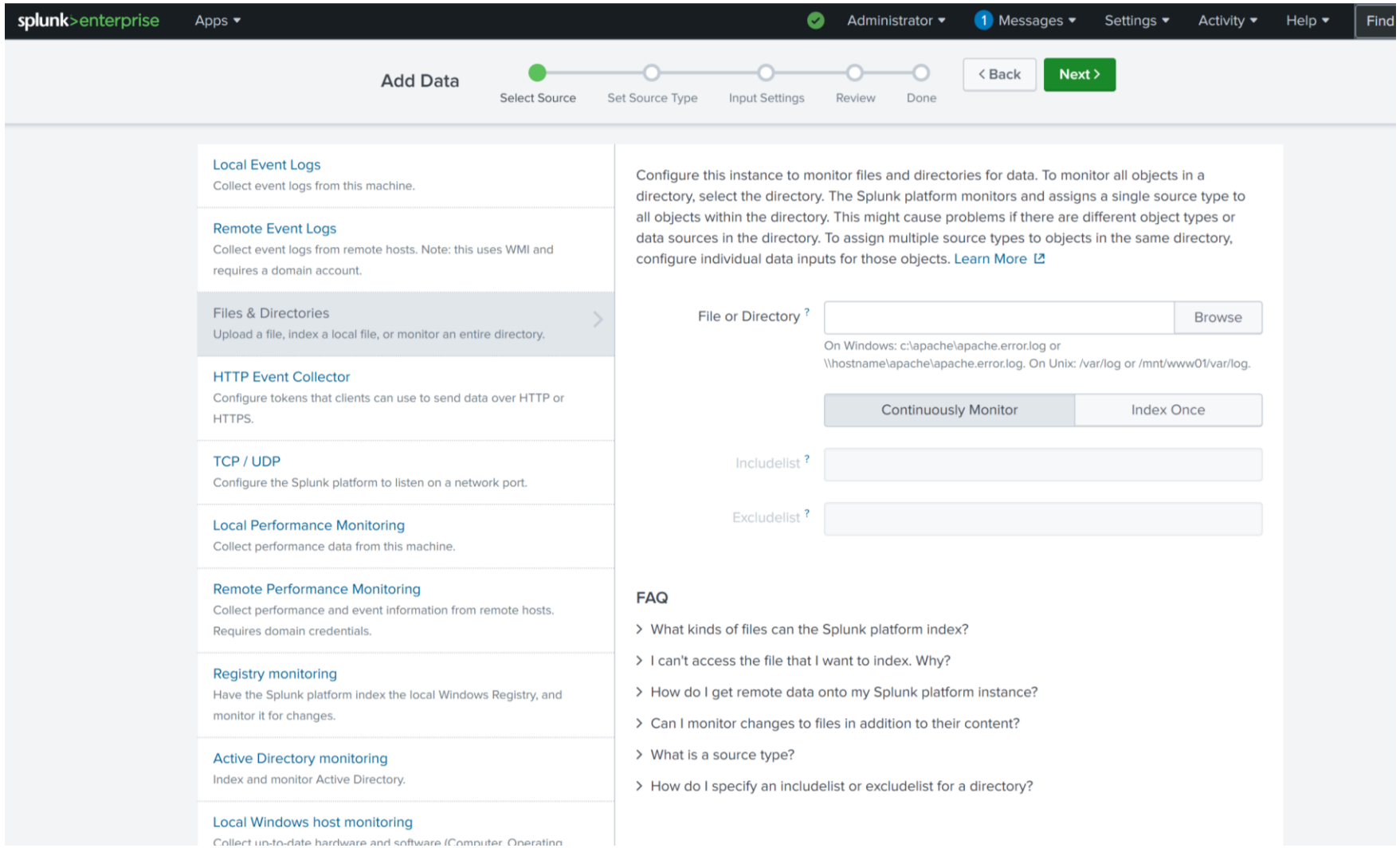
error.log

If a new file, Splunk automatically begins indexing it—no configuration changes are needed.

This approach also works with log rotation, where applications create new log files as older ones are archived.

© 2026 Innovation In Software Corporation

MONITORING FILES AND DIRECTORIES - 3



© 2026 Innovation In Software Corporation

INCLUDE VERSUS EXCLUDE LIST

Include List

The Include List limits which files Splunk monitors within a directory.

For example: *.log

Splunk monitors only files ending in .log. This is useful when a directory contains many different file types.

Exclude List

The Exclude List tells Splunk which files to ignore.

For example: *.bak

Splunk ignores backup files ending in .bak while continuing to monitor the remaining log files.

© 2026 Innovation In Software Corporation

CONTINUOUSLY MONITOR



Continuously Monitor is the default.

Splunk continuously watches the selected file or directory. As new log entries are written, Splunk automatically indexes only the new data.

This is the best choice for:

- Application logs
- Web server logs
- System logs

Any file that is actively growing

© 2026 Innovation In Software Corporation

INDEX ONCE



This option imports the data a single time.

Splunk reads the file once, indexes its contents, and then stops monitoring.

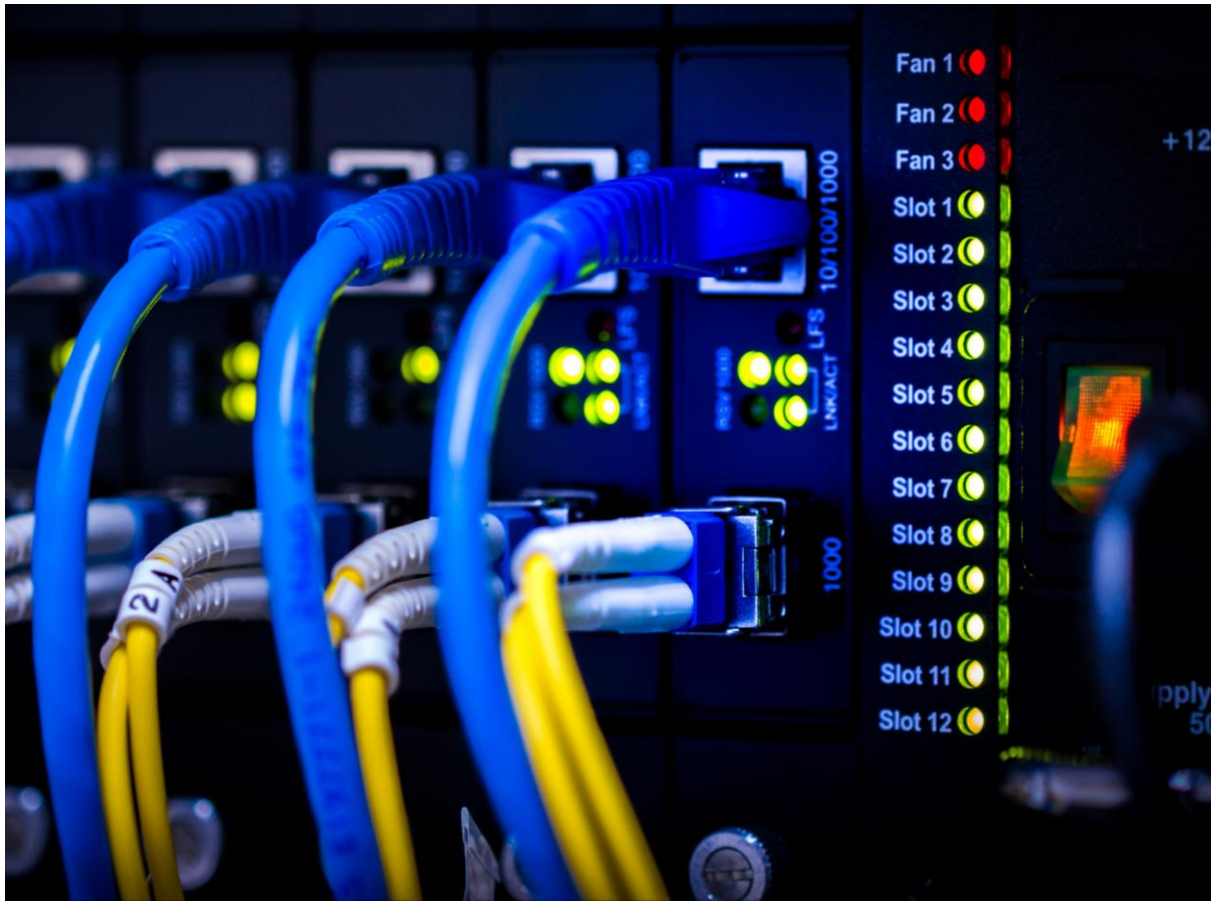
Use this option for:

- Historical log files
- Archived data
- One-time imports

Do not use this option for logs that are continuously updated.

© 2026 Innovation In Software Corporation

NETWORK MONITORING



Splunk can also monitor and collect data by listening for incoming network traffic. Instead of reading log files stored on disk, Splunk waits for devices and applications to send log events directly over the network.

Common protocols include TCP, UDP, and Syslog. Typical devices and systems that send data directly to Splunk include:

- Firewalls
- Routers and switches
- Servers
- Network appliances
- Security devices

© 2026 Innovation In Software Corporation

NETWORK MONITORING - 2

The important point is that Splunk does not monitor an IP address directly. Instead, it listens on a TCP or UDP port, and devices at one or more IP addresses send their log messages to that port.

For example, if your Splunk server has the IP address:

192.168.1.100

and listens on UDP port 514, a firewall at 192.168.1.50 can be configured to send its syslog messages to:

192.168.1.100:514

Splunk then indexes everything it receives

© 2026 Innovation In Software Corporation

TIME



Time is central to Splunk.

During indexing Splunk automatically:

- Detects timestamps
- Converts time zones
- Normalizes date formats
- Orders events chronologically

This enables powerful timeline analysis

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned what Splunk does and how it turns machine data into insight through its workflow, events, timelines, and timestamps. You explored proactive monitoring, the documentation, and Splunkbase.

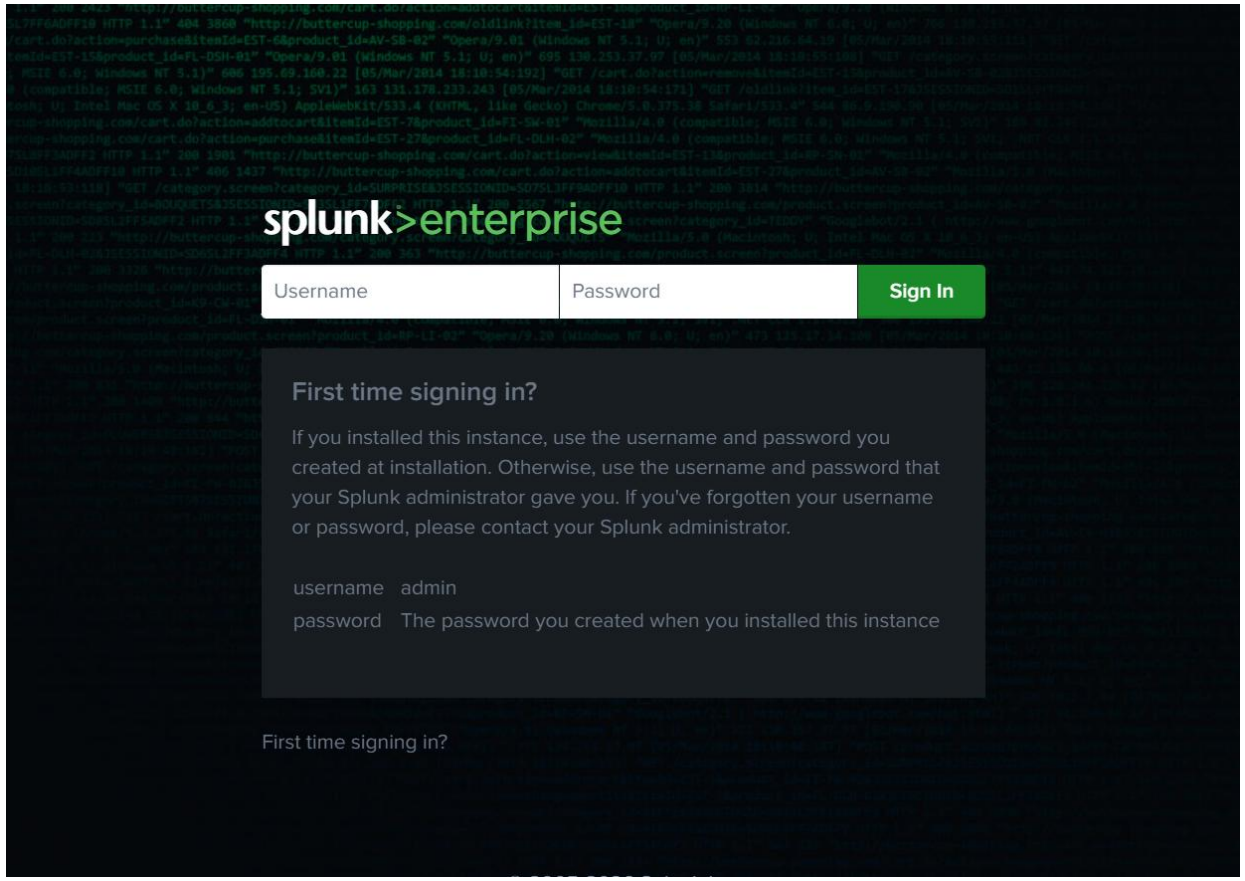
You also covered indexing concepts — hosts, source types, and metadata — and configured data inputs from files, directories, and the network.

© 2026 Innovation In Software Corporation

LAB 11 – SPLUNK ADD-ON



PART 1 – LOG IN TO SPLUNK



Open a web browser.

Browse to:

http://localhost:8000

Log in using your Splunk credentials.

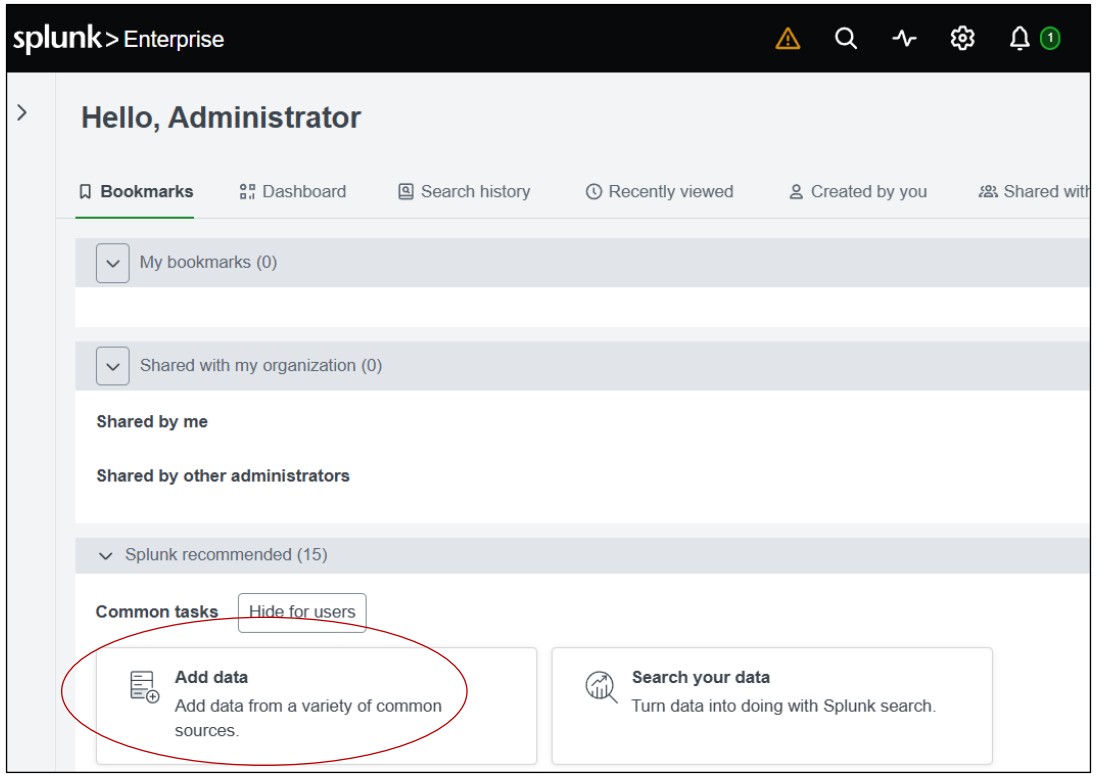
Username: admin

Password: password

After logging in, the Splunk Home page appears.

© 2026 Innovation In Software Corporation

PART 2 – UPDATE SAMPLE DATA



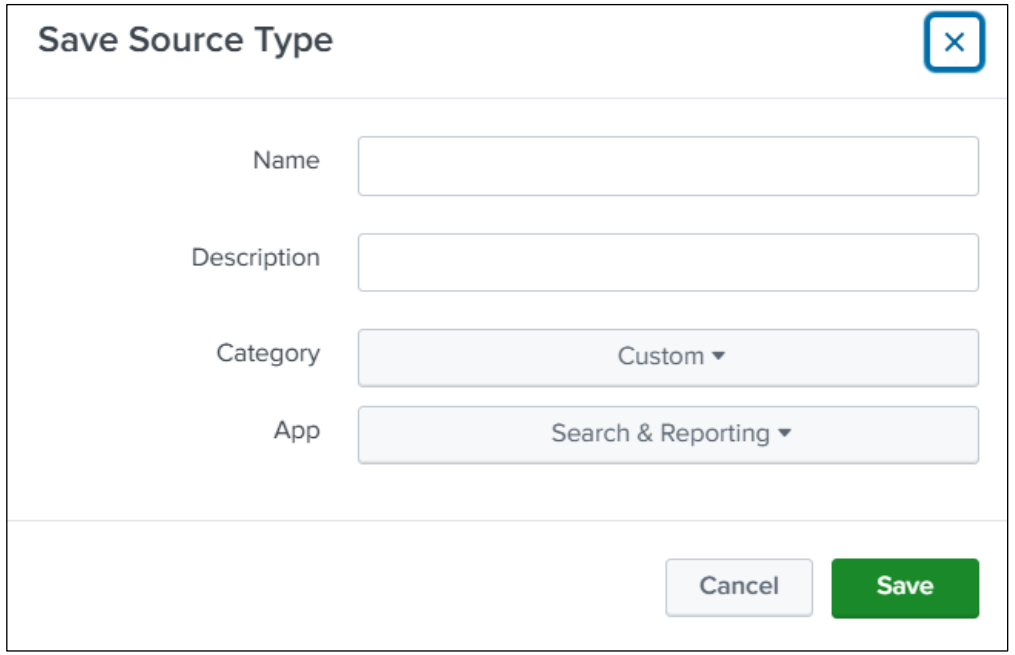
We will use a web server access log.

From the Splunk Home page, click Add Data.

- Select *Add data* and Upload button
- Browse and select the supplied log file:
- lab11\web.log
- Press Next buttons and accept the default settings.

© 2026 Innovation In Software Corporation

PART 2 – UPDATE SAMPLE DATA

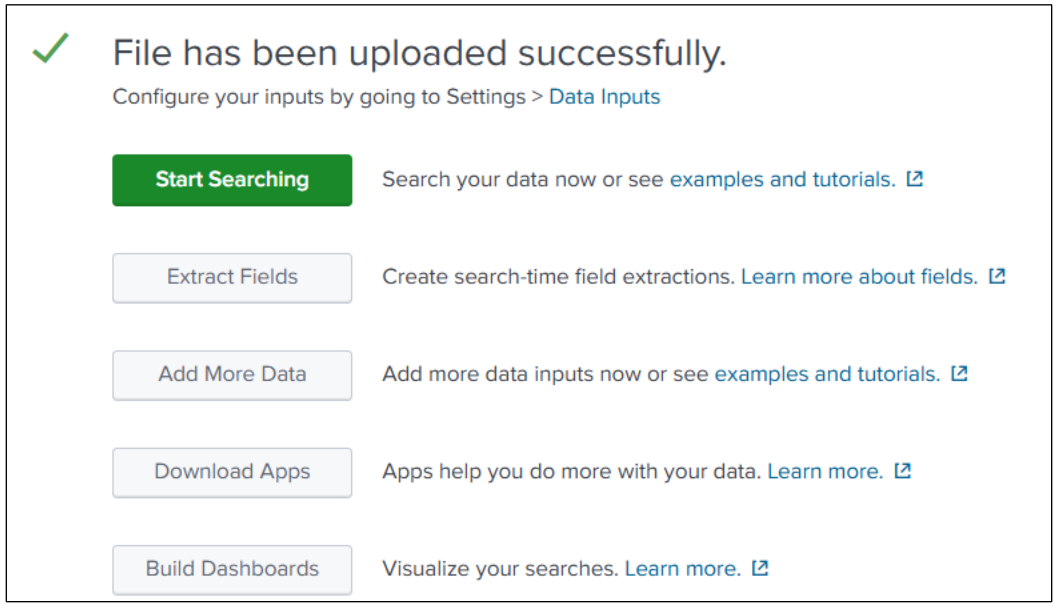


For the Save Source Type windows, enter Name as *First*.

- Press the *Save* button
- Press the *Review* button
- Press the *Submit* button

© 2026 Innovation In Software Corporation

PART 3 – VIEW ALL EVENTS



Press *Start Searching* button.

Enter the following as the New Search:

index=main

Notice:

- Every line is an event.
- Each event has a timestamp.
- Splunk automatically extracts many useful fields.

© 2026 Innovation In Software Corporation

PART 4 AND PART 5: MORE SEARCHING

Part 4 – Display the First Five Events

Because Splunk normally displays the newest events first, reverse the order.

Enter this search:

```
index=main | reverse | head 5
```

Part 5 – Returns the first 5 rows of the search

Enter this search:

```
index=main| head 5
```

The newest five events from the log will be shown.

© 2026 Innovation In Software Corporation

PART 6 – FIND FAILED WEB PAGES

Most web servers record failed requests using HTTP status codes such as:

- 404 – Page Not Found
- 500 – Internal Server Error
- 403 – Forbidden

Search:

```
index=main status>=400
```

Notice that Splunk immediately returns only the failed requests.

© 2026 Innovation In Software Corporation

PART 7 AND PART 8

Part 7 – Determine Which Pages Fail Most Often

Run the following search:

index=main status>=400 | top uri

This command counts how many times each page failed.

Notice that the most frequently failing pages appear at the top.

Part 8 – Save the Search as a Report

Click *Save As.*

Select Report.

Name the report:

Most Frequently Failing Pages

Click *Save*.

© 2026 Innovation In Software Corporation

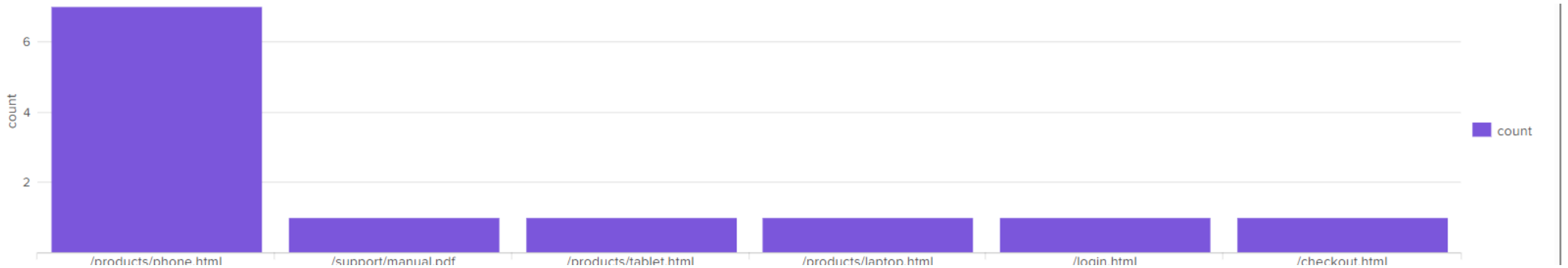
PART 9 – CREATE A CHART

Return to the search:

- Select the Visualization tab.
- Choose Bar Chart.

The chart displays:

- Web pages on the X-axis
- Number of failures on the Y-axis



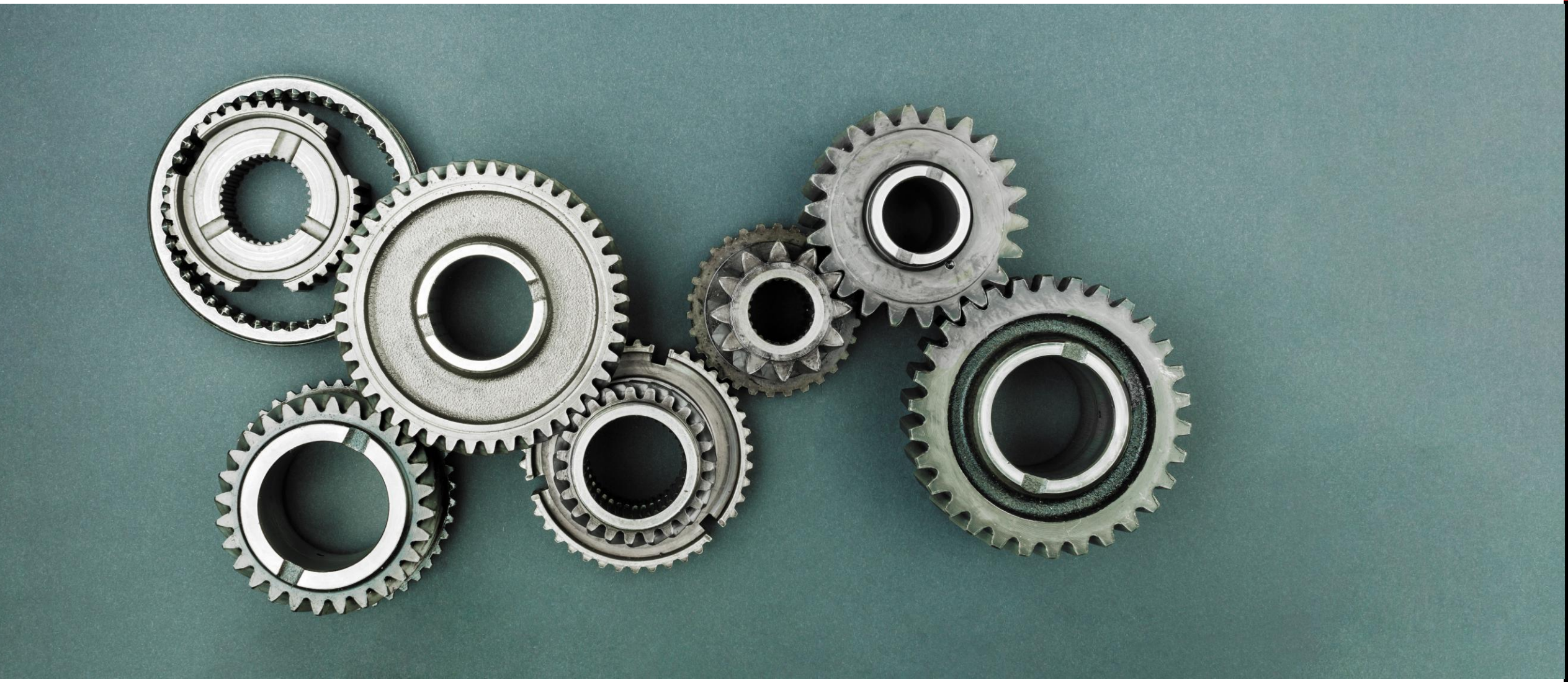
© 2026 Innovation In Software Corporation

Lab completed



TESTING

FASTAPI / TESTCLIENT



INTRODUCTION



A test does more than catch bugs; it stands as proof of a specific promise your API makes, turning a vague claim like "we validate input" into evidence anyone can inspect. Proving an API works also means proving it fails correctly, rejecting bad data and unauthorized requests exactly as documented.

In this module you will use FastAPI's TestClient to run tests in-process, without starting a server, and organize those tests so each one maps to a quality requirement. You will apply repeatable compliance patterns and work through negative testing.

© 2026 Innovation In Software Corporation

TESTING GOALS

Testing is not just about proving your API works; it is about proving which quality requirements it meets. In this module you will write tests that double as compliance evidence, mapping each test to a specific quality tier so an auditor, reviewer, or teammate can trace a requirement directly to the test that verifies it.

We will use FastAPI's TestClient to run tests in-process, organize suites by quality requirement, apply repeatable compliance test patterns, and cover the four core categories of negative testing. By the end, your test suite becomes a living document of what your API guarantees.



QUALITY TIERS

Every tier your API claims to satisfy needs proof. A statement like "the API validates all input" means nothing unless a test demonstrates that invalid input is rejected in a predictable way. Tests turn claims into verifiable, repeatable evidence.

This aligns with Agile and the Definition of Done, where a guarantee must be demoable—shown working as part of the increment—rather than asserted from prediction or theory. When tests are named and organized around requirements, the suite answers the question "how do you know?" for each guarantee. This is the difference between hoping your API is correct and being able to show it.



QUALITY TIERS- 2

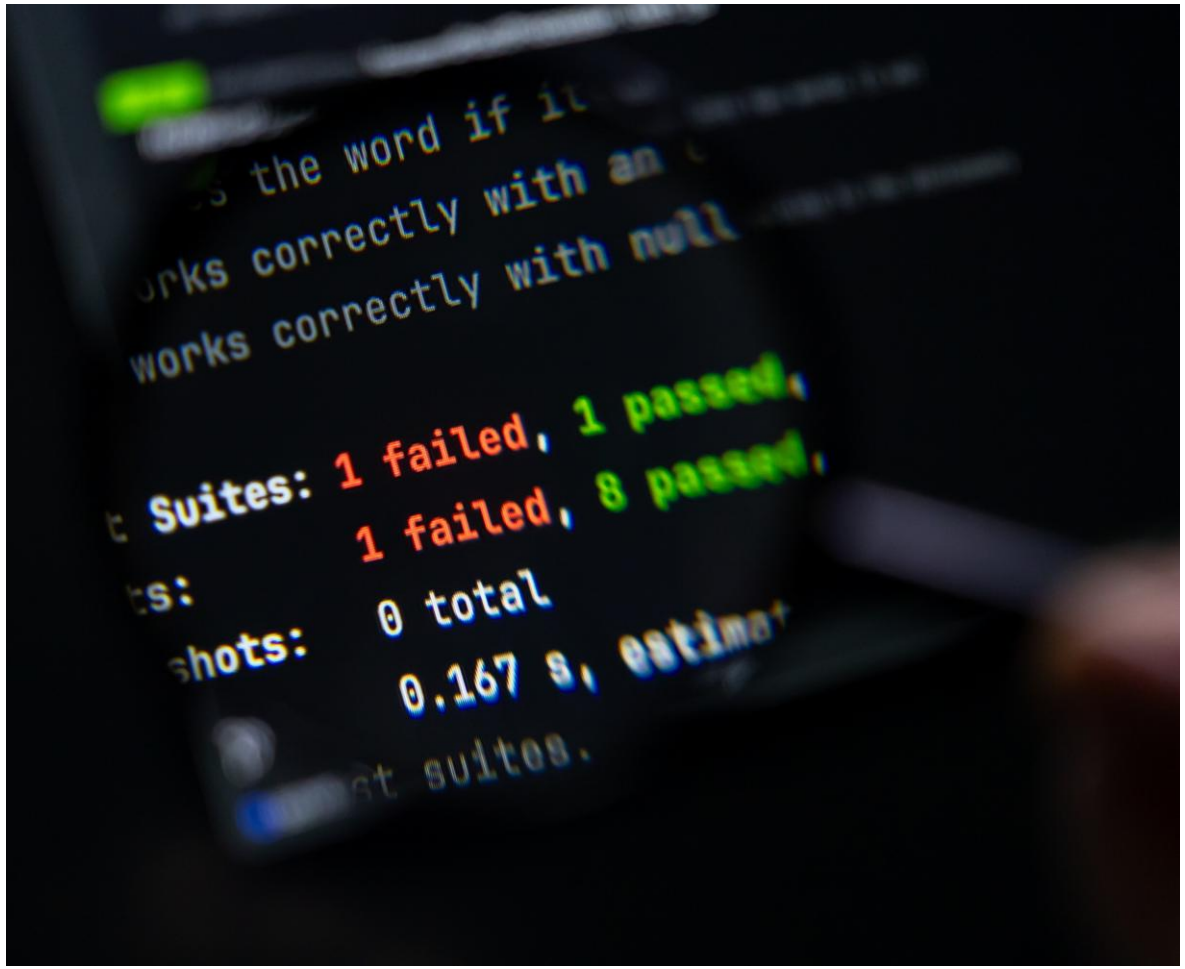
Think of quality tiers as layers of increasing rigor, where each tier is a promise:

- Functional correctness — does it return the right data?
- Input validation — does it reject bad data?
- Error contracts — does it fail in a documented way?
- Security or authorization — does it protect what it should?

Each promise deserves its own tests. Grouping tests by tier makes gaps obvious: if the security tier has no tests, the absence stands out immediately.



TESTCLIENT

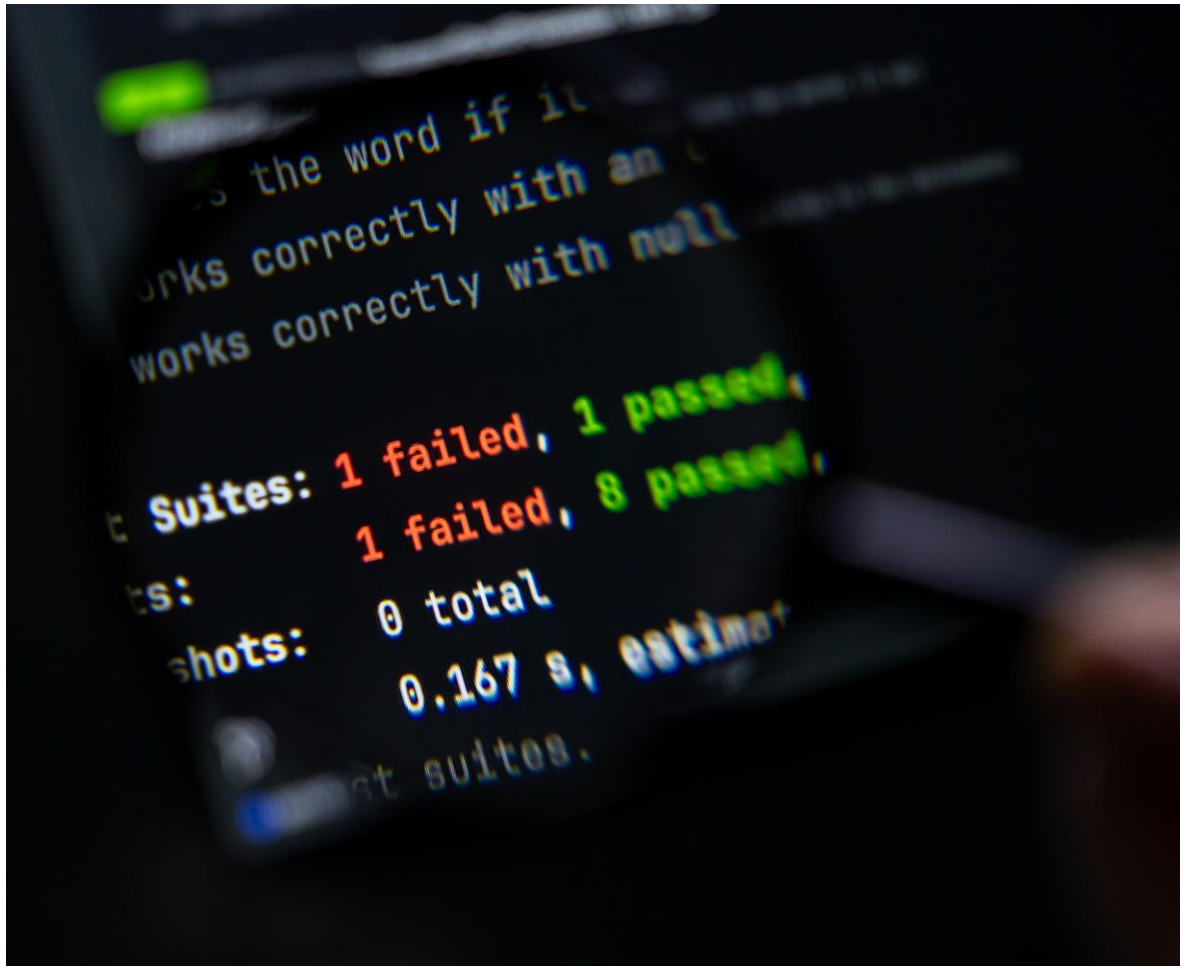


TestClient is FastAPI's built-in testing utility, based on HTTPX and Starlette. It lets you send requests to your application and inspect the responses without ever starting a real server. Everything runs in the same Python process as your test code.

Because it borrows the HTTPX interface, which itself mirrors the popular Requests library, the API feels familiar and intuitive. You send .get(), .post(), and similar calls, then assert against the status code and JSON body.

© 2026 Innovation In Software Corporation

TESTCLIENT - 2



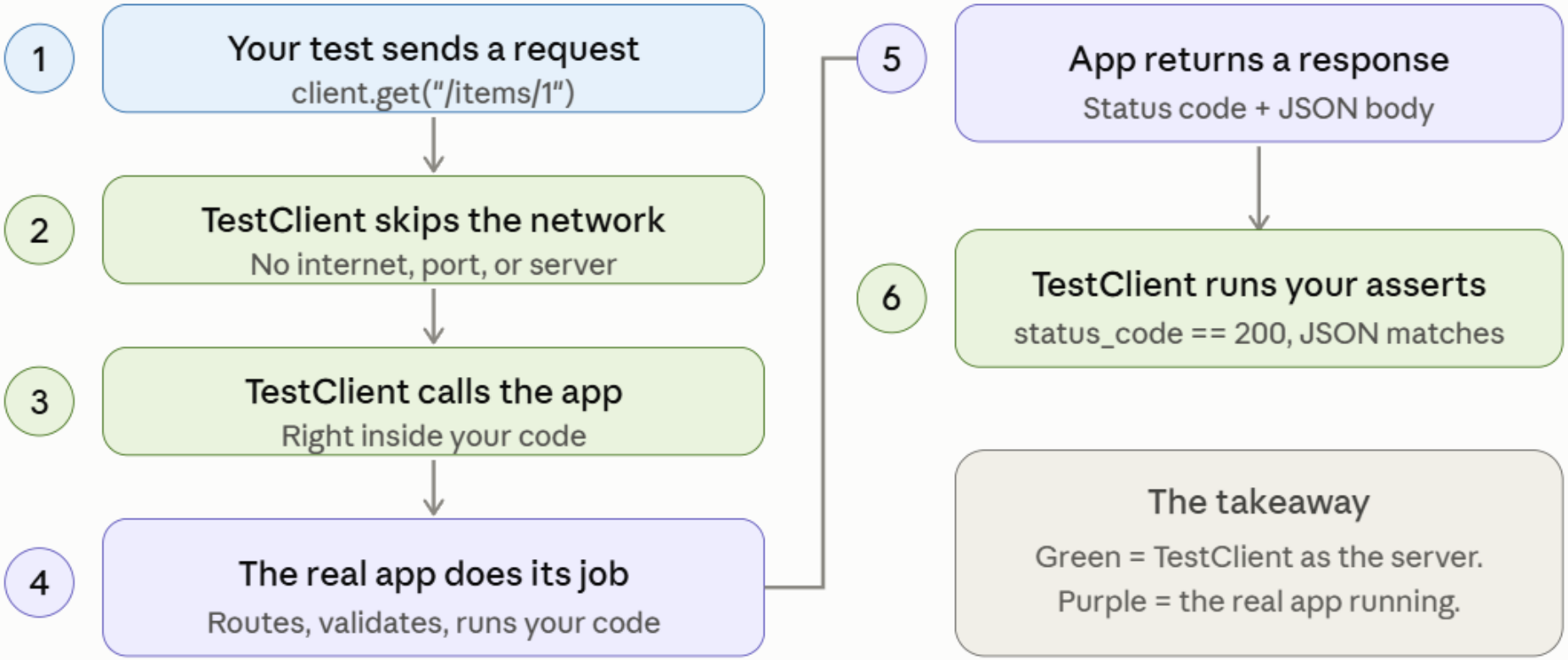
In-process testing means no network, no separate server, and no port binding. Your test creates the client, sends a request, and gets a response through direct function calls under the hood. This makes tests fast, deterministic, and easy to run anywhere, including CI pipelines.

Speed and determinism matter for compliance. A test suite that runs in seconds gets run often, and tests that produce the same result every time can be trusted as evidence. Flaky tests undermine the whole purpose.

© 2026 Innovation In Software Corporation

How TestClient works

It stands in for the server, so the real app runs with no network.



TestClient only replaces the server. Your test exercises the actual routing, validation, and code — just without a network in front of it.

© 202

PYTEST

Pytest is Python's most widely used testing framework. Its appeal is how little ceremony it requires: you write plain functions named test_, use ordinary assert statements to check results, and pytest discovers and runs them automatically. When an assertion fails, it shows exactly what the values were.

Beyond the basics, pytest scales through fixtures, which set up and tear down what a test needs, like a database or a client. It also runs the same test against many inputs and supports plugins and options like -v. That mix of a gentle start and room to grow makes it the default for testing Python, including FastAPI.

© 2026 Innovation In Software Corporation

PYTEST INSTALL



For your Windows/VS Code setup, run this in the VS Code integrated terminal from your project root:

```
pip install -U pytest
```

While testing FastAPI, you'll also want the TestClient dependencies (HTTPX is what it's built on):

```
pip install -U pytest httpx
```

Then verify it installed:

```
pytest --version
```

© 2026 Innovation In Software Corporation

GETTING STARTED

Getting started takes three steps: install HTTPX, import TestClient, and pass your FastAPI application instance to it. From that point you have a client object ready to exercise every endpoint in your app.

```
from fastapi.testclient import TestClient
from main import app


client = TestClient(app)
```

This single client can be reused across many test functions. In larger suites you will typically build it inside a pytest fixture so setup and teardown are handled consistently.

© 2026 Innovation In Software Corporation

TESTING FUNCTION

A basic test sends a request and asserts on both the status code and the response body. Pytest discovers any function whose name starts with test_. The naming matters for both discovery and readability.

```
def test_read_root_returns_ok():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello"}
```

The status code assertion verifies the HTTP contract; the body assertion verifies the data contract. Together they document exactly what a caller can expect from this endpoint.

© 2026 Innovation In Software Corporation

HOW TO RUN TESTS?

From the folder containing your project files, simply type pytest and press Enter — it will automatically find and run every test_ function.

```
PS C:\Code\Production-Ready REST API\Module12\helloapp> pytest -v

test_main.py::test_read_root_returns_ok PASSED
                                                                [100%]

============================================================ 1 passed in 0.48s ====
```


© 2026 Innovation In Software Corporation

HOW TO RUN TEST



Rather than one giant test file, group tests so each file or class maps to a quality tier. A common layout uses separate modules such as test_functional.py, test_validation.py, test_errors.py, and test_security.py. The structure itself communicates coverage.

This organization pays off during review. Anyone can open the folder and see, at a glance, which requirements have tests and which do not. The file tree becomes a checklist of the guarantees your API makes.

© 2026 Innovation In Software Corporation

IMPLEMENT TIERS



Rather than one giant test file, group tests so each file or class maps to a quality tier. A common layout uses separate modules such as test_functional.py, test_validation.py, test_errors.py, and test_security.py. The structure itself communicates coverage.

This organization pays off during review. Anyone can open the folder and see, at a glance, which requirements have tests and which do not. The file tree becomes a checklist of the guarantees your API makes.

© 2026 Innovation In Software Corporation

ARRANGE-ACT-ASSERT

Most compliance tests follow the Arrange-Act-Assert structure. You arrange the inputs and any preconditions, act by sending the request, and assert on the outcome. Keeping these three phases visually distinct makes tests easy to read and easy to trust.

```
def test_create_item_persists_data():
    payload = {"name": "Widget", "price": 9.99}   # Arrange
    response = client.post("/items", json=payload) # Act
    assert response.status_code == 200             # Assert
    assert response.json()["name"] == "Widget"
```

Because testing code serves as documentation, this predictable shape lets any reader follow the intent without reverse-engineering the logic.

© 2026 Innovation In Software Corporation

NEGATIVE TESTS



A requirement is only truly verified when you test both that the API succeeds when it should and fails when it should. Testing only the happy path leaves half the contract unproven. Every meaningful requirement needs at least one positive and one negative test.

For an endpoint that accepts positive integers, you write one test confirming a valid value is accepted and another confirming a zero or negative value is rejected. The pair, taken together, is the real evidence.

© 2026 Innovation In Software Corporation

NEGATIVE – INVALID INPUT

Invalid input tests confirm your API rejects malformed, out-of-range, or wrong-type data. FastAPI's Pydantic models return a 422 status for validation failures automatically, and your tests should assert on that status and the error structure.

```
def test_rejects_non_numeric_price():
    response = client.post("/items", json={"name": "X", "price": "free"})
    assert response.status_code == 422
```

These tests prove your validation tier is real, not just declared in a model somewhere.

© 2026 Innovation In Software Corporation

NEGATIVE – MISSING AND MALFORMED

This category checks what happens when required fields are absent, the JSON body is malformed, or required headers are missing. A robust API responds with a clear error rather than crashing or returning a confusing 500.

Assert that omitting a required field yields a 422 with a message identifying the missing field. These tests document the exact shape of your error contract, which downstream clients depend on to handle failures gracefully.

© 2026 Innovation In Software Corporation

NEGATIVE – AUTHENTICATION / RESOURCE

Authorization negative tests confirm that protected endpoints reject unauthenticated or unauthorized requests, typically with a 401 or 403. Use dependency_overrides to simulate different users and roles without real credentials. Resource-error tests confirm that requesting something that does not exist returns a 404 rather than leaking data or erroring unexpectedly.

```
def test_missing_item_returns_404():
    response = client.get("/items/does-not-exist")
    assert response.status_code == 404
```

Together these categories prove your security and resource-handling tiers behave as promised.

© 2026 Innovation In Software Corporation

RETROSPECTIVE



You have learned to treat tests as compliance evidence, using FastAPI's TestClient to verify each quality tier and worked through the four negative-testing categories that prove your API fails safely.

The takeaway is that a well-structured test suite is documentation of every guarantee your API makes. When tests are grouped around requirements, a passing run becomes a report anyone can trust.

© 2026 Innovation In Software Corporation

LAB 12 – ADD / SUBTRACT



GOAL

The goal of this lab is to give you hands-on experience testing a FastAPI application with the in-process TestClient. You will build a small app with two arithmetic endpoints, then write a test for each one that verifies both the HTTP status code and the returned value, seeing firsthand how a test confirms an endpoint behaves as expected.

By the end you will understand the full testing loop: installing pytest, structuring an app and its test file, and running the suite to confirm every function passes. This foundation prepares you to write the compliance and negative tests covered in the module, where the same TestClient patterns verify each quality tier your API promises to meet.

© 2026 Innovation In Software Corporation

CHANGE DIRECTORY

Open your terminal in VS Code and change into the lab directory.

```
cd labs\lab12
```

© 2026 Innovation In Software Corporation

INSTALL PYTEST

Install pytest along with httpx, which TestClient requires under the hood.

```
pip install pytest httpx
```

You can confirm the install worked by checking the version:

```
pytest --version
```


© 2026 Innovation In Software Corporation

CREATE AP

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/add/{a}/{b}")
def add(a: int, b: int):
    return {"result": a + b}

@app.get("/plus/{a}/{b}")
def plus(a: int, b: int):
    return {"result": a + b}
```

Create a file named main.py with two endpoints, add and subtract. Both take two numbers as path parameters and return their sum, giving you two simple functions to test.

© 2026 Innovation In Software Corporation

CREATE CLIENT TESTS

Create a second file named test_main.py. Import TestClient, pass it your app, and write one test for each endpoint. Each test sends a request and asserts on both the status code and the returned value.

```
from fastapi.testclient import TestClient
from main import app
client = TestClient(app)

def test_add():
    response = client.get("/add/2/3")
    assert response.status_code == 200
    assert response.json() == {"result": 5}
```


© 2026 Innovation In Software Corporation

CREATE CLIENT TESTS - 2

```
def test_subtract():
    response = client.get("/subtract/10/7")
    assert response.status_code == 200
    assert response.json() == {"result": 3}
```

© 2026 Innovation In Software Corporation

RUNNING TESTS

Run pytest from the lab12 folder. It will discover any function starting with test_ and run it.

You should see two passing tests:

```
test_main.py ..                          [100%]

======== 2 passed in 0.15s ========
```


© 2026 Innovation In Software Corporation

BONUS

Now it is your turn. Add two more endpoints to main.py: a multiply function and a divide function, following the same path-parameter pattern used for add and subtract.

Next, add one test function for each of your new endpoints in test_main.py. Each should send a request and assert on both the status code and the returned value.

© 2026 Innovation In Software Corporation

BONUS - 2

Finally, write a negative test for the divide function. A negative test confirms the API fails correctly when given bad input. Division is a natural choice here: dividing by zero should not succeed silently. Send a request that divides by zero and assert that the response is not a normal success.

Run pytest again.

Hint: check for a zero argument

© 2026 Innovation In Software Corporation

Lab completed



ADVANCED PATTERNS

IDEMPOTENCY, RATE LIMIT SIGNALING, AND CACHE-CONTROL



INTRODUCTION



In this module, you will explore the patterns that harden an API for the real world, moving beyond getting it to work toward reliability users can feel. You will learn idempotency and idempotency keys, then handle slow jobs with the asynchronous request-reply pattern.

You will also cover rate-limit signaling with SlowAPI, HTTP caching with Cache-Control, and ETags for optimistic concurrency.

© 2026 Innovation In Software Corporation

ADVANCED PATTERNS

Getting an API to work is the beginning, not the end. The real test comes later: What happens when thousands of people use it at once? When the network is flaky? When two systems fight over the same data? A "mature" API is one that has been hardened against all the ways the real world breaks things.

Think of it like the difference between a car that starts in the driveway and a car you'd trust on a cross-country road trip — both technically run, but only one has been built for the long haul. These five patterns — idempotency keys, async jobs, rate limit signaling, deep caching, and optimistic concurrency — are the upgrades that get you there. None of them are about new features for users to see; they're about reliability users can feel.

© 2026 Innovation In Software Corporation

GOOD ENOUGH

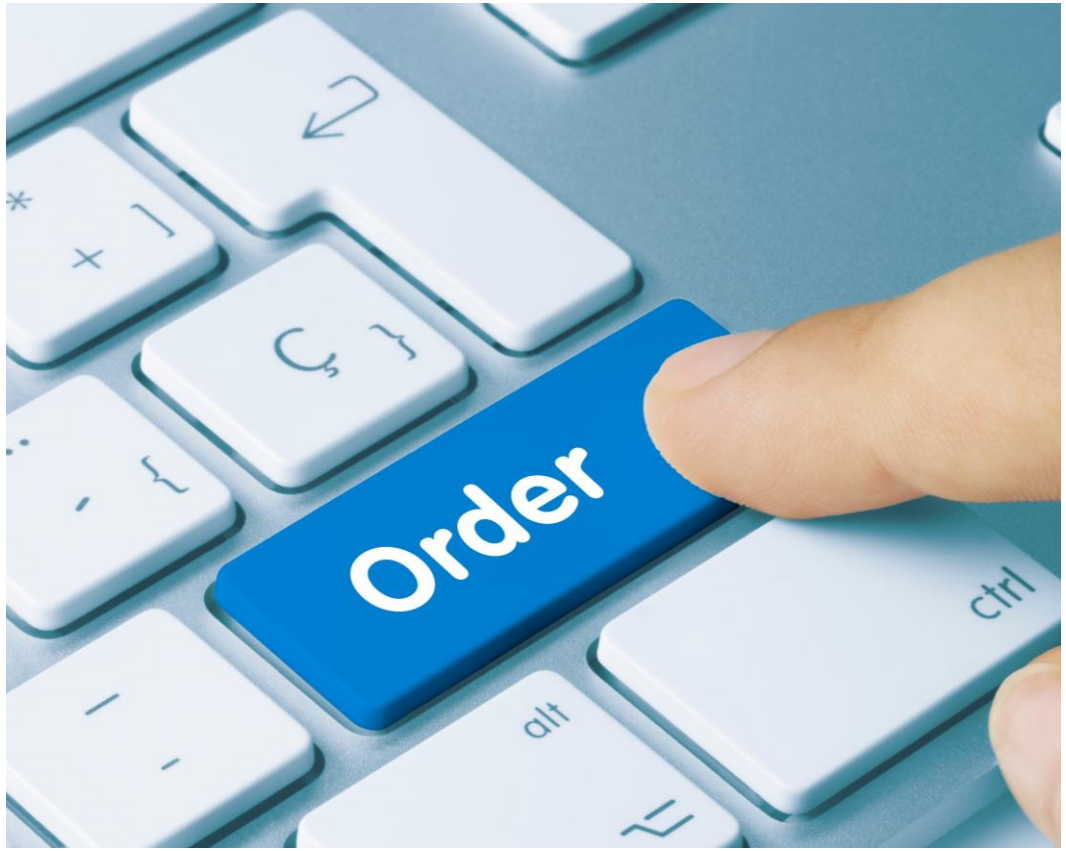
A basic API is built for the "happy path" — the tidy scenario where the request arrives cleanly, the server does its job, and the response comes back. That path is the exception, not the rule. In production, everyday chaos is constant:

- Networks drop mid-request, so the client can't tell if it succeeded
- Impatient users double-click the "Pay" button
- Two employees open the same record and both hit save
- A slow request times out — a four-minute report against a thirty-second connection limit

Every one of these is normal — and a "good enough" API turns each into a bug: a double charge, a lost edit, a frozen screen, a duplicate order. The goal is to absorb that chaos so it never reaches the user as a problem.

© 2026 Innovation In Software Corporation

IDEMPOTENT

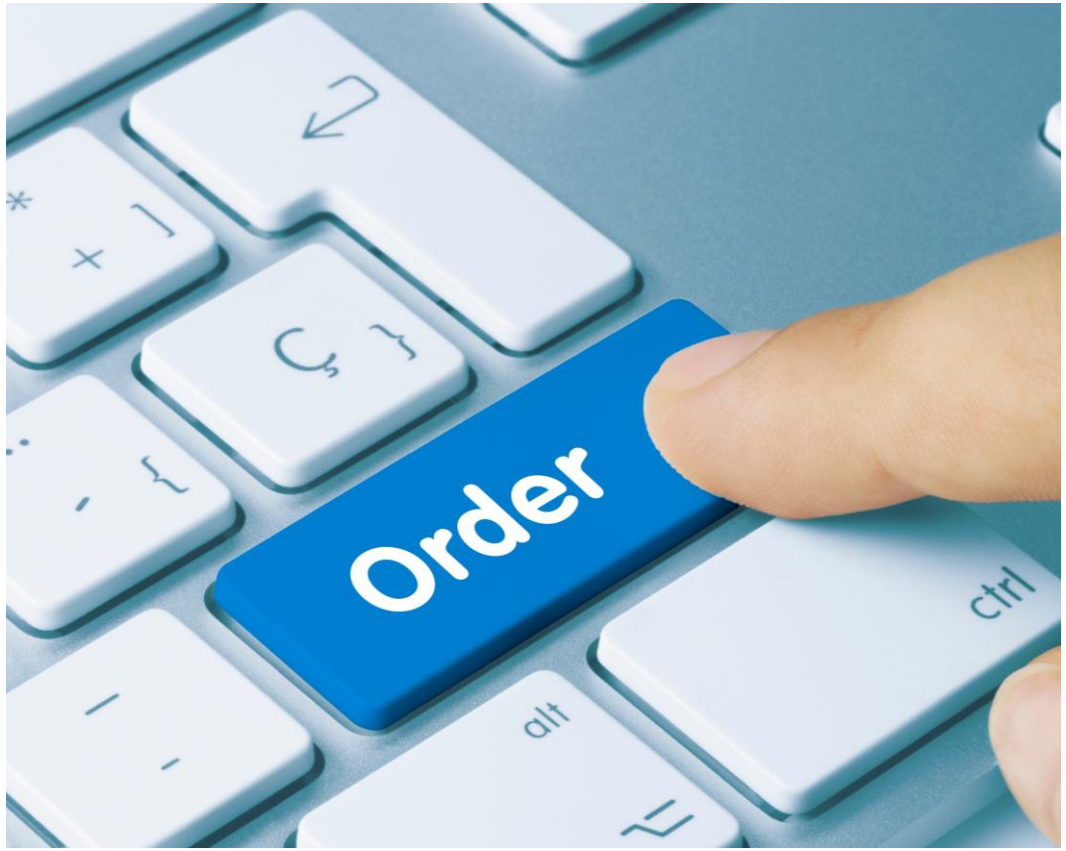


"Idempotent" sounds intimidating, but the idea is simple: an operation is idempotent if doing it twice has the same effect as doing it once. Think of an elevator button — jab it five times, only one elevator comes. Reading a web page works this way too; refreshing doesn't create new pages.

But many important operations *aren't* safe to repeat — charging a card, placing an order, transferring money. Do those twice and you've got a real problem. Idempotency keys make those actions behave like the elevator button: press as many times as you want, it happens exactly once.

© 2026 Innovation In Software Corporation

IDEMPOTENT



Idempotency keys can solve the idempotent problem.

- The setup. Before sending a request, the client (the app) makes up a unique ID — a random string like a1b2c3-payment-9f8e — and attaches it to the request as a label, usually in a header called Idempotency-Key. Think of it as a claim ticket: one specific ticket for this one specific payment.
- The server's job. When the server processes the request, it writes down that ticket number along with what happened ("ticket a1b2c3-payment-9f8e → charged $50, success"). It keeps that record.

© 2026 Innovation In Software Corporation

IDEMPOTENT - EXAMPLE

A customer taps "Pay $50." The charge goes through fine on the server.

- But the confirmation gets lost on the way back — a dropped network connection.
- The app sees no response and assumes it failed, so it automatically retries — sending the *same* ticket number again.
- The server looks at the ticket and recognizes it: "I've already handled this one." Instead of charging a second time, it just hands back the original result — the successful $50 charge.

© 2026 Innovation In Software Corporation

IDEMPOTENT - NOTES

The rule of thumb: if accidentally doing something twice would cost money or create confusion, protect it with a key. Prime candidates include:

- payments
- fund transfers
- order placement
- account creation
- notifications

Keys are not generally required for reads — fetching a customer's profile ten times is harmless, since nothing changes. You also don't strictly need keys for operations already designed to be idempotent, like setting a value to a fixed state.

© 2026 Innovation In Software Corporation

SLOW JOBS

Some jobs are simply too slow to make someone wait on the line while they finish. For example:

- Generating a 10,000-row spreadsheet export
- Transcoding (converting) a video
- Running a big overnight-style batch import
- Building a complex analytics report

Here's the problem if the API tries to do all that before answering:

- The connection times out — most give up after 30–60 seconds
- The user stares at a spinning wheel, unsure whether it's working or frozen
- Even if the job would've finished, the answer arrives too late to reach them

500

© 2026 Innovation In Software Corporation

ASYNCHRONOUS PATTERN

The asynchronous pattern flips this around. Instead of *"wait right here while I finish everything,"* the API says *"Got it — I've started the job. Come back in a moment for the result."*

The everyday analogy is ordering food:

- The slow way (synchronous): you stand at the counter the entire time your meal is cooked, unable to do anything else
- The async way: you place your order, take a buzzer, and go sit down — the kitchen works while you're free to do other things, and the buzzer tells you when it's ready

The payoff is the same in both cases: the slow work still gets done, but nobody is left standing frozen at the counter waiting for it. The user hands off the task, gets a quick "working on it," and checks back for the finished result.

© 2026 Innovation In Software Corporation

ASYNCHRONOUS FLOW

The asynchronous flow happens in three acts. The plain-English version is: hand off the job, check on it now and then, and collect the result when it's ready.

Act 1 — The handoff. The client (the app) sends the request, like "generate this report."

- The server doesn't do the work yet — it just says "Accepted, I'm on it."
- This reply comes back almost instantly, usually with the status code 202 (think of it as "received, working on it")
- Along with that, the server hands over a link to check on the job, like /jobs/12345 — that link is your claim ticket

© 2026 Innovation In Software Corporation

ASYNCHRONOUS FLOW - 2

Act 2 — The check-ins. The client visits that link every so often to ask *"Done yet?"* (This is called polling.)

- Early on, the status says pending (in the queue) or processing (actively being worked on)
- The client keeps checking on a schedule — every few seconds, say — rather than one endless wait
- Fancier systems skip the asking entirely with webhooks: instead of you checking, the server calls *you* the moment it's finished

© 2026 Innovation In Software Corporation

ASYNCHRONOUS FLOW - 3

Act 3 — The result. Once the job is complete, the status response includes the finished result — or a link to download it.

The everyday analogy ties it all together: this is the restaurant buzzer made digital.

- You get the buzzer the instant you order (Act 1 — the 202 and the link)
- You glance at it now and then while you wait (Act 2 — polling)
- It lights up when your food's ready to collect (Act 3 — the result)

© 2026 Innovation In Software Corporation

ASYNCHRONOUS BENEFITS

The async pattern has three benefits:

- Responsiveness — your API answers instantly and never freezes, no matter how heavy the task. The user always gets a quick "I'm on it" instead of a spinning wheel.
- Resilience — because the work runs in the background, a dropped connection doesn't kill it. The client can reconnect and check the status later, and the job keeps chugging along the whole time.
- Scalability — slow tasks don't clog the pipes. Without async, ten big report requests could tie up ten connections for minutes each, starving everyone else. With async, those jobs queue up in the background while the API stays snappy for everybody.

© 2026 Innovation In Software Corporation

RATE LIMIT SIGNALING

Every shared API needs limits, or a single misbehaving client could hammer it and ruin the experience for everyone — the digital equivalent of one person taking all the food at a buffet. For this reason, APIs cap how many requests you can make in a time window.

Rate limit signaling is the courteous part: rather than silently cutting you off when ceiling is hit, the API keeps you informed the whole way — telling you how much capacity is left and when the allowance refills. It's the difference between a gas gauge that shows you the fuel level versus a car that just stalls with no warning.

Signaling lets well-behaved clients pace themselves before they ever hit the wall.

© 2026 Innovation In Software Corporation

RATE LIMIT SIGNALING - 2

Sent on normal responses so a client can ease off *before* hitting the limit. These are widely-used conventions, not formal standards, so exact names can vary between APIs.

- X-RateLimit-Limit — your total allowance for the window; the size of your bucket.
- X-RateLimit-Remaining — how many requests you have left. The value a smart client watches most closely, spacing out requests as it nears zero.
- X-RateLimit-Reset — when the counter refills back to the full limit, given as a timestamp or seconds from now.

© 2026 Innovation In Software Corporation

RATE LIMIT SIGNALING - 3

Sent after:

- Retry-After — sent only after you exceed the limit: the server returns 429 Too Many Requests with Retry-After to say how long to wait, either seconds or an HTTP date.

The clean way to hold all four together: the X-RateLimit-* trio helps clients *avoid* the 429, while Retry-After tells them how to *recover* once they have hit it.

© 2026 Innovation In Software Corporation

RATE LIMIT SIGNALING - STANDARDS

One important caveat for anyone building against these headers: the X-RateLimit-* family is a de facto convention, not an official standard. They were popularized by early APIs like GitHub and Twitter, and so many providers copied them that they became the unofficial norm — but they were never formally defined by the IETF, the body that standardizes HTTP.

Because they're custom, the details vary between providers: some express Reset as a Unix timestamp, others as seconds remaining, and even the exact header names aren't guaranteed. Make sure that your implementation is well documented.

© 2026 Innovation In Software Corporation

SLOWAPI

For rate limiting in FastAPI, the most widely used community library is SlowAPI, a decorator-based wrapper that makes enforcing limits nearly effortless — a single @limiter.limit("5/minute") above an endpoint is all it takes. It's popular precisely because it hides the bookkeeping: it counts requests, tracks time windows, and returns a proper 429 Too Many Requests for you, delegating the underlying rate-limit logic to the well-established limits library.

It's a clean and capable choice, and it's freely available on PyPI under the permissive MIT license.

© 2026 Innovation In Software Corporation

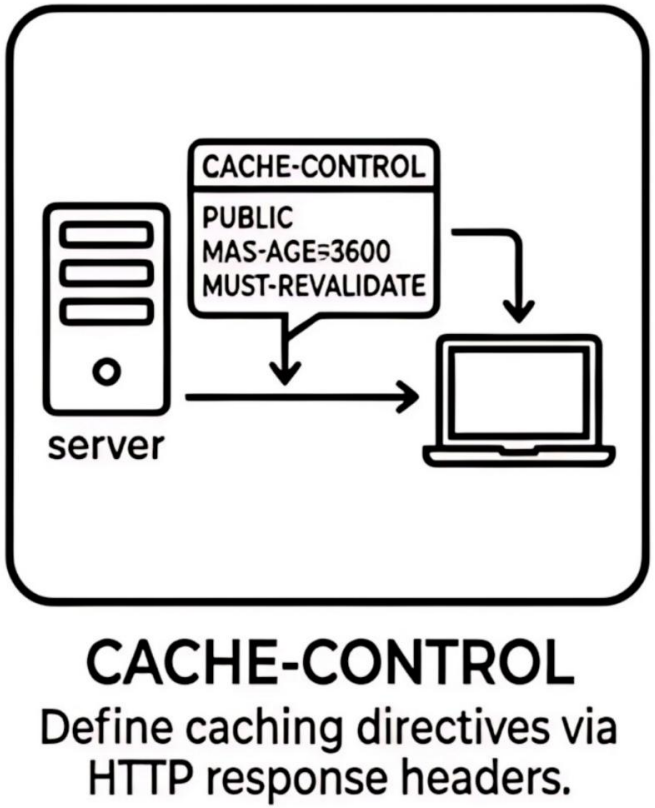
SLOWAPI - 2

SlowAPI is maintained by a single developer rather than a company or foundation, and new releases come out slowly. In a regulated environment like a bank, that matters. When a security issue appears, you need a fix quickly — and a one-person project can't promise that. Long-term support and dependency risk carry real weight here, so this governance profile is worth weighing carefully.

```
pip install slowapi
```


© 2026 Innovation In Software Corporation

CACHE CONTROL



Caching means keeping a saved copy of a response so it can be reused instead of asking the server for it all over again.

First, let's name the two players, because that's where this topic usually gets confusing:

- The server is the source of truth — the system that actually has the real, up-to-date answer (say, a store's official website).
- The client is whoever is requesting — most often your web browser or a phone app. The client is the one that keeps the saved copies.

© 2026 Innovation In Software Corporation

CACHE CONTROL - STORE



You look up a store's opening hours on their website (the server) and jot them on a sticky note by your door. Next time you wonder when they open, you glance at the note instead of looking it up again. It's instant, costs nothing, and works even if your internet is down.

The catch is obvious the moment you picture it: what if the store changed its hours? So the real question every cache has to answer is "is my saved copy still good, or has it gone out of date?"

The Cache-Control header solves this problem.

© 2026 Innovation In Software Corporation

CACHE-CONTROL HEADER



That is the purpose of Cache-Control: a directive the server attaches to each response, specifying the storage rules the client must follow:

- Whether the response may be stored at all. Some data should never be cached — a bank balance, for example.
- How long the stored copy remains valid. Store hours might be trustworthy for a month; a live stock price for only a second.
- Who is permitted to store it. Only the client that made the request, or also shared systems positioned in between.

© 2026 Innovation In Software Corporation

CACHE-CONTROL IMPLEMENTATION

The rule travels in one header the server attaches to its response:

Cache-Control: public, max-age=3600

In plain English: *"Anyone may save this, and the copy stays fresh for 3600 seconds — one hour."*

The directives include:

- public — any cache may store it; private — only the end user's browser may
- no-store — never save this at all (sensitive or always-live data)
- max-age=N — the copy is fresh for N seconds, then must be rechecked
- no-cache — a copy may be kept, but must be re-validated with the server before reuse

© 2026 Innovation In Software Corporation

ETAGS

ETag is a version stamp.

Imagine every piece of data on a server has a little sticker on it with a code, like "v7". Every time the data changes, the server peels off the old sticker and puts on a new one — "v8", "v9", and so on. That sticker is the ETag (short for "entity tag"). Its whole job is to answer one question: *"Is this still the version I last saw?"*

It's not the data itself or format — it's a version stamp of the data. Two pieces of identical data have the same ETag; change one character and the ETag changes.

© 2026 Innovation In Software Corporation

ETAGS - 2

When you request data, the server hands back the current version stamp with it:

```
GET /products/42
→ 200 OK
  ETag: "v7"
  { "name": "Coffee Mug", "price": 12.99 }
```

You hold onto that "v7" and send it back on later requests. Depending on which header you put it in, you get one of two behaviors.

© 2026 Innovation In Software Corporation

ETAGS - 3

Two reasons to use ETags:

- Job 1 — Save bandwidth (caching). Send the stamp in an If-None-Match header, asking "changed since v7?" If not, the server replies 304 Not Modified with no data body, and you reuse the copy you already have. This is how browsers skip re-fetching images you've already loaded.
- Job 2 — Prevent lost edits (concurrency). Send the stamp in an If-Match header when saving: "only apply my change if it's still v7." If someone edited it first (now "v8"), your save is rejected with 412 Precondition Failed — grab the latest and retry. No work gets silently overwritten.

© 2026 Innovation In Software Corporation

RETROSPECTIVE



In this module, you learned idempotency and idempotency keys, and the asynchronous request-reply pattern for handling slow jobs without blocking clients. You saw the full async flow and its benefits.

You also implemented rate-limit signaling with SlowAPI and applied HTTP caching through Cache-Control and ETags for efficient, well-behaved responses.

© 2026 Innovation In Software Corporation

Module completed



Excellent

You have completed the course successfully!



“Develop a passion for learning.”